

目 录

数学	5
基础运算	5
快速冥/乘	5
高精度	6
python高精	12
数学运算	13
sqrt	13
log	13
除法取整	14
分式运算	14
BigInt	16
初等数论	20
约数	20
公约数	21
最大公约数	21
扩展欧几里得	22
最小公倍数	23
分解只因数	23
约数分解	24
筛法求质因数	25
Pollard_Rho	26
约数个数	28
约数之和	29
素数	31
Miller-Rabin	31
素数筛	32
威尔逊定理	35
欧拉函数	35
欧拉筛	36
欧拉定理	36
拓展欧拉定理 (欧拉降幂)	38
模数	41

同余定理	41
线性同余	42
高次同余	43
乘法逆元	44
中国剩余定理	45
数论分块	47
阶&原根	48
狄利克雷卷积	48
莫比乌斯反演	49
杜教筛	52
类欧几里得算法	54
其它	56
组合数学	57
排列组合	57
杨辉三角	58
逆元求组合数	59
卢卡斯定理	60
高精度排列组合	62
错位排列	64
容斥原理	65
数列	66
等差/等比	66
数列递推	67
斐波那契	67
广义斐波那契数列	68
皮萨诺周期	69
卡特兰数	70
k-Dyck路	72
贝尔数 (集合划分)	72
分拆数	73
调和级数	74
康托展开 (全排列排名)	75
范德蒙卷积公式	77
线性代数	77

矩阵	77
矩阵快速幂	78
矩阵封装	81
行列式	84
线性基	86
概率论	88
期望	88
方差	91
多项式	92
快速傅里叶变换	92
快速数论变换	96
快速沃尔什变换	98
数值算法	99
数值积分	99
高斯消元	100
插值	104
博弈论	105
Nim游戏	105
SG函数	106
计算几何	108
距离	109
扫描线	110
面积并	110
周长并	112
二维偏序	115
平面几何	120
点线封装	120
其它	129
中位数	129
对顶堆	129
主席树	132
均值不等式	132
约瑟夫环	132
吉姆拉尔森日期公式	133

二维和一维坐标转化	134
-----------------	-----

数学

基础运算

快速幂/乘

```
1 //求a^b对p取模的值
2 long long qmi(long long a, long long b, long long p) {
3     long long ans = 1;
4     while (b) {
5         if (b & 1) { //如果指数为奇数
6             ans = ans * a % p; //收集好指数为奇数时分离出来的一次方, (不可写为ans*=a%p)
7         }
8         b >>= 1; //指数折半
9         a = a * a % p; //底数变平分
10    }
11    return ans % p;
12 }
```

```
1 //龟速乘
2 //求a*b对p取模的值 性能较差 o(log)
3 long long qmx(long long a, long long b, long long p) {
4     long long ans = 0;
5     while (b) {
6         if (b & 1) ans = (ans + a) % p;
7         b >>= 1;
8         a = (a + a) % p;
9     }
10    return ans;
11 }
12
13 //转为浮点运算 性能最好 o(1)
14 long long qmx(long long a, long long b, long long p) {
15     a %= p; b %= p;
16     long long r = a * b - p * (long long)(1.0L / p * a * b);
17     return r - p * (r >= p) + p * (r < 0);
18 }
19
20 //__int128 o(2) 需要64位GCC编译器
21 long long qmx(long long a, long long b, long long p) {
22     return __int128(a) * b % p;
23 }
```

求 n^k 的前三位数

$$n^k = 10^{k \lg n} = 10^{\lfloor k \lg n \rfloor} \times 10^{k \lg n - \lfloor k \lg n \rfloor}$$

前半部分为整数次幂，后半部分为小数次幂。 $n^k = \text{小数次幂} \times 10^{\text{整数次幂}}$ ，例如 $2^{20} = 1.048576 \times 10^6$
我们只需要取小数次幂的三位(乘100再取整即可)

```
1 //https://vjudge.net/problem/LightOJ-1282
2 int p = pow(10, k * log10(n) - floor(k * log10(n))) * 100;
```

高精度

[大整数在线计算工具\(gptkong.com\)](http://gptkong.com)

加

```
1 // A+B
2 #include<iostream>
3 #include<vector>
4 using namespace std;
5 string a, b;
6 vector<int> A, B;
7 vector<int> add(vector<int>&A, vector<int>&B) {
8     vector<int> C;
9     int t = 0;
10    for (int i = 0; i < A.size() || i < B.size(); i++) {
11        if (i < A.size()) t += A[i];
12        if (i < B.size()) t += B[i];
13        C.push_back(t % 10);    //无论是否有进位，都取 %10的余数
14        t /= 10;    //判断是否有进位
15    }
16    if (t) C.push_back(1);    //如果有最高位还有进位则在C数组最后加元素1
17    return C;
18 }
19 int main() {
20    cin >> a >> b;    //a = 123456
21    for (int i = a.size() - 1; i >= 0; i--) {
22        A.push_back(a[i] - '0');    //A = {6,5,4,3,2,1}
23    }
24    for (int i = b.size() - 1; i >= 0; i--) {
25        B.push_back(b[i] - '0');
26    }
27
28    A = add(A, B);    //auto 进行类型自动转换 在此相当于vector<int>;
29
30    for (int i = A.size() - 1; i >= 0; i--) {
31        printf("%d", A[i]);
32    }
33 }
```

减

```
1 // A-B
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5 string a, b;
6 vector<int>A, B, C; //判断a与b的大小
7 bool cmp(vector<int>& A, vector<int>& B) {
8     if (A.size() != B.size()) return A.size() > B.size(); //先比较位数
9     for (int i = A.size() - 1; i >= 0; i--) { //位数相同则从高位依次比下来
10         if (A[i] != B[i]) return A[i] > B[i];
11     }
12     return 1;
13 }
14
15 vector<int>sub(vector<int>& A, vector<int>& B) { //A >= B
16     vector<int>C;
17     int t = 0;
18     for (int i = 0; i < A.size(); i++) {
19         t += A[i];
20         if (i < B.size()) t -= B[i];
21         C.push_back((t + 10) % 10); //保证相减后取正数
22         if (t < 0) t = -1; //判断是否要借位
23         else t = 0;
24     }
25     while (C.size() > 1 && C.back() == 0) C.pop_back(); //去除前导0, pop_back删除
    容器中最后一个元素
26     return C;
27 }
28
29 int main() {
30     cin >> a >> b;
31     for (int i = a.size() - 1; i >= 0; i--) {
32         A.push_back(a[i] - '0');
33     }
34     for (int i = b.size() - 1; i >= 0; i--) {
35         B.push_back(b[i] - '0');
36     }
37
38     if (cmp(A, B)) A = sub(A, B);
39     else A = sub(B, A);
40
41     for (int i = A.size() - 1; i >= 0; i--) printf("%d", A[i]);
42 }
```

乘

```

 1 2 3 4
*  1 2
-----
   4 8
   3 6
   2 4
1 2
-----
1 4 8 0 8

```

```

1 //A*b   O(N)
2 vector<int> mul(vector<int>& A, int b) {
3     int t = 0;
4     vector<int> C;
5     for (int i = 0; i < A.size() || t; i++) { //注意加上||t;
6         if (i < A.size()) t += A[i] * b;    //将b当做一个整体分别与a的每一位相乘，再加
        上进位
7         C.push_back(t % 10);    //C的每一位取其%10
8         t /= 10;    //计算进位
9     }
10    return C;
11 }

```

```

1 //A*B   O(N*M)
2 vector<int> mul(vector<int> &A, vector<int> &B)
3     vector<int> C(A.size()+B.size());
4     for(int i=0;i<A.size();i++)
5         for(int j=0;j<B.size();j++)
6             C[i+j]+=A[i]*B[j];
7     for(int i=0,t=0;i<res.size();i++){
8         t+=C[i];
9         C[i]=t%10;
10        t/=10;
11    }
12    while(C.size() >= 2 && C.back()==0) C.pop_back();
13    return C;
14 }

```

```

1 //A*B   FFT实现 O(NlogN)
2 //https://www.luogu.com.cn/problem/P1919
3 const long double PI = std::acos(-1.0);
4 void FFT(std::vector<std::complex<long double>>& a, bool invert) {
5     int n = a.size();
6
7     for(int i = 1, j = 0; i < n; i++) {
8         int bit = n >> 1;
9         for (; j & bit; bit >>= 1) j ^= bit;

```

```

10     j ^= bit;
11     if (i < j) std::swap(a[i], a[j]);
12 }
13
14 for(int len = 2; len <= n; len <= 1) {
15     long double ang = 2 * PI / len * (invert ? -1 : 1);
16     std::complex<long double> wlen(cosl(ang), sinl(ang));
17     for (int i = 0; i < n; i += len) {
18         std::complex<long double> w(1.0);
19         for (int j = 0; j < len / 2; j++) {
20             std::complex<long double> u = a[i + j];
21             std::complex<long double> v = a[i + j + len / 2] * w;
22             a[i + j] = u + v;
23             a[i + j + len / 2] = u - v;
24             w *= wlen;
25         }
26     }
27 }
28 if(invert) {
29     for (std::complex<long double>& x : a) x /= n;
30 }
31 }
32
33 std::vector<long long> operator*(const std::vector<long long>&A, const
std::vector<long long>&B){
34     std::vector<long long>C;
35     int n = A.size(), m = B.size();
36
37     int MAX_N = 1;
38     while (MAX_N < n + m) MAX_N <= 1;
39
40     std::vector<std::complex<long double>> a(MAX_N, 0.0), b(MAX_N, 0.0);
41     for(int i = 0; i < n; i++) a[i] = std::complex<long double>(A[i], 0);
42     for(int i = 0; i < m; i++) b[i] = std::complex<long double>(B[i], 0);
43
44     FFT(a, false);
45     FFT(b, false);
46     for(int i = 0; i < MAX_N; i++) a[i] *= b[i];
47     FFT(a, true);
48
49     long long t = 0;
50     for(int i = 0; i < n + m; i++) {
51         long long val = (long long)(a[i].real()+0.5);
52         t += val;
53         C.emplace_back(t%10);
54         t/=10;
55     }
56     if(t) C.emplace_back(t);
57     while(C.size() >= 2 && C.back() == 0) C.pop_back();
58     return C;
59 }

```

除

```

      0 1 0 2
1 2 1 2 3 4
-----
      0
      1 2
      1 2
      ---
       0 3
       0 0
       ---
        3 4
        2 4
        ---
         1 0
```

```
1 //A/b及其余数
2 #include <iostream>
3 #include <vector>
4 #include <algorithm>
5 using namespace std;
6 vector<int>A;//C为商
7
8 vector<int>div(vector<int>&A,int b,int&r){ //r是引用
9     vector<int>C;
10    for (int i = A.size() - 1;i >= 0;i--){ //除法此处倒序，然后再翻转
11        r = r * 10 + A[i]; //上一位的余数*10再加上本位
12        C.push_back(r / b); //将其对b的商记录进C数组
13        r %= b; //然后变为其对b的余数供下一位使用
14    }
15    reverse(C.begin(), C.end());
16    while (C.size() > 1 && C.back() == 0) C.pop_back();
17    return C;
18 }
19
20 int main() {
21     string a; int b, r = 0; //r为余数
22     cin >> a >> b;
23     for (int i = a.size() - 1; i >= 0; i--) {
24         A.push_back(a[i] - '0');
25     }
26
27     A = div(A, b, r);
28
29     for (int i = A.size() - 1;i >= 0;i--){
30         printf("%d", A[i]);
31     }
32     cout << endl << r << endl;
33
34     return 0;
35 }
```

```

1 //a/b 保留k位小数
2 long long a,b,k;cin >> a >> b >> k;
3 cout << a/b << '.';
4 a = a%b*10;
5 while(k--){
6     cout << a/b;
7     a = a%b*10;
8 }

```

```

1 //求a/b的第k位小数 相当于a*10^k/b%10
2 long long a,b,k;cin >> a >> b >> k;
3 cout << a*qmi(10,k-1,b)*10/b%10;

```

模

给两个正整数a,b, 输出他们的最大公约数 $a \leq 1e10^6, b \leq 1e9$

首先有以下性质

1. $(a+b) \% \text{mod}$ 等价于 $a \% \text{mod} + b \% \text{mod}$

2. $a * b \% \text{mod}$ 等价于 $a \% \text{mod} * b \% \text{mod}$ (仅当 $a * b$ 没有溢出时)

该题求解gcd(a,b) a是大数

根据辗转相除法

$\text{gcd}(a,b) = \text{gcd}(b, a \% b)$

因此我们可以先求 $a \% b$ 把a限制在 $1e9$ 的范围内, 然后做gcd

因为a很大, 又可以表示为 $\sum_{i=1}^n a_i * 10^{n-i}$ (其中n为字符串的长度, a_i 为第i个字符)

又由性质1和2, 我们就可以对每个 a_i 求mod, 同时通过乘和累加求出

```

1 //A%b
2 //https://ac.nowcoder.com/acm/contest/86034/D
3 #include <iostream>
4 using namespace std;
5 using ll = long long;
6
7 ll gcd(ll a, ll b){return b?gcd(b,a%b):a;}
8
9 ll qmod(string& a, ll b){//高精度A % 低精度b
10     ll t = 0;
11     for(int i = 0; i < a.size(); i++) {
12         t = (t*10 + a[i] - '0') % b;
13     }
14     return t;
15 }
16
17 int main(){
18     string a;cin >> a;
19     ll b;cin >> b;
20     cout << gcd(b, qmod(a,b));

```

幂

```

1 //中精度  $2^n$   $2*n \leq 30000$  //n可以为负数
2 //仅适用于计算 $2^n$ 的精确值
3 #include <iostream>
4 #include <sstream>
5 #include <iomanip>
6 #include <cmath>
7 using namespace std;
8
9 int main(){
10     int n; cin>>n;
11     stringstream ss;
12     ss << fixed << setprecision(n>0?0:-n) << pow(2.0L,n);
13     //string s = ss.str(); //字符串流,也可以用sprintf
14     string s; ss >> s;
15
16     cout << s;
17 }
```

```

1 //高精度快速幂  $A^b$  一般b取不了太大
2 Bigint qmi(Bigint &a,int b){
3     Bigint ans = 1;
4     while(b){
5         if(b&1) ans = ans*a;
6         b >>= 1;
7         a = a*a;
8     }
9     return ans;
10 }
```

python高精

```

1 import sys
2 sys.set_int_max_str_digits(100005) #修改最大位数 默认值4300
3 a = int(input())
4 b = int(input())
5 print(a+b) #加
6 print(a-b) #减
7 print(a*b) #乘
8 print(a//b) #除
9 print(a%b) #模
10 print(a**b) #幂

```

数学运算

sqrt

```

1 //向下取整
2 long long qsqrt(long long n) {
3     long long s = std::sqrt(n);
4     while (s*s > n) { s--; }
5     while ((s+1)*(s+1) <= n) { s++; }
6     return s;
7 }

```

log

```

1 //向上取整
2 long long logi(long long a, long long b) { //log(a,b)  $a^t \geq b$ 
3     long long t = 0;
4     long long v = 1;
5     while (v < b) {
6         v *= a;
7         t++;
8     }
9     return t;
10 }
11
12 long long llog(long long a, long long b) { //loglog(a,b)  $a^{(a^t)} \geq b$ 
13     if (a <= b) {
14         int l = logi(a, b);
15         return (l == 0 ? 0 : std::__lg(2 * l - 1));
16     }

```

```

17     assert(b != 1);
18     long long l = logi(b, a + 1) - 1;
19     assert(l > 0);
20     return -std::__lg(l);
21 }

```

```

1 //预处理log2, (向下取整)
2 lg2[0] = -1;
3 for(int i = 1; i < N; i++){
4     lg2[i] = lg2[i>>1] + 1;
5 }

```

除法取整

```

1 long long ceilDiv(long long n, long long m) { //上取整
2     if (n >= 0) return (n + m - 1) / m;
3     else return n / m;
4 }
5
6 long long floorDiv(long long n, long long m) { //向下取整
7     if (n >= 0) return n / m;
8     else return (n - m + 1) / m;
9 }

```

分式运算

源自[jiangly分数四则运算 博客园\(cnblogs.com\)](http://jiangly.cnblogs.com)

Frac a(1,3); 表示 $\frac{1}{3}$ ，支持分式之间 + - * / 和比较大小

```

1 template<class T>
2 struct Frac { // num/den
3     T num;
4     T den;
5     Frac(T num_, T den_) : num(num_), den(den_) {
6         if (den < 0) {
7             den = -den;
8             num = -num;
9         }
10    }
11    Frac() : Frac(0, 1) {}
12    Frac(T num_) : Frac(num_, 1) {}
13    explicit operator double() const {
14        return 1. * num / den;
15    }

```

```

16     explicit operator long long() const{
17         return num / den;
18     }
19     friend long long floor(const Frac &x){
20         if(x.num >= 0) return x.num / x.den;
21         else return (x.num - x.den + 1) / x.den;
22     }
23     friend long long ceil(const Frac &x){
24         if(x.num >= 0) return (x.num + x.den - 1) / x.den;
25         else return x.num / x.den;
26     }
27     Frac &operator+=(const Frac &rhs) {
28         num = num * rhs.den + rhs.num * den;
29         den *= rhs.den;
30         return *this;
31     }
32     Frac &operator-=(const Frac &rhs) {
33         num = num * rhs.den - rhs.num * den;
34         den *= rhs.den;
35         return *this;
36     }
37     Frac &operator*=(const Frac &rhs) {
38         num *= rhs.num;
39         den *= rhs.den;
40         return *this;
41     }
42     Frac &operator/=(const Frac &rhs) {
43         num *= rhs.den;
44         den *= rhs.num;
45         if (den < 0) {
46             num = -num;
47             den = -den;
48         }
49         return *this;
50     }
51     friend Frac operator+(Frac lhs, const Frac &rhs) {
52         return lhs += rhs;
53     }
54     friend Frac operator-(Frac lhs, const Frac &rhs) {
55         return lhs -= rhs;
56     }
57     friend Frac operator*(Frac lhs, const Frac &rhs) {
58         return lhs *= rhs;
59     }
60     friend Frac operator/(Frac lhs, const Frac &rhs) {
61         return lhs /= rhs;
62     }
63     friend Frac operator-(const Frac &a) {
64         return Frac(-a.num, a.den);
65     }
66     friend bool operator==(const Frac &lhs, const Frac &rhs) {
67         return lhs.num * rhs.den == rhs.num * lhs.den;

```

```

68     }
69     friend bool operator!=(const Frac &lhs, const Frac &rhs) {
70         return lhs.num * rhs.den != rhs.num * lhs.den;
71     }
72     friend bool operator<(const Frac &lhs, const Frac &rhs) {
73         return lhs.num * rhs.den < rhs.num * lhs.den;
74     }
75     friend bool operator>(const Frac &lhs, const Frac &rhs) {
76         return lhs.num * rhs.den > rhs.num * lhs.den;
77     }
78     friend bool operator<=(const Frac &lhs, const Frac &rhs) {
79         return lhs.num * rhs.den <= rhs.num * lhs.den;
80     }
81     friend bool operator>=(const Frac &lhs, const Frac &rhs) {
82         return lhs.num * rhs.den >= rhs.num * lhs.den;
83     }
84     friend std::ostream &operator << (std::ostream &os, Frac x) {
85         T g = std::gcd(x.num, x.den);
86         if (x.den == g) { return os << x.num / g; } //
87         else { return os << x.num / g << "/" << x.den / g; }
88     }
89 };

```

BigInt

高精度整数运算，**不支持负数运算(待完善)**，写得一坨，勉强能用只能说是==

操作	A op b (高精度op低精度)	A op B (高精度op高精度)
加法	<code>+</code> 、 <code>+=</code>	<code>+</code> 、 <code>+=</code>
减法(仅限大 - 小)	<code>-</code> 、 <code>-=</code>	<code>-</code> 、 <code>-=</code>
乘法	<code>*</code> 、 <code>*=</code>	<code>*</code> 、 <code>*=</code> ($O(N*M)$ 模拟)，不推荐，建议换FFT)
除法(下取整)	<code>/</code> 、 <code>/=</code>	
取模	<code>%</code> 、 <code>%=</code>	
比较大小	<code>></code> <code><</code> <code>==</code> <code>!=</code> <code>>=</code> <code><=</code>	<code>></code> <code><</code> <code>==</code> <code>!=</code> <code>>=</code> <code><=</code>

允许直接A/B、A%B 但需要保证B在整型范围内 ($B \leq 9.2e17$)

初始化

```
1 BigInt A(123);
```

```
1 BigInt A("123");
```

```
1 int n = 123;  
2 BigInt A = n;
```

```
1 string s = 123;  
2 BigInt A = s;
```

```
1 cin >> A;
```

```
1 struct BigInt{  
2     std::vector<long long> a;  
3  
4     BigInt (){ }  
5  
6     BigInt(const std::string &s){  
7         for(int i = s.size()-1; i >= 0; i--){  
8             if(s[i] >= '0' && s[i] <= '9') a.emplace_back(s[i] - '0');  
9         }  
10        while(a.size() >= 2 && a.back() == 0) a.pop_back();  
11    }  
12  
13    BigInt(long long x){  
14        if(x == 0) {a.emplace_back(0); return;}  
15        if(x < 0) x = ~x + 1;  
16        while(x) a.emplace_back(x%10), x/=10;  
17    }  
18  
19    friend long long BigInt_to_int(const BigInt &B){  
20        long long b = 0;  
21        for(int i = B.a.size()-1; i >= 0; i--){  
22            b = b*10 + B.a[i];  
23        }  
24        return b;  
25    }  
26  
27    BigInt operator + (const BigInt &B){  
28        BigInt C;  
29        std::vector<long long>&c = C.a;  
30        const std::vector<long long>&b = B.a;  
31        long long t = 0;  
32        for(int i = 0; i < a.size() || i < b.size() || t; i++){  
33            if(i < a.size()) t += a[i];  
34            if(i < b.size()) t += b[i];  
35            c.emplace_back(t%10);
```

```

36         t/=10;
37     }
38     return C;
39 }
40
41 Bigint operator + (long long b){
42     return *this + Bigint(b);
43 }
44
45 Bigint operator * (long long b){
46     long long t = 0;
47     Bigint C;
48     std::vector<long long>&c = C.a;
49     for(int i = 0; i < a.size() || t; i++){
50         if(i < a.size()) t += a[i]*b;
51         c.emplace_back(t%10);
52         t/=10;
53     }
54     while(c.size() >= 2 && c.back() == 0) c.pop_back();
55     return C;
56 }
57
58 Bigint operator * (const Bigint &B){
59     const auto &A = this->a;
60     Bigint C;
61     C.a = std::vector<long long>(A.size()+B.a.size());
62     for(int i = 0; i < A.size(); i++){
63         for(int j = 0; j < B.a.size(); j++){
64             C.a[i+j] += A[i]*B.a[j];
65         }
66     }
67     long long t = 0;
68     for(int i = 0; i < C.a.size(); i++){
69         t += C.a[i];
70         C.a[i] = t%10;
71         t /= 10;
72     }
73     while(C.a.size() >= 2 && C.a.back() == 0) C.a.pop_back();
74     return C;
75 }
76
77 Bigint operator - (const Bigint &B){
78     Bigint C;
79     std::vector<long long>&c = C.a;
80     const std::vector<long long>&b = B.a;
81     long long t = 0;
82     for(int i = 0; i < a.size(); i++){
83         t += a[i];
84         if(i < b.size()) t -= b[i];
85         c.emplace_back((t+10)%10);
86         if(t < 0) t = -1;
87         else t = 0;

```

```

88     }
89     while(c.size() >= 2 && c.back() == 0) c.pop_back();
90     return C;
91 }
92
93 Bigint operator - (long long b){
94     return *this - Bigint(b);
95 }
96
97 Bigint operator / (long long b){
98     Bigint C;
99     std::vector<long long>&c = C.a;
100    long long t = 0;
101    for(int i = a.size()-1; i >= 0; i--){
102        t = t*10 + a[i];
103        c.emplace_back(t/b);
104        t %= b;
105    } //t as the remainder == A%b
106    reverse(c.begin(), c.end());
107    while(c.size() >= 2 && c.back() == 0) c.pop_back();
108    return C;
109 }
110
111 Bigint operator / (const Bigint &B){
112     return (*this)/Bigint_to_int(B);
113 }
114
115 Bigint operator % (long long b){
116     long long t = 0;
117     for(int i = a.size()-1; i >= 0; i--){
118         t = (t*10 + a[i]) %b;
119     }
120     return Bigint(t);
121 }
122
123 Bigint operator % (const Bigint &B){
124     return (*this)%Bigint_to_int(B);
125 }
126
127 int cmp (const Bigint &B){
128     const std::vector<long long>&b = B.a;
129     if(a.size() != b.size()) {
130         return a.size() > b.size() ? 1 : -1;
131     }
132     for(int i = a.size()-1; i >= 0; i--){
133         if(a[i] != b[i]){
134             return a[i] > b[i] ? 1 : -1;
135         }
136     }
137     return 0;
138 };
139

```

```

140 void operator += (const auto &b){*this = *this + b;}
141 void operator -= (const auto &b){*this = *this - b;}
142 void operator *= (const auto &b){*this = *this * b;}
143 void operator /= (const auto &b){*this = *this / b;}
144 void operator %= (const auto &b){*this = *this % b;}
145 bool operator > (const auto &b){return cmp(b) == 1;}
146 bool operator < (const auto &b){return cmp(b) == -1;}
147 bool operator == (const auto &b){return cmp(b) == 0;}
148 bool operator != (const auto &b){return cmp(b) != 0;}
149 bool operator >= (const auto &b){return cmp(b) != -1;}
150 bool operator <= (const auto &b){return cmp(b) != 1;}
151
152 friend std::ostream &operator << (std::ostream &o,const Bigint &t){
153     for(int i = t.a.size()-1;i >= 0;i--){
154         o << t.a[i];
155     }
156     return o;
157 }
158
159 friend std::istream &operator >> (std::istream &o,Bigint &t){
160     std::string s;o >> s;
161     t = s;
162     return o;
163 }
164 };

```

初等数论

常见符号

1. 整除符号: $x \mid y$, 表示 x 整除 y , 即 x 是 y 的因数。
2. 取模符号: $x \bmod y$, 表示 x 除以 y 得到的余数。
3. 互质符号: $x \perp y$, 表示 x, y 互质。
4. 最大公约数: $\gcd(x, y)$, 在无混淆意义的时候可以写作 (x, y) 。
5. 最小公倍数: $\text{lcm}(x, y)$, 在无混淆意义的时候可以写作 $[x, y]$ 。

约数

平凡约数 (平凡因数) : 对于整数 $b \neq 0$, ± 1 、 $\pm b$ 是 b 的平凡约数。当 $b = \pm 1$ 时, b 只有两个平凡约数。

对于整数 $b \neq 0$, b 的其他约数称为真约数 (真因数、非平凡约数、非平凡因数) 。

公约数

求一个数g的所有约数，或两个数x,y的所有公约数

$f(30) = \{1, 2, 3, 5, 6, 10, 15, 30\}$

$f(30, 5) = \{1, 5\}$

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5  int gcd(int a, int b){return b?gcd(b, a%b):a;}
6  vector<int>v;
7  int main(){
8      int x,y;cin >> x >> y;
9      int g = gcd(x,y);
10
11     //预处理所有公约数，这些公约数一定是最大公约数的约数
12     for (int i = 1; i <= g/i; ++i) {
13         if(g%i == 0) {
14             v.push_back(i);
15             if(i!=g/i) v.push_back(g/i);
16         }
17     }
18     sort(v.begin(),v.end());//排序
19
20     for (int i = 0; i < v.size(); ++i) {
21         cout << v[i] << ' ';
22     }
23
24     return 0;
25 }
```

最大公约数

[AcWing 872. 六种方法求最大公约数及其时间复杂度分析 - AcWing](#)

```
1  //GNU编译器
2  __gcd(a,b); // #include <algorithm> 返回a,b的最大公约数
```

```
1  //二进制优化,快个两三倍
2  int gcd(int a,int b){
3      int az = __builtin_ctz(a),bz = __builtin_ctz(b);//末尾元素0的个数,对于LL类型,需
    要使用__builtin_ctzll
4      if (b == 0) return a;
5      int z = std::min(az,bz);
6      b >>= bz;
7      while(a) {
8          a >>= az;
9          int dif = b-a;
```

```

10     az = __builtin_ctz(dif);
11     if(a < b) b = a;
12     if(dif < 0) a = -dif;
13     else a = dif;
14 }
15 return b << z;
16 }

```

[欧几里得算法][辗转相除法]求最大公约数:

```

1 //a和b的最大公约数是a%b和b的最大公约数
2 int gcd(int a, int b) {return b ? gcd(b,a%b) : a;}

```

相邻的两个数 $\gcd(n, n-1) = 1$

$GCD(a_1, a_2, a_3, \dots, a_n) = GCD(s_1, s_2, s_3, \dots, s_n)$ 其中 $s[i]$ 为 $a[i]$ 的前缀和数组(或者差分数组)

扩展欧几里得

求解形如 $a * x + b * y = c$ 的通解 (有解的充要条件: $c \% \gcd(a, b) == 0$)

裴蜀定理: 对于任意整数 a, b , 一定存在一对整数 x, y , 满足: $ax + by = \gcd(a, b)$

设 $d = \gcd(a, b)$

设 x, y 为当前层的一组解, x_1, y_1 为下一层的一组解

当前层方程为 $ax + by = d$

下一层方程为 $bx_1 + (a \% b) * y_1 = d$

$\implies bx_1 + (a - (a/b) * b) * y_1 = d$

$\implies ay_1 + b(x_1 - a/b * y_1) = d$

联立当前层与下一层, 则可得到当前层的解 x, y , 与下一层方程解 x_1, y_1 的关系

$ax + by = ay_1 + b(x_1 - a/b * y_1)$

下层往上回溯时用下一层的解 x_1, y_1 更新当前层的解 x, y

$x = y_1, y = x_1 - a/b * y_1$

$ax + by = \gcd(a, b)$ 的特解、通解、最小正整数解, x_0, y_0 为 $d = \text{exgcd}(a, b, x, y)$ 的一组解

特解: $x = x_0, y = y_0$, 若 $b \neq 0$, 那么必有 $|x_0| \leq b, |y_0| \leq a$

通解: $x = x_0 + \frac{b}{d} * k, y = y_0 - \frac{a}{d} * k$, 其中 $k \in \mathbb{Z}$, 由此可知 x, y 的增减性相反

x 的最小非负整数解: $x = (x_0 \% |\frac{b}{d}| + |\frac{b}{d}|) \% |\frac{b}{d}|$

$ax + by = c$ 的特解、通解、最小正整数解 (若 $c \% \gcd(a, b) \neq 0$ 则无解) 令 x_0, y_0 为

$d = \text{exgcd}(a, b, x, y)$ 的一组解

特解: $x = x_0 * \frac{c}{d}, y = y_0 * \frac{c}{d}$

通解: $x = x_0 * \frac{c}{d} + \frac{b}{d} * k, y = y_0 * \frac{c}{d} - \frac{a}{d} * k$, 其中 $k \in \mathbb{Z}$

x 的最小非负整数解: $x = x_0 * \frac{c}{d} \% |\frac{b}{d}| + |\frac{b}{d}| \% |\frac{b}{d}|$

```

1  int exgcd(int a,int b,int& x,int& y){//x,y引用传递
2      if(!b){
3          x = 1,y = 0;//当最终b = 0时, x = 1, y = 0 显然是方程的解a*1+0*0 = a
4          return a;
5      }
6      int d = exgcd(b,a%b,x,y);
7      int x1 = x,y1 = y;
8      x = y1;
9      y = x1 - a/b*y1;
10     return d;
11 }

```

类似的, 多元线性丢番图 $a_1x_1 + a_2x_2 + \cdots + a_nx_n = c$ 有整数解, 当且仅当 $d = \gcd(a_1, a_2 \dots a_n)$ 整除 c .

最小公倍数

重要推论: $\gcd(a, b) \times \text{lcm}(a, b) = a \times b$

```

1  long long gcd(long long a, long long b) { return b ? gcd(b, a % b) : a; }
2
3  long long lcm(long long a, long long b) { return a / gcd(a, b) * b; }
4  //先算除法避免数据越界
5
6  #define lcm(a, b) a / gcd(a, b) * b
7  //计算最小公倍数时,可以使用带参数的宏定义,比使用函数略微快一些(省去了函数的调用、返回和传参)

```

分解质因数

每个合数都可以写成几个质数相乘的形式, 其中每个质数都是这个合数的因数

如: $12 = 2^2 * 3^1$

试除法

时间复杂度 $O(\sqrt{N})$

`fac(12) = {(2,2),(3,1)}`

```

1  //https://www.acwing.com/problem/content/description/869/
2  #include <iostream>
3  using namespace std;
4

```

```

5  int main() {
6      int t; cin >> t;
7      while (t--){
8          int n; cin >> n;
9          for (int i = 2; i <= n / i; i++) {
10             if (n % i == 0) { //循环里的i一定是n的素因子
11                 int s = 0;
12                 while (n%i == 0){
13                     n /= i;
14                     s++;
15                 }
16                 cout << i << ' ' << s << endl;
17             }
18         }
19         if (n > 1) cout << n << ' ' << 1 << endl;
20         cout << endl;
21     }
22 }

```

约数分解

求一个数的所有约数

`get_div(12) = {1,2,3,4,6,12}`

```

1  //例题：试除法求约数 https://www.acwing.com/problem/content/871/
2  std::vector<long long> get_div(long long x) {
3      auto v = factorize(x); // p^k
4      std::vector<long long> ans = {1};
5      auto dfs = [&](auto&& self, int u, long long now) -> void {
6          if (u >= v.size()) return;
7          for (int i = u; i < v.size(); i++) {
8              long long w = 1;
9              for (int j = 0; j < v[i].second; j++) {
10                 w *= v[i].first;
11                 ans.emplace_back(now * w);
12                 self(self, i + 1, now * w);
13             }
14         }
15     };
16     dfs(dfs, 0, 1);
17     std::sort(ans.begin(), ans.end());
18     return ans;
19 }

```

筛法求质因数

$O(N)$ 预处理, $O(p(x))$ 查询, $p(x)$ 为 x 的因数个数

primes[] 存1~N的所有素数,0_idx

minp[x] 为x的最小质因子

maxp[x] 为x的最大质因子, 诺 maxp[x] == x 则x为质数

```
1  std::vector<int> primes, minp, maxp;
2  void sieve(int n = 1e6) {
3      minp.resize(n + 1);
4      maxp.resize(n + 1);
5      for (int i = 2; i <= n; i++) {
6          if (!minp[i]) {
7              minp[i] = maxp[i] = i;
8              primes.emplace_back(i);
9          }
10         for (auto &j : primes) {
11             if (j > minp[i] || j > n / i) break;
12             minp[i * j] = j;
13             maxp[i * j] = maxp[i];
14         }
15     }
16 }
17 std::vector<std::pair<int,int>> factorize(int n) { //pair{质因数,次方}
18     std::vector<std::pair<int,int>>ans;
19     while (n > 1) {
20         long long x = minp[n];
21         ans.push_back({x,1});
22         n /= x;
23         while(n % x == 0) {
24             ans.back().second++;
25             n /= x;
26         }
27     }
28     return ans;
29 }
30
31 int main(){
32     sieve(1000000);
33     int x; std::cin >> x;
34     for(auto [p,k]:factorize(x)){
35         std::cout << p << '^' << k << '\n';
36     }
37 }
```

Pollard_Rho

泼辣的肉，期望时间复杂度为 $O(\sqrt{p}) = O(N^{\frac{1}{4}})$

```
1 namespace Pollard_Rho{
2     long long qmi(long long a,long long b,long long p){
3         long long ans = 1;
4         while(b){
5             if(b&1) ans = __int128(ans) * a % p;
6             b>>=1;
7             a = __int128(a) * a % p;
8         }
9         return ans;
10    }
11
12    bool isprime(long long x) { //Miller-Rabin素数判断,时间复杂大概log~log^2
13        if (x < 2 || x % 6 % 4 != 1) return (x|1) == 3;
14        long long s = __builtin_ctzll(x-1), d = x >> s;
15        for (long long a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022})
16        {
17            long long p = qmi(a % x, d, x), i = s;
18            while (p != 1 && p != x - 1 && a % x && i-->0) {
19                p = __int128(p) * p % x;
20            }
21            if (p != x - 1 && i != s) return 0;
22        }
23        return 1;
24    }
25
26    long long gcd(long long a,long long b) {return b ? gcd(b,a%b) : a;}
27
28    long long max_factor;
29    long long Pollard_Rho(long long x) {
30        long long s = 0, t = 0;
31        long long c = (long long)rand() % (x - 1) + 1;
32        long long val = 1;
33        for (int goal = 1;; goal <= 1, s = t, val = 1) { //倍增优化
34            for (int step = 1; step <= goal; ++step) {
35                t = ((__int128)t*t%x + c) % x;
36                val = (__int128)val*std::abs(t-s)%x;
37                if ((step % 127) == 0) {
38                    long long d = gcd(val, x);
39                    if (d > 1) return d;
40                }
41            }
42            long long d = gcd(val, x);
43            if (d > 1) return d;
44        }
45
46        void fac(long long x) {
47            if (x <= max_factor || x < 2) return;
```

```

48     if (isprime(x)) {
49         max_factor = std::max(max_factor, x);
50         return;
51     }
52     long long p = x;
53     while (p >= x) p = Pollard_Rho(x);
54     while ((x % p) == 0) x /= p;
55     fac(x), fac(p);
56 }
57 long long get_maxprime(long long x){//返回x的最大质因子
58     max_factor = 0;
59     fac(x);
60     return max_factor;
61 }
62
63 std::vector<std::pair<long long,int>> factorize(long long x){//返回x的质因子
vec{prime,k次方}
64     std::vector<std::pair<long long,int>> ans;
65     while(x > 1) {
66         long long now = get_maxprime(x);
67         if(ans.empty() || now != ans.back().first) ans.push_back({now,1});
68         else ans.back().second++;
69         x /= now;
70     }
71     std::reverse(ans.begin(),ans.end());
72     return ans;
73 }
74
75 std::vector<long long> get_div(long long x) { //约数分解:get_div(30)=
{1,2,3,5,6,10,15,30}
76     auto v = factorize(x); // p^k
77     std::vector<long long> ans = {1};
78     auto dfs = [&](auto&& self, int u, long long now) -> void {
79         if (u >= v.size()) return;
80         for (int i = u; i < v.size(); i++) {
81             long long w = 1;
82             for (int j = 0; j < v[i].second; j++) {
83                 w *= v[i].first;
84                 ans.emplace_back(now * w);
85                 self(self, i + 1, now * w);
86             }
87         }
88     };
89     dfs(dfs, 0, 1);
90     std::sort(ans.begin(), ans.end());
91     return ans;
92 }
93 };
94 using
Pollard_Rho::isprime,Pollard_Rho::get_maxprime,Pollard_Rho::factorize,PollardRh
o::get_div;
95

```

```

96 int main(){//使用方法示例
97     long long x; std::cin >> x;
98     std::cout << isprime(x) << '\n';//素数判断
99     std::cout << get_maxprime(x) << '\n';//最大质因子
100     for(auto &[p,k]:factorize(x)){//分解质因数(从小到大排序)
101         std::cout << p << '^' << k << '\n';
102     }
103 }

```

[阶乘 \(n!\) 的素因数分解 正整数 \(sohu.com\)](#)

详见高精度排列组合

约数个数

如果 $N = p_1^{c_1} * p_2^{c_2} * \dots * p_k^{c_k}$

约数个数 = $(c_1 + 1) * (c_2 + 1) * \dots * (c_k + 1) = \prod_{i=1}^k (c_i + 1)$

约数之和 = $(p_1^0 + p_1^1 + \dots + p_1^{c_1}) * \dots * (p_k^0 + p_k^1 + \dots + p_k^{c_k}) = \prod_{i=1}^k (\sum_{j=0}^{c_i} p_i^j)$

```

1 //https://www.acwing.com/problem/content/872/
2 //给定 n 个正整数 a[], 请你输出这些数的乘积的约数个数, 答案对 1e9+7 取模。
3 #include <iostream>
4 #include <unordered_map>
5 using namespace std;
6 const int mod = 1e9 + 7;
7 unordered_map<int, int>primes;
8
9 int main() {
10     int t; cin >> t;
11     while (t--){
12         int n; cin >> n;
13         for (int i = 2; i <= n / i; i++) {
14             while (n % i == 0) {
15                 n /= i;
16                 primes[i]++;
17             }
18         }
19         if (n > 1) primes[n]++;
20     }
21
22     long long ans = 1;
23     for (auto& i : primes) {
24         int a = i.second;
25         ans = ans * (a + 1) % mod;
26     }
27     cout << ans << endl;
28 }

```

$O(N \log N)$ 预处理求约数个数, $O(1)$ 查询

```
1  const int N = 200005;
2  int f[N];
3  void init() {
4      for (int i = 1; i < N; i++) {
5          for (int j = i; j < N; j += i) { // 枚举i的倍数
6              f[j]++; // i是j的一个约数
7          }
8      }
9  }
```

约数之和

如果 $N = p_1^{c_1} * p_2^{c_2} * \dots * p_k^{c_k}$

约数个数 $= (c_1 + 1) * (c_2 + 1) * \dots * (c_k + 1) = \prod_{i=1}^k (c_i + 1)$

约数之和 $= (p_1^0 + p_1^1 + \dots + p_1^{c_1}) * \dots * (p_k^0 + p_k^1 + \dots + p_k^{c_k}) = \prod_{i=1}^k (\sum_{j=0}^{c_i} p_i^j)$

求 a^b 约数之和 % mod, $0 \leq a, b \leq 5e7$

实现一个 sum 函数, $\text{sum}(p, k)$ 表示 $p^0 + p^1 + \dots + p^{k-1}$

方法一 $O(k)$: 递推求 $p^0 + p^1 + \dots + p^k$

```
1  ll sum0(ll p, ll k) {
2      ll res = 1;
3      for (int i = 1; i <= k; i++) {
4          res = (res * p + 1) % mod;
5      }
6      return res;
7  }
```

方法二 $O(\log K)$: 递归求

k 为偶数时 $\text{sum}(p, k) \Rightarrow p^0 + p^1 + \dots + p^{\frac{k}{2}-1} + p^{\frac{k}{2}} + p^{\frac{k}{2}+1} + \dots + p^{k-1}$

$\Rightarrow p^0 + p^1 + \dots + p^{\frac{k}{2}-1} + p^{\frac{k}{2}} * (p^0 + p^1 + \dots + p^{\frac{k}{2}-1})$

$\Rightarrow \text{sum}(p, \frac{k}{2}) + p^{\frac{k}{2}} * \text{sum}(p, \frac{k}{2})$

$\Rightarrow (p^{\frac{k}{2}} + 1) * \text{sum}(p, \frac{k}{2})$

当 k 为奇数时, 为了方便调用我们写的偶数项情况, 可以单独拿出最后一项, 把剩下的项转化为求偶数项的情况来考虑, 再加上最后一项, 就是奇数项的情况了, 也即 $\text{sum}(p, k-1) + p^{k-1}$

```
1  ll sum1(ll p, ll k) { // sum1(p, k+1), 调用时k要加1
2      if (k == 1) return 1; // 边界条件
3      if (! (k & 1)) return (qmi(p, k/2 + 1) * sum1(p, k/2)) % mod; // k为偶数
4      return (qmi(p, k-1) + sum1(p, k-1)) % mod; // k为奇数, k-1为偶数
5  }
```

方法三(OlogK): 等比求和公式

$$sum = p^0 + p^1 + p^2 + \dots + p^k \quad \text{-----①}$$

$$sum * p = p^1 + p^2 + p^3 + \dots + p^{k+1} \quad \text{-----②}$$

②-①化简得 $sum = \frac{p^{k+1}-1}{p-1}$, 利用快速幂求逆元求解

```
1  ll sum2(ll p,ll k){
2      if((p-1)%mod == 0) return k+1; //p-1与mod不互质,逆元不存在
3      return (qmi(p,k+1)-1)%mod*qmi(p-1,mod-2)%mod;
4  }
```

```
1 //https://www.acwing.com/problem/content/99/
2 #include <iostream>
3 #include <map>
4 using namespace std;
5 using ll = long long;
6 const int mod = 9901;
7
8 ll qmi(ll a,ll b){
9     ll ans = 1;
10    while(b){
11        if(b&1) ans = ans*a%mod;
12        b >>= 1;
13        a = a*a%mod;
14    }
15    return ans%mod;
16 }
17
18 ll sum0(ll p,ll k){//递推求p^0 + p^1 + ... + p^k
19     ll res = 1;
20     for(int i = 1;i <= k;i++){
21         res = (res*p+1)%mod;
22     }
23     return res;
24 }
25
26 ll sum1(ll p,ll k){//递归求p^0 + p^1 + ... + p^(k-1)
27     if(k == 1) return 1;//边界条件
28     if(!(k&1)) return (qmi(p,k/2+1)*sum1(p,k/2)%mod);//k为偶数
29     return (qmi(p,k-1) + sum1(p,k-1))%mod;//k为奇数, k-1为偶数
30 }
31
32 ll sum2(ll p,ll k){//等比求和公式[p^(n+1)-a^0]/(p-1)
33     if((p-1)%mod == 0) return k+1; //p-1与mod不互质,逆元不存在
34     return (qmi(p,k+1)-1)%mod*qmi(p-1,mod-2)%mod;
35 }
36
37 int main(){
38     ll a,b;cin >> a >> b;
39     if(a == 0) return cout << 0,0;
40     map<ll,ll>mp;
```

```

41     for(int i = 2; i <= a/i; i++){
42         while(a%i == 0){
43             mp[i] += b;
44             a /= i;
45         }
46     }
47     if(a > 1) mp[a] += b;
48
49     ll ans1 = 1;
50     for(auto &[p,k]:mp){
51         ans1 = ans1 * sum1(p, k+1) % mod; // ans *= p^(0~k), sum为(0~k-1), 所以k要加1
52         // ans2 = ans2 * sum2(p, k) % mod;
53     }
54     cout << ans1;
55     // cout << (ans2 % mod + mod) % mod;
56 }

```

素数

[素数的判断方法总结](#)

试除法

时间复杂度 $O(\sqrt{x})$

```

1 bool isPrime(int n){
2     if (n <= 1 || n == 4) return 0;
3     if (n == 2 || n == 3) return 1;
4     if (n % 6 != 1 && n % 6 != 5) return 0;
5     for (int i = 5; i <= n/i; i += 6) {
6         if (n % i == 0 || n % (i + 2) == 0) return 0;
7     }
8     return 1;
9 }

```

Miller-Rabin

快速判断一个数 x 是不是素数，时间复杂度大概为 $\log(x) \sim \log^2(x)$

```

1 long long qmi(long long a, long long b, long long p){
2     long long ans = 1;
3     while(b){
4         if(b&1) ans = __int128(ans) * a % p;
5         b >>= 1;
6         a = __int128(a) * a % p;
7     }

```

```

8     return ans;
9 }
10
11 bool isprime(long long x) {
12     if (x < 2 || x % 6 % 4 != 1) return (x|1) == 3;
13     long long s = __builtin_ctzll(x-1), d = x >> s;
14     for (long long a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
15         long long p = qmi(a % x, d, x), i = s;
16         while (p != 1 && p != x - 1 && a % x && i--) {
17             p = __int128(p) * p % x;
18         }
19         if (p != x - 1 && i != s) return 0;
20     }
21     return 1;
22 }

```

素数筛

$O(N)$ 预处理, $O(1)$ 查询

```

1 //线性筛 https://www.luogu.com.cn/problem/P3912
2 #include <iostream>
3 using namespace std;
4 const int N = 1e8 + 5;
5 bool st[N]; // i >= 2 且 st[i] == 0 则 i 是素数
6 int primes[N], cnt; // primes 存质数
7
8 void init(int n) {
9     // st[0] = st[1] = 1;
10    for (int i = 2; i <= n; i++) {
11        if (!st[i]) primes[++cnt] = i; // 如果没被筛过, 则 i 是素数
12        for (int j = 1; primes[j] <= n/i; j++) {
13            st[i*primes[j]] = 1;
14            if (i%primes[j] == 0) break;
15        }
16    }
17 }
18
19 int main() {
20     int n; cin >> n;
21     init(n);
22     cout << cnt;
23 }

```

求 $[l, r]$ 之间的所有质数 $1 \leq l \leq r < 2^{31}$ $r - l \leq 1e6$

```

1 // https://vjudge.net/problem/LightOJ-1197

```

```

2 //用数组p存储√r以内的所有质数
3 //再用p筛选出l~r之间的所有合数，剩下的即为质数
4 #include <iostream>
5
6 const int N = 100005;
7 bool vis[N];
8 int primes[N],cnt;
9 void get_primes(int n){
10     for(int i = 2;i <= n;i++){
11         if(!vis[i]) primes[++cnt] = i;
12         for(int j = 1;i*primes[j] <= n;j++){
13             vis[i*primes[j]] = 1;
14             if(i%primes[j] == 0) break;
15         }
16     }
17 }
18
19 bool st[N];
20 void sol(){
21     long long l,r; std::cin >> l >> r;
22     for(int i = 0;i < r-l+1;i++) st[i] = 0;
23
24     for(int i = 1;(long long)primes[i]*primes[i] <= r;i++){
25         long long p = primes[i]; //p[i]的倍数即为合数
26         for(long long j = std::max(p+p,p*((l+p-1)/p)); j <= r;j += p){
27             st[j-l] = 1; //j-l为数j相对于l的偏移位置
28         }
29     }
30     int ans = 0;
31     for(int i = 0;i < r-l+1;i++){
32         if(!st[i] && l+i > 1) { //特判1
33             ans++;
34         }
35     }
36     std::cout << ans << '\n';
37 }
38
39 int main(){
40     get_primes(N-1);
41     int t; std::cin >> t;
42     for(int i = 1;i <= t;i++){
43         printf("Case %d: ",i);
44         sol();
45     }
46 }

```

Min25筛

求区间 $[1, N]$ 内素数个数，，其中 $2 \leq N \leq 10^{11}$

时间复杂度 $O(\frac{n^{\frac{3}{4}}}{\log n})$

```
1 //https://loj.ac/p/6235
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 using ll = long long;
6 const int N = 8e5 + 5;
7 ll n;
8
9 int id1[N], id2[N], m;
10 int primes[N], vis[N], cnt;
11 ll f[N], v[N];
12
13 inline int get(ll x) { return x < N ? id1[x] : id2[n/x]; }
14
15 inline void init(int n) {
16     for (int i = 2; i <= n; i++) {
17         if (!vis[i]) primes[++cnt] = i;
18         for (int j = 1; i * primes[j] <= n; j++) {
19             vis[i * primes[j]] = 1;
20             if (i % primes[j] == 0) break;
21         }
22     }
23 }
24
25 int main() {
26     init(N - 1);
27
28     scanf("%lld", &n);
29
30     for (ll l = 1, r; l <= n; l = r + 1) {
31         long long t = n / l;
32         r = n / t;
33         v[++m] = t;
34         if (v[m] < N) id1[t] = m;
35         else id2[n / t] = m;
36         f[m] = (t + 1) / 2;
37     }
38     for (int j = 2; j <= cnt; j++) {
39         for (int i = 1; i <= m && 1LL * primes[j] * primes[j] <= v[i]; i++) {
40             f[i] -= f[get(v[i]/primes[j])] - f[get(primes[j-1])];
41         }
42     }
43
44     printf("%lld\n", f[get(n)]);
45 }
```

威尔逊定理

对于素数 $p > 1$, $(p-1)! \equiv -1 \pmod{p}$ 是 p 为素数的充分必要条件。

$$(p-1)! \equiv p-1 \pmod{p} \equiv -1 \pmod{p}$$

欧拉函数

$\varphi(N)$ 表示 $1 \sim N$ 中与 N 互质的数的个数

积性函数, 对于 $\gcd(a, b) = 1$ 的整数有 $\varphi(ab) = \varphi(a)\varphi(b)$

特别地当 n 是奇数时有 $\varphi(2n) = \varphi(n)$

诺将 N 分解质因数: $N = p_1^{a_1} p_2^{a_2} p_3^{a_3} \cdots p_k^{a_k}$
则欧拉函数计算公式 $\phi(N) = N \cdot \frac{p_1-1}{p_1} \cdot \frac{p_2-1}{p_2} \cdot \frac{p_3-1}{p_3} \cdots \frac{p_k-1}{p_k}$

```
1 //求一个数的欧拉函数    √N
2 //根据计算公式,在分解质因数时顺便求欧拉函数
3 //https://www.acwing.com/problem/content/875/
4 #include <iostream>
5 using namespace std;
6
7 int phi(int n){
8     int ans = n;
9     for(int i = 2; i <= n/i; i++){
10         if(n%i == 0){
11             ans = ans/i*(i-1);
12             while(n % i == 0){ n /= i; }
13         }
14     }
15     if(n >= 2) ans = ans/n*(n-1);
16     return ans;
17 }
18
19 int main(){
20     int tt; cin >> tt;
21     while(tt--){
22         int x; cin >> x;
23         cout << phi(x) << endl;
24     }
25 }
```

欧拉筛

```
1 //筛选法求欧拉函数 O(N)
2 //在筛质数时顺便求欧拉函数
3 //https://www.acwing.com/problem/content/876/
4 #include <iostream>
5 using namespace std;
6 const int N = 1000006;
7 int n;
8 bool st[N];
9 int primes[N], cnt;
10 int phi[N];
11
12 void get_phi(int n){
13     phi[1] = 1;
14     for(int i = 2; i <= n; i++){
15         if(!st[i]) {
16             primes[++cnt] = i;
17             phi[i] = i-1; //若i为质数, 则phi[i] = i-1
18         }
19         for(int j = 1; primes[j] <= n/i; j++){
20             st[i*primes[j]] = 1;
21             if(i%primes[j] == 0) {
22                 phi[i*primes[j]] = phi[i]*primes[j];
23                 break;
24             }
25             else phi[i*primes[j]] = phi[i]*(primes[j]-1);
26         }
27     }
28 }
29
30 int main(){
31     cin >> n;
32     get_phi(n);
33 }
```

欧拉定理

若正整数 a 与 n 互质, 则 $a^{\phi(n)} \equiv 1 \pmod{n}$

欧拉定理推论:

若正整数 a 与 n 互质, 对于任意正整数 b , 有 $a^b \equiv a^{b\% \phi(n)} \pmod{n}$

面对乘方算式对质数 p 取模, 可以先把底数对 p 取模、指数对 $\phi(p)$ 取模再计算乘方

若正整数 a 与 n 互质, 则满足 $a^x \equiv 1 \pmod{n}$ 的最小正整数 x_0 是 $\phi(n)$ 的约数。

$$\sum_{d|n} \phi(d) = n \quad (n \text{ 为正整数})$$

```

1  for(int i = 1; i <= n; i++){
2      if(n%i == 0){ ans += phi[i]; }
3  } //ans == n

```

求 $\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j)$

在结论 $n = \sum_{d|n} \varphi(d)$ 中代入 $n = \gcd(a, b)$, 则有
 $\gcd(a, b) = \sum_{d|\gcd(a, b)} \varphi(d) = \sum_d [d|a][d|b] \varphi(d)$

其中, $[\cdot]$ 称为 [Iverson 括号](#), 只有当命题 P 为真时 $[P]$ 取值为 1, 否则取 0。对上式求和, 就可以得到

$$\sum_{i=1}^n \gcd(i, n) = \sum_d \sum_{i=1}^n [d|i][d|n] \varphi(d) = \sum_d \left\lfloor \frac{n}{d} \right\rfloor [d|n] \varphi(d) = \sum_{d|n} \left\lfloor \frac{n}{d} \right\rfloor \varphi(d).$$

这里关键的观察是 $\sum_{i=1}^n [d|i] = \left\lfloor \frac{n}{d} \right\rfloor$, 即在 1 和 n 之间能够被 d 整除的 i 的个数是 $\left\lfloor \frac{n}{d} \right\rfloor$ 。

利用这个式子, 就可以遍历约数求和了。需要多组查询的时候, 可以预处理欧拉函数的前缀和, 利用数论分块查询。

仿照上文的推导, 可以得出

$$\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) = \sum_{d=1}^n \left\lfloor \frac{n}{d} \right\rfloor^2 \varphi(d).$$

```

1  //https://www.luogu.com.cn/problem/P2398
2  //O(N) 预处理 O(sqrt(N)) 询问
3  #include <iostream>
4  using namespace std;
5  using ll = long long;
6  const int N = 100005;
7  int primes[N], cnt;
8  int phi[N];
9  ll sum[N];
10 bool st[N];
11
12 void get_phi(int n){
13     phi[1] = 1;
14     for(int i = 2; i <= n; i++){
15         if(!st[i]){
16             primes[++cnt] = i;
17             phi[i] = i-1;
18         }
19         for(int j = 1; i*primes[j] <= n; j++){
20             st[i*primes[j]] = 1;
21             if(i%primes[j] == 0){
22                 phi[primes[j]*i] = phi[i]*primes[j];
23                 break;
24             }
25             else phi[primes[j]*i] = phi[i]*(primes[j]-1);

```

```

26     }
27 }
28 for(int i = 1; i <= n; i++) {
29     sum[i] = sum[i-1] + phi[i];
30 }
31 }
32
33 int main(){
34     int n; cin >> n;
35     get_phi(n);
36     ll ans = 0;
37     for(int i = 1; i <= n; i++){
38         ans += (long long)(n/i) * (n/i) * phi[i];
39     }
40     /*for(int l = 1, r; l <= n; l = r+1){ // 可以用整除分块优化为 O(sqrt(n)) 询问
41         int t = n/l;
42         r = n/t;
43         ans += (long long)t*t * (sum[r] - sum[l-1]); // sum为phi的前缀和数组
44     }*/
45     cout << ans;
46 }

```

```

1 //O(NlogN) 询问
2 #include <iostream>
3
4 const int N = 2000006;
5 long long f[N];
6
7 int main(){
8     int n; std::cin >> n;
9     long long ans = 0;
10    for(int i = n; i; i--){
11        f[i] = (long long)(n/i) * (n/i);
12        for(int j = i << 1; j <= n; j += i) {
13            f[i] -= f[j];
14        }
15        ans += f[i] * i;
16    }
17    std::cout << ans;
18 }

```

拓展欧拉定理 (欧拉降幂)

$$a^b \equiv \begin{cases} a^{b \bmod \varphi(m)}, & \gcd(a, m) = 1, \\ a^b, & \gcd(a, m) \neq 1, b < \varphi(m), \\ a^{(b \bmod \varphi(m)) + \varphi(m)}, & \gcd(a, m) \neq 1, b \geq \varphi(m). \end{cases} \pmod{m}$$

第一个要求a和m互质，第二个和第三个是**广义欧拉降幂**，不要求a和m互质，但要求b和phi(m)的大小关系。

P5091 【模板】扩展欧拉定理 - 洛谷 (luogu.com.cn)

求 $a^{b \% m}$

```
1  #include <iostream>
2  using namespace std;
3
4  int phi(int n){
5      int ans = n;
6      for(int i = 2; i <= n/i; i++){
7          if(n%i == 0){
8              ans = ans/i*(i-1);
9              while(n % i == 0){n /= i;}
10         }
11     }
12     if(n >= 2) ans = ans/n*(n-1);
13     return ans;
14 }
15
16 long long qmi(long long a, long long b, long long p){
17     long long ans = 1;
18     while(b){
19         if(b&1) ans = ans*a%p;
20         b >>= 1;
21         a = a*a%p;
22     }
23     return ans%p;
24 }
25
26 int mo(string &b, int pm){//高精度取模，顺便与phi(m)比较大小
27     int ans = 0;
28     bool flag = 0;
29     for(int i = 0; i < b.size(); i++){
30         int x = b[i] - '0';
31         ans = ans*10+x;
32         if(ans >= pm) flag = 1;
33         ans %= pm;
34     }
35     if(flag) return ans+pm;//b >= phi(m)
36     return ans;//b < phi(m)
37 }
38
39 int main(){
40     int a, m; string b; cin >> a >> m >> b;
41     int pm = phi(m);
42     int ans = qmi(a, mo(b, pm), m);
43     cout << ans;
```

[P4139_上帝与集合的正确用法 - 洛谷 \(luogu.com.cn\)](#)

给定 p , 求 $2^{2^{2^{\dots}}} \% p$

即 $a_0 = 1, a_n = 2^{a_{n-1}}$, 可以证明 $b_n = a_n \bmod p$ 在某一项后都是同一个值, 求这个值。

```

1  #include <iostream>
2  using namespace std;
3  const int N = 10000007;
4  int p;
5  int phi[N], primes[N], cnt;
6  bool st[N];
7
8  void get_phi(int n){
9      phi[1] = 1;
10     for(int i = 2; i <= n; i++){
11         if(!st[i]) {
12             primes[++cnt] = i;
13             phi[i] = i-1;
14         }
15         for(int j = 1; primes[j] <= n/i; j++){
16             st[i*primes[j]] = 1;
17             if(i%primes[j] == 0){
18                 phi[i*primes[j]] = phi[i]*primes[j];
19                 break;
20             }
21             else phi[i*primes[j]] = phi[i]*(primes[j]-1);
22         }
23     }
24 }
25
26 long long qmi(long long a, long long b, long long p){
27     long long ans = 1;
28     while(b){
29         if(b&1) ans = ans*a%p;
30         b >>= 1;
31         a = a*a%p;
32     }
33     return ans%p;
34 }
35
36 int sol(int p){
37     if(p == 1) return 0;
38     return qmi(2, sol(phi[p])+phi[p], p);
39 }
40
41 int main(){
42     get_phi(N-1);

```

```

43     int t;cin >> t;
44     while(t--){
45         cin >> p;
46         cout << sol(p) << '\n';
47     }
48 }

```

[P10414 [蓝桥杯 2023 国 A\] 2023 次方 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P10414)

求 $2^{3^{4^{\dots^{2023}}}} \% 2023$

```

1  int sol(int a,int p){
2      if(p == 1) return 0;
3      return qmi(a,sol(a+1,phi[p])+phi[p],p);
4  }
5  cout << sol(2,2023) << '\n';

```

模数

$a \% b$ 也可以表示为 $a - \lfloor \frac{a}{b} \rfloor * b$

c++取模运算中 $-7 \% 4 = -3$

而数学取模中 $-7 \% 4 = 4$

题目往往要求数学取模

```

1  int f(int x,int mod){//将c++取模转为数学取模
2      return (x % mod + mod) % mod;
3  }

```

运算法则

模运算与基本四则运算有些相似，但是**[除法例外]****[详见：乘法逆元]**。其规则如下：

```

(a + b) % p = (a % p + b % p) % p
(a - b) % p = (a % p - b % p) % p
(a * b) % p = (a % p * b % p) % p
(a * b * c) % p = (a % p * b % p * c % p) % p

```

同余定理

同余定理：给定一个正整数 m ，如果两个整数 a 和 b 满足 $a-b$ 能够被 m 整除，那么就称整数 a 与 b 对模 m 同余， $a - b \equiv 0(mod\ m)$ ，记作 $a \equiv b(mod\ m)$ 。对模 m 同余是整数的一个等价关系。

若数组 $a[]$ 的所有元素对 m 取余相同，则任意两个元素的差都能被 m 整除。 m 必须是区间内所有元素两两差值的最大公约数 GCD 。即 $m = GCD(a_{l+1} - a_l, a_{l+2} - a_{l+1}, \dots, a_r - a_{r-1})$ 。 <https://codeforces.com/contest/2050/problem/F>

线性同余

扩展欧几里得算法求线性同余方程 $ax \equiv b \pmod{m}$ ，因为未知数 x 的指数为 1，所以称作线性同余或一次同余。

给定 a, b, m ，求出 x 使得 $ax \equiv b \pmod{m}$ ，如果无解则输出 impossible。

问题等价于 $ax - b$ 是 m 的倍数，假设为 $-y$ 倍，则问题可转化为 $ax + my = b$ 求解 x 。

```
1 //https://www.acwing.com/problem/content/880/
2 #include <iostream>
3 using namespace std;
4
5 int exgcd(int a,int b,int &x,int &y){
6     if(!b){
7         x = 1,y = 0;
8         return a;
9     }
10    int d = exgcd(b,a%b,x,y);
11    int x1 = x,y1 = y;
12    x = y1;
13    y = x1 - a/b*y1;
14    return d;
15 }
16
17 int f(int a,int b,int m){
18     int x,y,d = exgcd(a,m,x,y);
19     if(b%d) return -1;//若不能整除最大公约数则无解
20     int t = std::abs(m/d);
21     return ((long long)x*(b/d) % t + t) % t;//返回最小正整数解
22 }
23
24 int main(){
25     int q;cin >> q;
26     while(q--){
27         int a,b,m;cin >> a >> b >> m;
28         int x = f(a,b,m);
29         if(x == -1) cout << "impossible\n";
30         else cout << x << '\n';
31     }
32 }
```

高次同余

BSGS

求解 $a^x \equiv b \pmod{p}$ 的最小非负整数解 x (或返回无解), 其中 a, p 互质, $2 \leq a, b < p < 2^{31}$ 。

BSGS(baby-step giant-step, 大步小步算法)常用于求解离散对数问题, 该算法可以在 $O(\sqrt{N})$ 的时间内求解(用map则多一个log)。

```
1 //模版题 https://www.luogu.com.cn/problem/P3846
2 int bsgs(int a,int b,int p){//无解则返回-1, 否则返回最小非负整数解
3     map<int,int>hs;
4     b %= p;
5     int t = (int)sqrt(p) + 1;
6     for(int j = 0;j < t;j++){
7         int val = (long long) b * qmi(a,j,p) % p;
8         hs[val] = j;
9     }
10    a = qmi(a,t,p);
11    if(a == 0) return b == 0 ? 1 : -1;
12    for(int i = 0;i <= t;i++){
13        int val = qmi(a,i,p);
14        int j = hs.find(val) == hs.end() ? -1 : hs[val];
15        if(j >= 0 && i * t - j >= 0) return i * t - j;
16    }
17    return -1;
18 }
```

扩展BSGS

求解 $a^x \equiv b \pmod{p}$ 的最小非负整数解 x (或返回无解), 其中 a, p 不一定互质, $1 \leq a, b, p \leq 10^9$ 或 $a = b = p = 0$ 。

```
1 //模版题 https://www.luogu.com.cn/problem/P4195
2 int exbsgs(int a,int b,int p){
3     a%=p; b%=p;
4     if(b == 1 || p == 1) return 0;
5     int d,ax=1,cnt=0,x,y;
6     while((d=exgcd(a,p,x,y))<1){
7         if(b%d) return -1;
8         b/=d; p/=d; cnt++;
9         ax=1ll*ax*(a/d)%p;
10        if(ax == b) return cnt;
11    }
12    exgcd(ax,p,x,y);
13    int inv=(x%p+p)%p;
14    b=1ll*b*inv%p;
15
16    //下面为bsgs
17    map<int,int>hs;
```

```

18     b %= p;
19     int t = (int)sqrt(p) + 1;
20     for(int j = 0; j < t; j++){
21         int val = (long long) b * qmi(a, j, p) % p;
22         hs[val] = j;
23     }
24     a = qmi(a, t, p);
25     if(a == 0) return b == 0 ? 1 : -1;
26     for(int i = 0; i <= t; i++){
27         int val = qmi(a, i, p);
28         int j = hs.find(val) == hs.end() ? -1 : hs[val];
29         if(j >= 0 && i * t - j >= 0) return i * t - j + cnt; //这里+cnt
30     }
31     return -1;
32 }

```

乘法逆元

费马小定理：对于任何一个整数a，以及素数p：

1. 如果a是p的倍数, $a^p \equiv a \pmod{p}$
2. 如果a不是p的倍数, $a^{p-1} \equiv 1 \pmod{p}$

将上述公式2变形得到:

$$a * a^{p-2} \equiv 1 \pmod{p}$$

$$a^{p-2} \equiv a^{-1} \pmod{p}$$

所以: $a^{-1} \equiv a^{p-2} \pmod{p}$ 其中a和p互质(有逆元的充要条件)

$$\frac{a}{b} \% p = a * b^{p-2} \% p = a \% p * b^{p-2} \% p$$

```

1 //快速幂求逆元
2 //给定n组a,p,其中p是质数,求a模p的乘法逆元,若逆元不存在则输出impossible
3 #include <iostream>
4 using namespace std;
5 using ll = long long;
6
7 ll qmi(ll a, ll b, ll p){
8     ll ans = 1;
9     while(b){
10         if(b&1) ans = ans*a%p;
11         b>>=1;
12         a = a*a%p;
13     }
14     return ans%p;
15 }
16
17 ll gcd(ll a, ll b){return b?gcd(b,a%b):a;}
18
19 int main(){
20     int t; cin >> t;

```

```

21     while(t--){
22         int a,p;cin >> a >> p;
23         if(gcd(a,p) != 1) cout << "impossible" << endl;
24         //a有逆元的充要条件是a与p互质
25         else cout << qmi(a,p-2,p) << endl;
26     }
27 }

```

线性递推求逆元

```

1  inv[1] = 1
2  for (int i = 2; i < N; i++)
3      inv[i] = (mod - 1LL * mod / i) * inv[mod % i] % mod;

```

中国剩余定理

扩展中国剩余定理(EXCRT): 模数不互质

$$\begin{cases} x \equiv m_1 \pmod{a_1} \\ x \equiv m_2 \pmod{a_2} \\ \vdots \\ x \equiv m_n \pmod{a_n} \end{cases}$$

给定 $a[1 \sim n]$ 和 $m[1 \sim n]$, 求一个最小的非负整数 x , 满足 $\forall i \in [1, n], x \equiv m_i \pmod{a_i}$ 。

输出最小非负整数解 x , 如果 x 不存在, 则输出 -1 。 $a \geq 1, 0 \leq m < a$, a_i 之间可以不互质

选第一个式子和第二个式子有:

$$x \equiv m_1 \pmod{a_1}, x \equiv m_2 \pmod{a_2}$$

$$\implies x = k_1 a_1 + m_1, x = k_2 a_2 + m_2 \text{ //即求最小非负整数} k_1$$

$$\implies k_1 a_1 + m_1 = k_2 a_2 + m_2$$

$$\implies k_1 a_1 - k_2 a_2 = m_2 - m_1 \text{ -----①}$$

令 $d = \text{exgcd}(a_1, a_2)$, 解为 k'_1, k'_2

如果 $m_2 - m_1$ 不能整除 d 则无解, 返回 -1

否则一对整数解为 $k_1 = (m_2 - m_1) / d * k'_1, k_2 = (m_2 - m_1) / d * k'_2$

对于①式又有性质 $k_1 = (k_1 + \frac{ka_2}{d}), k_2 = (k_2 + \frac{ka_1}{d})$

k_1 最小非负整数解为 $k_1 = k_1 \% |\frac{a_2}{d}|$ // 不确定 d 的正负, 取绝对值

$$x = (k_1 + \frac{ka_2}{d}) a_1 + m_1$$

$$\implies x = k_1 a_1 + m_1 + k * \frac{a_1 a_2}{d}$$

$$\implies x = k_1 a_1 + m_1 + k * \text{lcm}(a_1, a_2)$$

$$\text{令 } m_0 = k_1 a_1 + m_1, a_0 = \text{lcm}(a_1 a_2)$$

则 $x = ka_0 + m_0$ //再用此式子与其它式子依次递推, $x=m$ 即为当前 x 最小正整数解

[P4777 【模板】扩展中国剩余定理 \(EXCRT\)](#)

给定 n 组非负整数 a_i, b_i , 求解关于 x 的方程组的最小非负整数解。

$$\begin{cases} x \equiv b_1 \pmod{a_1} \\ x \equiv b_2 \pmod{a_2} \\ \dots \\ x \equiv b_n \pmod{a_n} \end{cases}$$

时间复杂度 $O(N)$

```
1 //https://www.luogu.com.cn/problem/P4777
2 //数据略强: q <= 1e5;a[i],b[i] <= 1e12;
3 #include <iostream>
4 #include <vector>
5 using namespace std;
6 const int N = 100005;
7
8 struct node{
9     long long a,b;
10 };
11
12 long long exgcd(long long a,long long b,long long& x,long long& y){
13     if(!b){
14         x = 1,y = 0;
15         return a;
16     }
17     long long d = exgcd(b,a%b,x,y);
18     long long x1 = x,y1 = y;
19     x = y1,y = x1 - a/b*y1;
20     return d;
21 }
22
23 long long excrt(const std::vector<node> &e){//0_idx
24     long long ans = e[0].b,M = e[0].a,x = 0,y = 0;
25     for(int i = 1;i < e.size();i++){
26         long long B = ((e[i].b - ans) % e[i].a + e[i].a) % e[i].a;
27         long long d = exgcd(M,e[i].a,x,y);
28         if(B%d) {return -1;}//若B不能整除最大公约数d,则无解。
29         x = (__int128)x*(B/d) % e[i].a;//__int128或龟速乘防止爆long long
30         ans += M*x;
31         M *= e[i].a / d;
32         ans = (ans + M) % M;//ans即为前i(0_idx)个柿子的解
33     }
34     return ans;
35 }
36
37 int main(){
```

```

38     int n;cin >> n;
39     std::vector<node>e(n);
40     for(int i = 0;i < n;i++) {
41         std::cin >> e[i].a >> e[i].b;//x%a == b
42     }
43     cout << excrt(e);
44 }

```

数论分块

快速求解形如 $\sum_{i=1}^n f(i)g(\lfloor \frac{n}{i} \rfloor)$ 的和式。当可以在 $O(1)$ 内计算 $f(r) - f(l)$ 或已经预处理出 f 的前缀和时，数论分块就可以在 $O(\sqrt{N})$ 的时间内计算上述和式的值。

对于常数 n ，使得式子

$$\left\lfloor \frac{n}{i} \right\rfloor = \left\lfloor \frac{n}{j} \right\rfloor$$

成立且满足 $i \leq j \leq n$ 的 j 值最大为 $\left\lfloor \frac{n}{\left\lfloor \frac{n}{i} \right\rfloor} \right\rfloor$ ，即值 $\left\lfloor \frac{n}{i} \right\rfloor$ 所在块的右端点为 $\left\lfloor \frac{n}{\left\lfloor \frac{n}{i} \right\rfloor} \right\rfloor$ 。

[P2261 [CQOI2007](https://www.luogu.com.cn/problem/P2261)]余数求和 - 洛谷([luogu.com.cn](https://www.luogu.com.cn))

给定 n, k ，求 $\sum_{i=1}^n k \% i$ 。

$$\text{原式} = \sum_{i=1}^n (k - i * \lfloor \frac{k}{i} \rfloor) = n * k - \sum_{i=1}^n (i * \lfloor \frac{k}{i} \rfloor)$$

例如样例 $n = 10, k = 5$

i	1	2	3	4	5	6	7	8	9	10
$t = \lfloor \frac{k}{i} \rfloor$	5	2	1	1	1	0	0	0	0	0

发现 $\lfloor \frac{k}{i} \rfloor$ 分别在一定的区域内相等。

左端点 l 从 1 开始枚举，令 $t = \lfloor \frac{k}{l} \rfloor$ ，则数值 t 所在的右端点 $r = \lfloor \frac{k}{t} \rfloor$ ，在区间 $[l, r]$ 内 t 的值相等，于是我们可以快速求出这一段区间的取值，再让 $l = r + 1$ 。

```

1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      long long n,k;cin >> n >> k;
6      long long ans = n * k;
7      for(int l = 1,r;l <= n;l = r + 1){
8          long long t = k/l;
9          if(t == 0) r = n;

```

```

10     else r = min(k/t,n);
11     ans -= t * (1+r)*(r-1+1)/2;//区间[1,r]值均为t,快速计算
12 }
13 cout << ans;
14 }

```

向上取整的数列分块

$$\left\lceil \frac{n}{i} \right\rceil = \left\lceil \frac{n}{j} \right\rceil$$

成立且满足 $i \leq j \leq n$ 的 j 值最大为 $\left\lfloor \frac{n-1}{\left\lfloor \frac{n-1}{i} \right\rfloor} \right\rfloor$, 即值 $\left\lceil \frac{n}{i} \right\rceil$ 所在块的右端点为 $\left\lfloor \frac{n-1}{\left\lfloor \frac{n-1}{i} \right\rfloor} \right\rfloor$ 。

注意：当 $i = n$ 时，上式会出现分母为 0 的错误，需要特殊处理。

阶&原根

阶

对于 $a \in \mathbf{Z}, m \in \mathbf{N}_+$ 且 $a \perp m$, 满足同余式 $a^n \equiv 1 \pmod{m}$ 的最小正整数 n 称作 a 模 m 的阶 (the order of a modulo m) , 记作 $\delta_m(a)$ 或 $\text{ord}_m(a)$ 。

原根

对于 $m \in \mathbf{N}_+$, 如果存在 $g \in \mathbf{Z}$ 且 $g \perp m$ 使得 $\delta_m(g) = |\mathbf{Z}_m^*| = \varphi(m)$, 就称 g 为模 m 的原根 (primitive root modulo m) 。其中, $\varphi(m)$ 是欧拉函数。

狄利克雷卷积

对于两个数论函数 $f(x)$ 和 $g(x)$, 则他们的狄利克雷卷积得到的结果 $h(x)$ 定义为：

$$(f * g)(n) = \sum_{d|n} f(d)g\left(\frac{n}{d}\right)$$

积性函数：对于任意互质的整数 a 和 b 有性质： $f(ab) = f(a)f(b)$ 的函数。

完全积性函数：对于任意整数 a 和 b 有性质： $f(ab) = f(a)f(b)$ 的函数。

常见的积性函数：

- 单位函数： $\varepsilon(n) = [n = 1]$ 。 (完全积性)
- 恒等函数 (幂函数)： $\text{id}_k(n) = n^k$, $\text{id}_1(n)$ 通常简记作 $\text{id}(n)$ 。 (完全积性)

- 常数函数: $1(n) = 1$ 。(完全积性)
- 除数函数: $\sigma_k(n) = \sum_{d|n} d^k$ 。 $\sigma_0(n)$ 通常简记作 $d(n)$ 或 $\tau(n)$, $\sigma_1(n)$ 通常简记作 $\sigma(n)$ 。
- 欧拉函数: $\varphi(n) = \sum_{i=1}^n [(i, n) = 1]$ 。
- 莫比乌斯函数: $\mu(n) = \begin{cases} 1 & n = 1 \\ 0 & \exists d > 1, d^2 \mid n, \text{ 其中 } \omega(n) \text{ 表示 } n \text{ 的本质不同质因子个数。} \\ (-1)^{\omega(n)} & \text{otherwise} \end{cases}$

莫比乌斯反演

莫比乌斯反演是数论中的重要内容。对于一些函数 $f(n)$, 如果很难直接求出它的值, 而容易求出其倍数和或约数和 $g(n)$, 那么可以通过莫比乌斯反演简化运算, 求得 $f(n)$ 的值。

μ 为**莫比乌斯函数**, 定义为

$$\mu(n) = \begin{cases} 1 & n = 1 \\ 0 & n \text{ 含有平方因子} \\ (-1)^k & k \text{ 为 } n \text{ 的本质不同质因子个数} \end{cases}$$

莫比乌斯函数不仅是积性函数, 还有如下**性质**

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & n = 1 \\ 0 & n \neq 1 \end{cases}$$

即 $\sum_{d|n} \mu(d) = \varepsilon(n)$, $\mu * 1 = \varepsilon$

线性筛

```

1  int mu[N],primes[N],cnt;
2  bool vis[N];
3  void get_mu(int n){
4      mu[1] = 1;
5      for(int i = 2;i <= n;i++){
6          if(!vis[i]) {
7              primes[++cnt] = i;
8              mu[i] = -1;
9          }
10         for(int j = 1;i*primes[j] <= n;j++){
11             vis[i*primes[j]] = 1;
12             if(i%primes[j] == 0) {
13                 mu[i*primes[j]] = 0;
14                 break;
15             }
16             else mu[i*primes[j]] = -mu[i];

```

```

17 |         }
18 |     }
19 | }

```

莫比乌斯变换

形式一（因数关系）：如果有 $f(n) = \sum_{d|n} g(d)$ ，那么有 $g(n) = \sum_{d|n} \mu(d) f(\frac{n}{d})$ 。

这种形式下，数论函数 $f(n)$ 称为数论函数 $g(n)$ 的莫比乌斯变换，数论函数 $g(n)$ 称为数论函数 $f(n)$ 的莫比乌斯逆变换（反演）。

容易看出，数论函数 $g(n)$ 的莫比乌斯变换，就是将数论函数 $g(n)$ 与常数函数 1 进行狄利克雷卷积。

形式二（倍数关系）：如果有 $f(n) = \sum_{n|d} g(d)$ ，那么有 $g(n) = \sum_{n|d} \mu(\frac{d}{n}) f(d)$ 。

两种形式都常用，根据具体问题选择。

[P2257 YY的GCD - 洛谷](#)

求 $\sum_{x=1}^n \sum_{y=1}^m [\gcd(x, y) \in \text{素数}]$ 的 (x, y) 有多少对。 $n, m \leq 10^7$ 。

设 $n \leq m$ ，枚举 $\gcd(x, y) = k$ (k 为素数)：

$$\sum_{\substack{k=1 \\ k \in \mathbb{P}}}^n \sum_{x=1}^n \sum_{y=1}^m [\gcd(x, y) = k]$$

同时除以 k ：

$$\sum_{\substack{k=1 \\ k \in \mathbb{P}}}^n \sum_{x=1}^{\lfloor n/k \rfloor} \sum_{y=1}^{\lfloor m/k \rfloor} [\gcd(x, y) = 1]$$

使用莫比乌斯反演 $\sum_{d|n} \mu(d) [n = 1]$ 替换为 $[\gcd(x, y) = 1] = \sum_{d|\gcd(x, y)} \mu(d)$ ：

$$\sum_{\substack{k=1 \\ k \in \mathbb{P}}}^n \sum_{d=1}^{\lfloor n/k \rfloor} \mu(d) \sum_{x=1}^{\lfloor n/k \rfloor} [d | x] \sum_{y=1}^{\lfloor m/k \rfloor} [d | y]$$

内层计数为 $\lfloor n/(kd) \rfloor$ 和 $\lfloor m/(kd) \rfloor$ ：

$$\sum_{\substack{k=1 \\ k \in \mathbb{P}}}^n \sum_{d=1}^{\lfloor n/k \rfloor} \mu(d) \left\lfloor \frac{n}{kd} \right\rfloor \left\lfloor \frac{m}{kd} \right\rfloor$$

令 $T = kd$ ，交换求和次序：

$$\sum_{T=1}^n \left\lfloor \frac{n}{T} \right\rfloor \left\lfloor \frac{m}{T} \right\rfloor \sum_{\substack{k|T \\ k \in \mathbb{P}}} \mu\left(\frac{T}{k}\right)$$

定义

$$g(T) = \sum_{\substack{k|T \\ k \in \mathbb{P}}} \mu\left(\frac{T}{k}\right)$$

则原式化为

$$\sum_{T=1}^n g(T) \left\lfloor \frac{n}{T} \right\rfloor \left\lfloor \frac{m}{T} \right\rfloor$$

$O(\max(N, M))$ 预处理, $O(\sqrt{\min(N, M)})$ 查询

```

1  #include <iostream>
2
3  const int N = 10000007;
4  int mu[N], primes[N], cnt;
5  int s[N];
6  bool vis[N];
7  long long g[N];
8  void get_mu(int n){
9      mu[1] = 1;
10     for(int i = 2; i <= n; i++){
11         if(!vis[i]) {
12             primes[++cnt] = i;
13             mu[i] = -1;
14         }
15         for(int j = 1; i*primes[j] <= n; j++){
16             vis[i*primes[j]] = 1;
17             if(i%primes[j] == 0) {
18                 mu[i*primes[j]] = 0;
19                 break;
20             }
21             else mu[i*primes[j]] = -mu[i];
22         }
23     }
24     for(int i = 1; i <= cnt; i++){
25         for(int j = 1; j*primes[i] <= n; j++){
26             g[j*primes[i]] += mu[j];
27         }
28     }
29     for(int i = 1; i <= n; i++){
30         s[i] = s[i-1] + g[i];
31     }
32 }
33
34 void sol(){
35     int n, m; std::cin >> n >> m;
36     long long ans = 0;
37     for(int l = 1, r; l <= std::min(n, m); l = r+1){
38         r = std::min(n/(n/l), m/(m/l));
39         ans += (long long)(s[r]-s[l-1]) * (n/l) * (m/l);
40     }
41     std::cout << ans << '\n';

```

```

42 }
43
44 int main(){
45     get_mu(N-1);
46     int t; std::cin >> t;
47     while(t--) sol();
48 }

```

杜教筛

杜教筛被用于处理一类数论函数的前缀和问题。对于数论函数 f ，杜教筛可以再低于线性时间的复杂度内计算 $s(n) = \sum_{i=1}^n f(i)$

我们想办法构造一个 $S(n)$ 关于 $S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$ 的递推式。

对于任意一个数论函数 g ，必满足：

$$\begin{aligned} \sum_{i=1}^n (f * g)(i) &= \sum_{i=1}^n \sum_{d|i} g(d) f\left(\frac{i}{d}\right) \\ &= \sum_{i=1}^n g(i) S\left(\left\lfloor \frac{n}{i} \right\rfloor\right) \end{aligned}$$

其中 $f * g$ 为数论函数 f 和 g 的 [狄利克雷卷积](#)。

那么可以得到递推式：

$$\begin{aligned} g(1)S(n) &= \sum_{i=1}^n g(i)S\left(\left\lfloor \frac{n}{i} \right\rfloor\right) - \sum_{i=2}^n g(i)S\left(\left\lfloor \frac{n}{i} \right\rfloor\right) \\ &= \sum_{i=1}^n (f * g)(i) - \sum_{i=2}^n g(i)S\left(\left\lfloor \frac{n}{i} \right\rfloor\right) \end{aligned}$$

假如我们可以构造恰当的数论函数 g 使得：

1. 可以快速计算 $\sum_{i=1}^n (f * g)(i)$;
2. 可以快速计算 g 的前缀和，以用数论分块求解 $\sum_{i=2}^n g(i)S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$ 。

则我们可以在较短时间内求得 $g(1)S(n)$ 。

求 $ans_1 = \sum_{i=1}^n \varphi(i)$ 和 $ans_2 = \sum_{i=1}^n \mu(i)$, 其中 $1 \leq n < 2^{21}$

欧拉函数前缀和, $\varphi * 1 = \text{id}$, 从而

$$\begin{aligned} S(n) &= \sum_{i=1}^n i - \sum_{i=2}^n S\left(\left\lfloor \frac{n}{i} \right\rfloor\right) \\ &= \frac{1}{2}n(n+1) - \sum_{i=2}^n S\left(\left\lfloor \frac{n}{i} \right\rfloor\right) \end{aligned}$$

莫比乌斯前缀和, $\epsilon = [n = 1] = \mu * 1 = \sum_{d|n} \mu(d)$,

$$\begin{aligned} S_1(n) &= \sum_{i=1}^n \epsilon(i) - \sum_{i=2}^n S_1\left(\left\lfloor \frac{n}{i} \right\rfloor\right) \\ &= 1 - \sum_{i=2}^n S_1\left(\left\lfloor \frac{n}{i} \right\rfloor\right) \end{aligned}$$

前 $N^{\frac{2}{3}}$ 的值预处理, $O(1)$ 询问。后面的值 $O(N^{\frac{2}{3}})$ 询问。

```
1  #include <iostream>
2  #include <unordered_map>
3
4  const int N = 5000006; //预处理前  $N^{2/3}$  的值
5  long long primes[N], cnt, mu[N], phi[N];
6  bool vis[N];
7  std::unordered_map<long long, long long> s_mu, s_phi;
8
9  void init(int n){
10     mu[1] = phi[1] = 1;
11     for(int i = 2; i <= n; i++){
12         if(!vis[i]){
13             primes[++cnt] = i;
14             phi[i] = i-1;
15             mu[i] = -1;
16         }
17         for(int j = 1; i*primes[j] <= n; j++){
18             vis[i*primes[j]] = 1;
19             if(i%primes[j] == 0){
20                 phi[i*primes[j]] = phi[i]*primes[j];
21                 mu[i*primes[j]] = 0;
22                 break;
23             }
24             else{
25                 phi[i*primes[j]] = phi[i] * (primes[j]-1);
26                 mu[i*primes[j]] = -mu[i];
27             }
28         }
29     }
30     for(int i = 1; i <= n; i++){
31         mu[i] += mu[i-1];
32         phi[i] += phi[i-1];
33     }
```

```

34 }
35
36 long long sp(long long x){//欧拉函数的前缀和(也可以用莫比乌斯反演求)
37     if(x < N) return phi[x];
38     if(s_phi[x]) return s_phi[x];
39     long long ans = x*(x+1)/2;
40     for(long long l = 2,r;l <= x;l = r+1){
41         long long t = x/l;
42         r = x/t;
43         ans -= (r-l+1) * sp(t);
44     }
45     return s_phi[x] = ans;
46 }
47
48 long long sm(long long x){//莫比乌斯函数的前缀和
49     if(x < N) return mu[x];
50     if(s_mu[x]) return s_mu[x];
51     long long ans = 1;
52     for(long long l = 2,r;l <= x;l = r+1){
53         long long t = x/l;
54         r = x/t;
55         ans -= (r-l+1) * sm(t);
56     }
57     return s_mu[x] = ans;
58 }
59
60 void sol(){
61     long long n; std::cin >> n;
62     std::cout << sp(n) << ' ' << sm(n) << '\n';
63 }
64
65 int main(){
66     init(N-1);
67     int t; std::cin >> t;
68     while(t--) sol();
69 }

```

类欧几里得算法

求解 $f(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{a \times i + b}{c} \rfloor$

时间复杂度 $O(\log \min(a, c, n))$

```

1 long long sol(long long a, long long b, long long c, long long n) {
2     long long n2 = n * (n + 1) / 2;
3     if (a >= c || b >= c) return sol(a % c, b % c, c, n) + a / c * n2 + b / c *
(n + 1);
4     long long m = (a * n + b) / c;
5     if (!m) return 0;
6     return m * n - sol(c, c - b - 1, a, m - 1);
7 }

```

P5170 【模板】类欧几里德算法 - 洛谷

给定 a, b, c, n 求解以下公式, 答案对 mod 取模, $0 \leq n, a, b, c \leq 10^9, c \neq 0$ 。

$$f(a, b, c, n) = \sum_{i=0}^n \left\lfloor \frac{ai + b}{c} \right\rfloor,$$

$$g(a, b, c, n) = \sum_{i=0}^n i \left\lfloor \frac{ai + b}{c} \right\rfloor,$$

$$h(a, b, c, n) = \sum_{i=0}^n \left\lfloor \frac{ai + b}{c} \right\rfloor^2.$$

```

1 #include <iostream>
2
3 struct Data {
4     long long f, g, h;
5 };
6
7 Data solve(long long a, long long b, long long c, long long n) {
8     constexpr long long M = 998244353;
9     constexpr long long i2 = (M + 1) / 2;
10    constexpr long long i6 = (M + 1) / 6;
11    long long n2 = (n + 1) * n % M * i2 % M;
12    long long n3 = (2 * n + 1) * (n + 1) % M * n % M * i6 % M;
13    Data res = {0, 0, 0};
14    if (a >= c || b >= c) {
15        auto tmp = solve(a % c, b % c, c, n);
16        long long aa = a / c, bb = b / c;
17        res.f = (tmp.f + aa * n2 + bb * (n + 1)) % M;
18        res.g = (tmp.g + aa * n3 + bb * n2) % M;
19        res.h = (tmp.h + 2 * bb * tmp.f % M + 2 * aa * tmp.g % M +
20                aa * aa % M * n3 % M + bb * bb % M * (n + 1) % M +
21                2 * aa * bb % M * n2 % M) % M;
22        return res;
23    }
24    long long m = (a * n + b) / c;
25    if (!m) return res;
26    auto tmp = solve(c, c - b - 1, a, m - 1);
27    res.f = (m * n - tmp.f + M) % M;
28    res.g = (m * n2 + (M - tmp.f) * i2 + (M - tmp.h) * i2) % M;
29    res.h = (n * m % M * m - tmp.f - tmp.g * 2 + 3 * M) % M;
30    return res;

```

```

31 }
32
33 int main() {
34     int t; std::cin >> t;
35     while (t--) {
36         int n, a, b, c;
37         std::cin >> n >> a >> b >> c;
38         auto res = solve(a, b, c, n);
39         std::cout << res.f << ' ' << res.h << ' ' << res.g << '\n';
40     }
41 }

```

其它

一些数学常数

圆周率: $\pi = \arccos(-1) = 3.14159265358979323846264338327950288419716939937510$

自然常数: $e = 2.7182818284590452353602874713526624977572470936999595749$

⑨的倍数

如果一个数能被 9或3 整除, 那么它的所有位数之和也能被 9或3 整除

x个k连在一起组成的正整数

可以表示为 $\frac{k(10^x - 1)}{9}$, 如 $x = 6, k = 8, f(x, k) = 888888$

一堆正整数相乘后, 末尾0的个数

cnt2 统计所有乘数中质因数 2 的个数

cnt5 统计所有乘数中质因数 5 的个数

则末尾0的个数 = $\min(\text{cnt2}, \text{cnt5})$

孪生素数

相差为2的素数对

10^7 以内不重合的孪生素数对 $> 10^5$ 个

组合数学

排列组合

排列 (nPr) : $A_n^m = \frac{n!}{(n-m)!} = \underbrace{n(n-1)(n-2)\dots(n-m+1)}_{m\text{个因子}}$

组合 (nCr) : $C_n^m = \frac{A_n^m}{m!} = \frac{n!}{m!(n-m)!}$

$$A[7,3] = 7 * 6 * 5 = 210$$

$$C[7,3] = (7 * 6 * 5) / (3 * 2 * 1) = 6$$

性质

$$C_n^m = C_{n-1}^{m-1} + C_{n-1}^m$$

$$C_n^0 + C_n^1 + \dots + C_n^n = 2^n$$

$$C_n^m = C_n^{n-m}$$

组合数判定奇偶：对于C(n,m)，若 n&m==m 则为奇数，否则为偶数。

多重集

多重集是指包含重复元素的广义集合，设 $S = \{n_1\text{个}a_1, n_2\text{个}a_2, \dots, n_k\text{个}a_k\}$

则S的不同排列个数 $A = \frac{(\sum n[i]!)!}{\prod (n[i]!)}$

从 k 种元素(每种均可选任意个)中选 r 个元素组成一个多重集，产生的不同多重集的数量为

$$C_{k+r-1}^{k-1} = C_{k+r-1}^r$$

二项式定理

$$(a+b)^n = \sum_{k=0}^n C_n^k a^k b^{n-k}$$

一些常见的组合计数

n*m的网格从左上角(1,1)走到右下角(n,m)，每次只能往下或右走，不同的路径的总数 C_{n+m-2}^{n-1} 或 C_{n+m-2}^{m-1} 种

需要的总步数为n+m-2，其中向下走n-1步，向右走m-1步，不同的路径总共有 C_{n+m-2}^{n-1} 或 C_{n+m-2}^{m-1} 种。

n个相同的物品分成m组，每组至少1个元素的方案数： C_{n-1}^{m-1}

考虑拿 m-1个板子 插到n个元素形成的 n-1个空 里面，将物品分成m组。答案就是 C_{n-1}^{m-1} 。

本质是求 $x_1 + x_2 + \dots + x_m = n$ 的正整数解的组数。其中 $x_i \geq 1$

n个相同的物品分成m组，每组可以有0个元素的方案数： C_{n+m-1}^n 或 C_{n+m-1}^{m-1}

显然此时没法直接插板了，因为可能出现很多块板子插到同一个空里面，不好计算。

先借m个元素过来，在这n+m个形成的n+m-1个空里面插入m-1个板子，方案数为 C_{n+m-1}^n 或 C_{n+m-1}^{m-1} 。

开头借了m个元素用于保证每组至少有一个元素，插完板后将借来的m个元素删除，因为元素是相同的，所以转化过的情况和转化前的情况可以一一对应，答案也就是相等的。

也相当于求多重集{n个物品,m-1个板子}的全排列数。

本质是求 $x_1 + x_2 + \dots + x_m = n$ 的非负整数解的组数。其中 $x_i \geq 0$

n个物品分成m组，要求对于第i组，至少要分到 a_i 个元素($\sum a_i \leq n$)，方案数为 $C_{n+m-1-\sum a_i}^{n-\sum a_i}$

类比上一个问题，我们借 $\sum a_i$ 个元素过来，保证第i组至少能分到 a_i 个，也就是令 $x'_i = x_i - a_i$ 。得到新方程：

$$\begin{aligned}(x'_1 + a_1) + (x'_2 + a_2) + \dots + (x'_k + a_k) &= n \\ x'_1 + x'_2 + \dots + x'_k &= n - a_1 - a_2 - \dots - a_k \\ x'_1 + x'_2 + \dots + x'_k &= n - \sum a_i\end{aligned}$$

其中 $x'_i \geq 0$ 。然后就转化为了上一个问题，直接用插板法公式得到答案为 $C_{n+m-1-\sum a_i}^{n-\sum a_i}$

本质是求 $x_1 + x_2 + \dots + x_m = n$ 的解的数目，其中 $x_i \geq a_i$

从1~n的自然数中选m个，且这m个数中任意两个数差值大于k的方案有 $C_{n-k(m-1)}^m$ 种。(其中 $n - k(m-1) \geq m$ ，否则为0种。)

例题: [Don't be too close \(★6\) - AtCoder typical90 o - \(vjudge.net\)](#)

杨辉三角

$O(N^2)$ 预处理 $O(1)$ 询问

$$C_n^m = C_{n-1}^{m-1} + C_{n-1}^m$$

```

1  const int mod = 1e9+7,N = 2005;
2  long long C[N][N];
3  void init() { //初始化
4      for (int i = 0; i <= 2000; i++){
5          for (int j = 0; j <= i; j++){
6              if (!j) C[i][j] = 1;
7              else C[i][j] = (C[i-1][j] + C[i-1][j-1]) % mod;
8          }
9      }
10 }

```

逆元求组合数

$O(N)$ 预处理 $O(1)$ 询问

$$C_n^m = \frac{n!}{m!(n-m)!} \% P = n! * (m!^{P-2}) * (n-m)!^{P-2} \% P$$

```

1  #include <iostream>
2  using ll = long long;
3  using namespace std;
4  const int N = 100005,P = 1e9+7;
5  ll fact[N],infact[N];
6
7  ll qmi(ll a,ll b,ll p){
8      ll ans = 1;
9      while(b){
10         if(b&1) ans = ans * a % p;
11         b >>= 1;
12         a = a * a % p;
13     }
14     return ans%p;
15 }
16
17 void init(){
18     fact[0] = infact[0] = 1; //0的阶乘等于1
19     for(int i = 1; i < N; i++){ //预处理阶乘
20         fact[i] = fact[i-1]*i%p;
21     }
22     infact[N-1] = qmi(fact[N-1],P-2,P); //预处理阶乘的逆元,倒着处理只用算一次快速幂求逆元
23     for(int i = N-2; i >= 1; i--){
24         infact[i] = infact[i+1]*(i+1)%P;
25     }
26 }
27
28 ll C(int a,int b){
29     //if(a < b) return 0;
30     return fact[a]*infact[b]%P*infact[a-b]%P;
31 }

```

```

32
33 int main(){
34     initi();
35     int t;cin >> t;
36     while(t--){
37         int a,b;cin >> a >> b;
38         cout << C(a,b) << '\n';
39     }
40 }

```

卢卡斯定理

$M \leq N \leq 10^{18}$, $P \leq 10^5$, 其中 P 为质数

$O(P \log N \log P)$ 询问

$$C_n^m \equiv C_{n/p}^{m/p} C_{n \% p}^{m \% p} (\text{mod } p)$$

```

1  #include <iostream>
2  using namespace std;
3  using ll = long long;
4
5  ll qmi(ll a,ll b,ll p){
6      ll ans = 1;
7      while(b){
8          if(b&1) ans = ans*a%p;
9          b >>= 1;
10         a = a*a%p;
11     }
12     return ans%p;
13 }
14
15 ll C(ll a,ll b,ll p){//O(b log P) 朴素求组合数
16     ll ans = 1;
17     for(int i = 1,j = a;i <= b;i++,j--){
18         ans = ans*j%p;
19         ans = ans*qmi(i,p-2,p)%p;
20     }
21     return ans;
22 }
23
24 ll lucas(ll a,ll b,ll p){
25     if(a < p && b < p) return C(a,b,p);
26     return C(a/p,b/p,p)*lucas(a%p,b%p,p)%p;
27 }
28
29 int main(){
30     int q; cin >> q;

```

```

31     while(q--){
32         ll a,b,p; cin >> a >> b >> p;
33         cout << lucas(a,b,p) << endl;
34     }
35
36     return 0;
37 }

```

[P4720 【模板】扩展卢卡斯定理/exLucas - 洛谷](#)

$M \leq N \leq 10^{18}$, $P \leq 10^6$, 其中 P 不一定为质数

```

1  #include <iostream>
2  #include <cmath>
3
4  long long mod;
5
6  long long qmi(long long a,long long b,long long p){
7      long long ans = 1;
8      while(b){
9          if(b&1) ans = ans*a%p;
10         b>>=1;
11         a = a*a%p;
12     }
13     return ans%p;
14 }
15
16 long long exgcd(long long a,long long b,long long& x,long long& y){
17     if(!b){
18         x = 1,y = 0;
19         return a;
20     }
21     long long d = exgcd(b,a%b,x,y);
22     long long x1 = x,y1 = y;
23     x = y1,y = x1 - a/b*y1;
24     return d;
25 }
26
27 long long fac(long long n,long long pi,long long pk){
28     if(!n) return 1;
29     long long ans = 1;
30     for(long long i = 2;i <= pk;i++){
31         if(i%pi) ans = ans*i%p;
32     }
33     ans = qmi(ans,n/pk,pk);
34     for(long long i = 2;i <= n/pk;i++){
35         if(i%pi) ans = ans*i%p;
36     }
37     return ans * fac(n/pi,pi,pk) % pk;
38 }

```

```

39
40 long long inv(long long n,long long p){
41     long long x,y;
42     exgcd(n,p,x,y);
43     return (x%p+p)%p;
44 }
45
46 long long CRT(long long b,long long p) {
47     return b*inv(mod/p,p)%mod*(mod/p)%mod;
48 }
49
50 long long C(long long n,long long m,long long pi,long long pk){
51     long long up = fac(n,pi,pk),d1 = fac(m,pi,pk),d2 = fac(n-m,pi,pk);
52     long long k = 0;
53     for(long long i = n;i;i /= pi) k += i/pi;
54     for(long long i = m;i;i /= pi) k -= i/pi;
55     for(long long i = n-m;i;i /= pi) k -= i/pi;
56     return up * inv(d1,pk)%pk * inv(d2,pk)%pk * qmi(pi,k,pk)%pk;
57 }
58
59 long long exlucas(long long n,long long m){
60     long long ans = 0,tmp = mod;
61     int lim = std::sqrt(mod);
62     for(int i = 2;i <= lim;i++){
63         if(tmp % i == 0){
64             long long pk = 1;
65             while(tmp % i == 0){
66                 pk *= i;
67                 tmp /= i;
68             }
69             ans = (ans + CRT(C(n,m,i,pk),pk)) % mod;
70         }
71     }
72     if(tmp > 1) ans = (ans + CRT(C(n,m,tmp,tmp),tmp)) % mod;
73     return ans;
74 }
75
76 int main(){
77     long long n,m; std::cin >> n >> m >> mod;
78     std::cout << exlucas(n,m);
79 }

```

高精度排列组合

$$n, m \leq 5000$$

阶乘质因数分解+高精度乘法

[阶乘 \(n!\) 的素因数分解 正整数\(sohu.com\)](http://sohu.com)

算术基本定理：任意一个大于1的正整数n,它都可以分解为以下形式,其中p为质数,a为正整数

$$n = P_1^{a_1} P_2^{a_2} \dots P_k^{a_k}$$

$$m = P_1^{b_1} P_2^{b_2} \dots P_k^{b_k}$$

$$n - m = P_1^{c_1} P_2^{c_2} \dots P_k^{c_k}$$

其中P[] 为n以内的所有素数

$$\text{则有 } C_n^m = \frac{n!}{m!(n-m)!} = \prod p_i^{a_i - b_i - c_i}$$

```
1 //https://www.acwing.com/problem/content/890/
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5 const int N = 5003;
6 int primes[N],cnt;
7 bool st[N];
8 int sum[N];
9
10 void initi(int n){
11     for(int i = 2;i <= n;i++){
12         if(!st[i]) primes[++cnt] = i;
13         for(int j = 1;primes[j] <= n/i;j++){
14             st[primes[j]*i] = 1;
15             if(i % primes[j] == 0) break;
16         }
17     }
18 }
19
20 int get(int n,int p){//阶乘分解：计算n!里面分解为p^k的k值
21     //12! = 1*2*3*4*5*6*7*8*9*10*11*12
22     //p = 2 {2,4,6,8,10,12} ans+=6
23     //p^2 = 4 {4,8,12} ans+=3
24     //p^3 = 8 {8} ans+=1
25     //get(12,2) = 6+3+1 = 10
26     int ans = 0;
27     while(n){
28         ans += n/p;
29         n/=p;
30     }
31     return ans;
32 }
33
34 vector<int> mul(vector<int>&A,int b){
35     vector<int>C;
36     int t = 0;
37     for(int i = 0;i < A.size();i++){
38         t += A[i]*b;
39         C.push_back(t%10);
40         t /= 10;
41     }
42     while(t){
```

```

43     C.push_back(t%10);
44     t/=10;
45 }
46 return C;
47 }
48
49 int main(){
50     int a,b;cin >> a >> b;
51     init1(a);
52
53     for(int i = 1;i <= cnt;i++){
54         int p = primes[i];
55         sum[i] = get(a,p) - get(b,p) - get(a-b,p);
56     }
57
58     vector<int>A(1,1);
59     for(int i = 1;i <= cnt;i++){
60         for(int j = 1;j <= sum[i];j++){//A *= primes[i]^sum[i]
61             A = mul(A,primes[i]);
62         }
63     }
64
65     for(int i = A.size()-1;i >= 0;i--){
66         cout << A[i];
67     }
68     return 0;
69 }

```

错位排列

没有任何元素出现在原有位置的排列。即，对于1~n的排列P，如果满足 $P_i \neq i$ ，则称P是n的错位排列。
如三元错位排列有： $\{2, 3, 1\}$ 和 $\{3, 1, 2\}$

递推关系式：

$$D_n = (n-1)(D_{n-1} + D_{n-2}) = n(D_{n-1} + (-1)^n)$$

```

1 //递推计算错位排列数列个数 O(N)
2 a[0] = 1;
3 for(int i = 1;i < N;i++){
4     a[i] = i*a[i-1] + (i&1?-1:1);
5 }
6 for(int i = 1;i < N;i++) cout << a[i] << ' '; //0 1 2 9 44 265 ...

```

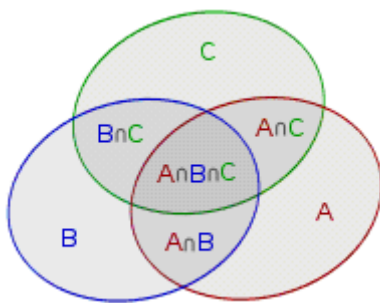
其它关系：

错位排列数有一个简单的取整表达式，增长速度与阶乘仅相差常数： $D_n = \begin{cases} \lceil \frac{n!}{e} \rceil, & n \text{ 为偶数} \\ \lfloor \frac{n!}{e} \rfloor, & n \text{ 为奇数} \end{cases}$

随着元素数量的增加，形成错位排列的概率 P 接近： $P = \lim_{n \rightarrow \infty} \frac{D_n}{n!} = \frac{1}{e}$

容斥原理

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C| - |C \cap A| + |A \cap B \cap C|$$



把上述问题推广到一般情况，就是我们熟知的容斥原理

把包含于某内容中的所有对象的数目先计算出来，然后再把计数时重复计算的数目排斥出去，使得计算的结果既无遗漏又无重复

$$\begin{aligned} \left| \bigcup_{i=1}^n S_i \right| &= \sum_i |S_i| - \sum_{i < j} |S_i \cap S_j| + \sum_{i < j < k} |S_i \cap S_j \cap S_k| - \dots \\ &\quad + (-1)^{m-1} \sum_{a_i < a_{i+1}} \left| \bigcap_{i=1}^m S_{a_i} \right| + \dots + (-1)^{n-1} |S_1 \cap \dots \cap S_n| \end{aligned}$$

即

$$\left| \bigcup_{i=1}^n S_i \right| = \sum_{m=1}^n (-1)^{m-1} \sum_{a_i < a_{i+1}} \left| \bigcap_{i=1}^m S_{a_i} \right|$$

```
1 //https://www.acwing.com/problem/content/description/892/
2 //给定一个整数 n 和 m 个不同的质数 p1,p2,...,pm。
3 //请你求出 1~n中能被 p1,p2,...,pm 中的至少一个数整除的整数有多少个。
4 #include <iostream>
5 using namespace std;
6 using ll = long long;
7 int p[20];
8 ll ans = 0;
9
10 int main(){
11     int n,m;cin >> n >> m;
12     for(int i = 1;i <= m;i++){
13         cin >> p[i];
14     }
```

```

15
16     for(int i = 1; i < 1 << m; i++){//二进制1~11...11枚举所有状态
17         ll cnt = 0, t = 1;
18         for(int k = 0; k < m; k++){
19             if(i >> k & 1){
20                 cnt++;
21                 t *= p[k+1]; // 本题所有数为不同质数,这里累乘.当除数为任意整数时用lcm
22                 if(t > n){t = -1; break;}
23             }
24         }
25         if(t == -1) continue;
26         if(cnt&1) ans += n/t;
27         else ans -= n/t;
28     }
29     cout << ans;
30     return 0;
31 }

```

数列

等差/等比

等差数列

通项公式: $a_n = a_1 + (n - 1) * d$

求和公式: $S_n = \frac{(a_1 + a_n) * n}{2}$

等比数列

通项公式: $a_n = a_1 * q^{n-1}$

求和公式:
$$\begin{cases} S_n = \frac{a_1(q^n - 1)}{q - 1} & (q \neq 1) \\ S_n = n * a_1 & (q = 1) \end{cases}$$

若无法求逆元可以考虑递归:
$$S(k) = \begin{cases} A & k = 1 \\ S(k-1) + A^k & k \text{ 为奇数} \\ S(k/2) + S(k/2) * A^{\frac{k}{2}} & k \text{ 为偶数} \end{cases}$$

$$\sum_{x=1}^n x = \frac{n(n+1)}{2}$$

$$\sum_{x=1}^n x^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{x=1}^n x^3 = \left(\sum_{x=1}^n x\right)^2 = \left(\frac{n(n+1)}{2}\right)^2$$

数列递推

求解递推数列 $a_n = a_{n-1} + k * n + b$ 的第 n 项 (其中 k, b 为常数)

$$a_n = a_1 + \sum_{i=2}^n (ki + b) = a_1 + k \sum_{i=2}^n i + b \sum_{i=2}^n 1$$

$$\text{其中 } \sum_{i=2}^n i = \frac{n(n+1)}{2} - 1, \quad \sum_{i=2}^n 1 = n - 1$$

$$\text{代入求和得 } a_n = \frac{k}{2}n^2 + (b + \frac{k}{2})n + (a_1 - b - k)$$

斐波那契

[斐波那契数列相关 - zhoukangyang - 博客园\(cnblogs.com\)](http://zhoukangyang.cnblogs.com)

$$f(0) = 0, f(1) = 1, f(n) = f(n-2) + f(n-1)$$

0 1 1 2 3 5 8 13 21

一些性质

1. 卡西尼性质: $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$ 。
2. 附加性质: $F_{n+k} = F_kF_{n+1} + F_{k-1}F_n$ 。
3. 取上一条性质中 $k = n$, 我们得到 $F_{2n} = F_n(F_{n+1} + F_{n-1})$ 。
4. 由上一条性质可以归纳证明, $\forall k \in \mathbb{N}, F_n | F_{nk}$ 。
5. 上述性质可逆, 即 $\forall F_a | F_b, a | b$ 。
6. GCD 性质: $GCD(F_n, F_m) = F_{GCD(n, m)}$ 。

快速倍增法

$O(\log N)$ 询问 比矩阵乘法常数更小, 返回值为 `pair<Fib(n), Fib(n+1)>`

$$F_{2k} = F_k(2F_{k+1} - F_k)$$

$$F_{2k+1} = F_{k+1}^2 + F_k^2$$

```

1 pair<ll,ll> fib(ll n) {
2     if (n == 0) return {0,1};
3     auto [a,b] = fib(n >> 1);
4     ll c = a * (2*b%mod - a + mod)%mod;
5     ll d = (a*a%mod + b*b%mod)%mod;
6     if (n&1) return {d,c+d};
7     else return {c,d};
8 }
9 //cout << fib(n).first;

```

广义斐波那契数列

$O(\log N)$ 询问

例：求 $f(n) = p * f(n-1) + q * f(n-2)$

可以得到矩阵公式：
$$\begin{bmatrix} f(n) & f(n-1) \end{bmatrix} = \begin{bmatrix} f(n-1) & f(n-2) \end{bmatrix} \times \begin{bmatrix} p & 1 \\ q & 0 \end{bmatrix}$$

$$ans = \begin{bmatrix} f(2) & f(1) \\ 0 & 0 \end{bmatrix}$$

$$base = \begin{bmatrix} p & 1 \\ q & 0 \end{bmatrix}$$

利用矩阵快速幂求 $ans * base^{n-2}$ ，对于 $n \leq 2$ 的情况，直接输出答案即可（因为第一次相乘即得到 $F[3]$ ，所以是 $n-2$ 次方）

有关base转移矩阵的构造： $F[N] = p * F[n-1] + q * F[n-2]$ ，

$F[N-1] = 1 * F[N-1] + 0 * F[N-2]$

实际是转为了二维： $F[i][0] = F[i-1][0] * p + F[i-1][1] * q$ ， $F[i][1] = F[i-1][0] * 1$

	F[n]	F[n-1]
F[n-1]	p	1
F[n-2]	q	0

```

1 //https://www.luogu.com.cn/problem/P1349
2 //给定p,q和两项f(1),f(2) 求f(n)%mod
3 #include <iostream>
4 using namespace std;
5 int mod;
6
7 struct mat{
8     long long a[2][2];
9 };
10
11 mat operator * (const mat &a,const mat &b){

```

```

12     mat ans = {};
13     for(int i = 0; i < 2; i++){
14         for(int j = 0; j < 2; j++){
15             for(int k = 0; k < 2; k++){
16                 ans.a[i][j] = (ans.a[i][j] + a.a[i][k] * b.a[k][j]) % mod;
17             }
18         }
19     }
20     return ans;
21 }
22
23 mat qmi(mat a, long long b){
24     mat ans = {};
25     for(int i = 0; i < 2; i++){
26         ans.a[i][i] = 1;
27     }
28     while(b){
29         if(b & 1) ans = ans * a;
30         b >>= 1;
31         a = a * a;
32     }
33     return ans;
34 }
35
36 int main(){
37     int p, q, a1, a2, n; cin >> p >> q >> a1 >> a2 >> n >> mod;
38     if(n == 1){cout << a1; return 0;}
39     if(n == 2){cout << a2; return 0;}
40
41     mat ans = {a2, a1};
42     mat base = {p, 1, q, 0};
43     ans = ans * qmi(base, n-2);
44
45     cout << ans.a[0][0];
46 }

```

皮萨诺周期

模 p 意义下的斐波那契数列最下正周期被称为皮萨诺周期。

皮萨诺周期总是**不超过** $6p$ ，且只有在 $p = 2 * 5^k$ 形式下才取得等号

当需要计算第 n 项斐波那契数模 m 的值的时候，如果 n 非常大，就需要计算斐波那契数模 m 的周期。当然，只需要计算周期，不一定是最小正周期。

如果 a 与 b 互素， ab 的皮萨诺周期就是 a 的皮萨诺周期与 b 的皮萨诺周期的最小公倍数

```

1 //https://codeforces.com/contest/2033/problem/F
2 //G(n,k)为第n个能被k整除的数的下标 n<=1e18,k<=1e5
3 //暴力找出第一次能被k整除时的下标i, n*i即为第n个能被k整除的下标
4 void sol(){
5     ll n,k;cin >> n >> k;
6     ll l = 0,r = 1;
7     for(ll i = 1;;i++){
8         tie(l,r) = make_pair(r%k,(l+r)%k);
9         if(!l) { cout << n%mod*i%mod << '\n'; return; }
10    }
11 }

```

卡特兰数

组合数学中一个常出现在各种计数问题中出现的数列

[卡特兰数 \(Catalan\) 公式、证明、代码、典例. 卡特兰数公式](#)

1,1,2,5,14,42,132,429...(从第0项开始)

通项公式: $h(n) = \frac{1}{n+1} C_{2n}^n = C_{2n}^n - C_{2n}^{n+1}$

递推公式: $h[0] = 1, h[i] = \frac{2*(2i-1)}{i+1} * h[i-1]$

例题: n对括号有多少种匹配方式? (若有k种括号, 则答案为 $h(n) * k^n$)

求n个节点能够构成的不同的二叉树的个数?

一个栈的进栈序列为1, 2, 3, ..., n有多少个不同的出栈序列?

给出一个n,要求一个长度为2n的01序列,使得序列的任意前缀中1的个数不少于0的个数, 有多少个不同的01序列?

[889. 满足条件的01序列 - AcWing题库](#)

```

1 //组合数求卡特兰数
2 //O(N)预处理, O(1)询问
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const int mod = 1e9 + 7;
7 const int N = 200005; // 空间开到2倍
8
9 long long qmi(long long a, long long b, long long p){
10     long long ans = 1;

```

```

11     while(b){
12         if(b&1) ans = ans*a%p;
13         b>>=1;
14         a = a*a%p;
15     }
16     return ans%p;
17 }
18
19 long long fact[N], infact[N], inv[N];
20 void init() {
21     inv[1] = fact[1] = infact[1] = 1;
22     for (int i = 2; i < N; i++) {
23         fact[i] = fact[i - 1] * i % mod;
24         inv[i] = (mod - mod / i) * inv[mod % i] % mod;
25     }
26     infact[N - 1] = qmi(fact[N - 1], mod - 2, mod);
27     for (int i = N - 2; i > 1; i--) {
28         infact[i] = infact[i + 1] * (i + 1) % mod;
29     }
30 }
31
32 long long C(int n, int m) {
33     return fact[n] * infact[m] % mod * infact[n - m] % mod;
34 }
35
36 int main() {
37     init();
38     int n; std::cin >> n;
39     std::cout << C(2 * n, n) * inv[n + 1] % mod;
40 }

```

```

1 //递推求卡特兰数
2 //O(N)预处理,O(1)询问
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const int mod = 1e9 + 7;
7 const int N = 100005;
8
9 long long qmi(long long a, long long b, long long p){
10     long long ans = 1;
11     while(b){
12         if(b&1) ans = ans*a%p;
13         b>>=1;
14         a = a*a%p;
15     }
16     return ans%p;
17 }
18
19 long long inv[N], h[N];
20 void init() {

```

```

21     inv[1] = 1;
22     for (int i = 2; i < N; i++) {
23         inv[i] = (mod - mod / i) * inv[mod % i] % mod;
24     }
25     h[0] = 1;
26     for (int i = 1; i < N; i++) {
27         h[i] = h[i - 1] * (4 * i - 2) % mod * inv[i + 1] % mod;
28     }
29 }
30
31 int main() {
32     init();
33     int n; std::cin >> n;
34     std::cout << h[n];
35 }

```

```

1  #python高精度求卡特兰数
2  import math
3  n = int(input())
4  A = math.factorial(2 * n)
5  B = math.factorial(n)
6  ans = A/B/B/(n+1);
7  print(ans)

```

k-Dyck路

k-Dyck路是组合数学计数问题中的一个概念，有的地方也称为K阶卡特兰数，定义：

在笛卡尔坐标系中，起点为 $(0, 0)$ ，终点为 $((k+1)n, 0)$ ，每次只能走向量 $(1, 1)$ 或 $(1, -k)$ ，且路径不能低于x轴的路径被称为 k-Dyck路。求解所有合法的k-Dyck路的方案数。

方案数：
$$D(n, k) = \frac{1}{(k+1)n+1} C_{(k+1)n+1}^n$$

当 $k = 1$ 时，答案就是经典的卡特兰数 $D(n, 1) = H_n = \frac{1}{n+1} C_{2n}^n$

贝尔数 (集合划分)

$$B_0 = 1, B_1 = 1, B_2 = 2, B_3 = 5, B_4 = 15, B_5 = 52, B_6 = 203, \dots$$

B_n 是基数为 n 的集合的划分方法的数目。集合 S 的一个划分是定义为 S 的两两不相交的非空子集的族，它们的并是 S 。例如 $B_3 = 5$ 因为 3 个元素的集合 a, b, c 有 5 种不同的划分方法：

$\{\{a\}, \{b\}, \{c\}\}$
 $\{\{a\}, \{b, c\}\}$
 $\{\{b\}, \{a, c\}\}$
 $\{\{c\}, \{a, b\}\}$
 $\{\{a, b, c\}\}$

```

1 //O(N^2)预处理 O(1)询问
2 //贝尔三角形,每行的首项即为贝尔数
3 const int N = 2003,mod = 1e9+7;
4 void get_bell(int n) {
5     bell[0][0] = 1;
6     for (int i = 1; i <= n; i++) {
7         bell[i][0] = bell[i-1][i-1];
8         for (int j = 1; j <= i; j++)
9             bell[i][j] = (bell[i-1][j-1] + bell[i][j-1])%mod;
10    }
11 }
12
13 for(int i = 0;i <= n;i++) cout << bell[i][0] << endl;

```

分拆数

分拆数：将自然数 n 写成 k 个递减正整数和的表示。

$$n = a_1 + a_2 + \cdots + a_k \quad a_1 \geq a_2 \geq \cdots \geq a_k \geq 1$$

递推式： $p(n, k) = p(n-1, k-1) + p(n-k, k)$

时间复杂度： $O(nk)$ 预处理

互异分拆数：分拆数的 k 部分都不同

$$n = a_1 + a_2 + \cdots + a_k \quad a_1 > a_2 > \cdots > a_k \geq 1$$

递推式： $pd(n, k) = pd(n-k, k-1) + pd(n-k, k)$

时间复杂度： $O(nk)$ 预处理。第二维空间可以用滚动数组优化。

五边形数定理：可以在 $O(N\sqrt{N})$ 求分拆数

```

1 //https://vjudge.net/problem/LibreOJ-6268
2 include <cstdio>
3 const int mod = 998244353;
4 const int N = 100005;

```

```

5 long long a[N << 1];
6 long long p[N] = {1,1,2};
7
8 void init(){
9     for(int i = 1;i < N;i++){ /*递推式系数1,2,5,7,12,15,22,26...i*(3*i-1)/2,i*
10        (3*i+1)/2*/
11        a[i << 1] = (long long)i * (i*3 - 1) / 2; /*五边形数为1,5,12,22...i*(3*i-
12        1)/2*/
13        a[i << 1 | 1] = (long long)i * (i*3 + 1) / 2;
14    }
15    for(int i = 3;i < N;i++){ /*p[n]=p[n-1]+p[n-2]-p[n-5]-p[n-
16    7]+p[12]+p[15]-...+p[n-i*[3i-1]/2]+p[n-i*[3i+1]/2]*/
17        p[i] = 0;
18        for(int j = 2;a[j] <= i;j++){
19            if(j&2){
20                p[i] = (p[i] + p[i-a[j]] + mod) % mod;
21            }
22            else{
23                p[i] = (p[i] - p[i-a[j]] + mod) % mod;
24            }
25        }
26    }
27 }
28
29 int main() {
30     init();
31     int n; scanf("%d",&n);
32     for(int i = 1;i <= n;i++) {
33         printf("%d\n",p[i]);
34     }
35 }

```

调和级数

harmonic series

$$H_n = \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

在 n 较小时可以直接预处理

n 较大时约等于 $\ln(n) + \gamma + \frac{1}{2n}$

其中 γ 为欧拉常数, 约等于 0.577215664901532860606512090082402431042159335

```

1 inv[1] = h[1] = 1;
2 for (int i = 2; i < N; i++) {
3     inv[i] = mod - mod / i * inv[mod % i] % mod; //i的逆元
4     h[i] = (h[i - 1] + inv[i]) % mod; //第i个调和级数
5 }

```

康托展开 (全排列排名)

康托展开：给定一个长度为N的全排列a[]，求它是第几个排列。

逆康托展开：给定一个全排列的长度N，求第rk个排列是什么？

重要柿子： $rk = \sum_{i=1}^n S(i) * (n - i)!$

s(i) 表示 1 ~ a[i]-1 中未出现过的数的个数，它可以看作是一种特殊的进制，也叫做阶乘进制。

如： $(463)_{10} = (341010)_! = 3 \times 5! + 4 \times 4! + 1 \times 3! + 0 \times 2! + 1 \times 1! + 0 \times 0!$

时间复杂度 $O(N \log N)$

假设原排列为a[]，阶乘进制为s[]，排名为rk，下面直接给出转换代码（排名从0开始，数组下标均从1开始）

$a[] \rightleftharpoons s[] \rightleftharpoons rk$

```
1 vector<long long> a_to_s(int n,vector<long long>&a){
2     vector<long long>s(n+1);
3     vector<long long>t(n+1);
4     auto add = [&](int i,int x){
5         while(i <= n){
6             t[i] += x;
7             i += i&-i;
8         }
9     };
10    auto query = [&](int i)->long long{
11        long long ans = 0;
12        while(i){
13            ans += t[i];
14            i -= i&-i;
15        }
16        return ans;
17    };
18    for(int i = 1;i <= n;i++){
19        add(i,1);
20    }
21    for(int i = 1;i <= n;i++){
22        s[i] = query(a[i]-1);
23        add(a[i],-1);
24    }
25    return s;
26 }
```

```
1 vector<long long> s_to_a(int n,vector<long long>&s){
2     vector<long long>a(n+1);
3     struct st{
4         int l,r,sum;
5     };
6     st* t = new st[(n<<2)+5]; //权值线段树,求全局第k小
7 }
```

```

8     auto pushup = [&](int p){
9         t[p].sum = t[p<<1].sum + t[p<<1|1].sum;
10    };
11
12    auto build = [&](auto& build, int p, int l, int r) -> void {
13        t[p].l = l;
14        t[p].r = r;
15        if (l == r) {
16            t[p].sum = 1;
17            return;
18        }
19        int mid = (l + r) >> 1;
20        build(build, p << 1, l, mid); build(build, p << 1 | 1, mid + 1, r);
21        pushup(p);
22    };
23
24    auto kth = [&](auto &kth, int p, int l, int r, int k) -> int {
25        if (t[p].l == t[p].r) {
26            t[p].sum = 0; //找到第k小时,顺便将他删除
27            return t[p].l;
28        }
29        int mid = t[p].l + t[p].r >> 1;
30        int res = 0;
31        if (k <= t[p<<1].sum) res = kth(kth, p<<1, l, mid, k);
32        else res = kth(kth, p<<1|1, mid+1, r, k-t[p<<1].sum);
33        pushup(p);
34        return res;
35    };
36
37    build(build, 1, 1, n);
38
39    for (int i = 1; i <= n; i++) {
40        a[i] = kth(kth, 1, 1, n, s[i]+1);
41    }
42    //delete[] t;
43    return a;
44 }

```

```

1 long long s_to_rk(int n, vector<long long>&s) {
2     vector<long long> fact(n+1);
3     fact[0] = 1;
4     for (int i = 1; i <= n; i++) {
5         fact[i] = fact[i-1] * i % mod;
6     }
7     long long rk = 0;
8     for (int i = 1; i <= n; i++) {
9         rk = (rk + s[i] * fact[n-i]) % mod;
10    }
11    return rk;
12 }
13

```

```

14 void s_add_rk(int n,vector<long long>&s,long long rk){
15     s[n] += rk;
16     for(int i = n;i >= 1;i--){
17         s[i-1] += s[i] / (n-i+1);
18         s[i] %= (n-i+1);
19     }
20 }

```

```

1 vector<long long> rk_to_s(int n,long long rk){
2     vector<long long>s(n+1);
3     for(int i = 1;i <= n;i++){
4         s[n-i+1] = rk % i;
5         rk /= i;
6     }
7     return s;
8 };

```

一些例题: $a[] \rightarrow s[] \rightarrow rk$: [P5367 【模板】康托展开 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P5367)

$s[] \rightarrow a[]$: [UVA11525 Permutation - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/UVA11525)

$a[] \rightarrow s[]$, $s[] + rk \rightarrow a[]$ [U72177 火星人plus - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/U72177)

范德蒙卷积公式

在数量为 $n + m$ 的堆中选 k 个元素, 和分别在数量为 n 、 m 的堆中选 i 、 $k - i$ 个元素的方案数是相同

的, 即
$$\sum_{i=0}^k \binom{n}{i} \binom{m}{k-i} = \binom{n+m}{k}$$

变体:

- $\sum_{i=0}^k C_{i+n}^i = C_{k+n+1}^k$
- $\sum_{i=0}^k C_n^i * C_m^i = \sum_{i=0}^k C_n^i * C_m^{m-i} = C_{n+m}^n$

线性代数

矩阵

矩阵乘法

矩阵相乘只有在第一个矩阵的列数和第二个矩阵的行数相同时才有意义。

设 A 为 $P \times M$ 的矩阵, B 为 $M \times Q$ 的矩阵, 设矩阵 C 为矩阵 A 与 B 的乘积,

其中矩阵 C 中的第 i 行第 j 列元素可以表示为矩阵 A 的第 i 行与矩阵 B 的第 j 列分别相乘再求和:

$$C_{i,j} = \sum_{k=1}^M A_{i,k} B_{k,j}$$

矩阵乘法满足结合律，不满足一般的交换律。利用结合律，矩阵乘法可以用快速幂的思想优化。

矩阵快速幂

时间复杂度 $O(N^3 \log K)$

[P3390 【模板】矩阵快速幂 - 洛谷 \(luogu.com.cn\)](#)

给点一个 $n \times n$ 的矩阵 A ，求 A^k ，对 $1e9+7$ 取模。

```

1  #include <iostream>
2  using namespace std;
3  const int N = 105, mod = 1e9+7;
4  long long n, k;
5
6  struct mat{
7      long long a[N][N];
8  };
9
10 mat operator * (const mat &a, const mat &b){
11     mat ans = {};
12     for(int i = 1; i <= n; i++){ //循环顺序可以优化为ikj
13         for(int j = 1; j <= n; j++){
14             for(int k = 1; k <= n; k++){
15                 ans.a[i][j] = (ans.a[i][j] + a.a[i][k] * b.a[k][j]) % mod;
16             }
17         }
18     }
19     return ans;
20 }
21
22 mat qmi(mat a, long long b){
23     mat ans = {};
24     for(int i = 1; i <= n; i++){ //单位矩阵
25         ans.a[i][i] = 1;
26     }
27     while(b){
28         if(b & 1) ans = ans * a;
29         b >>= 1;
30         a = a * a;
31     }
32     return ans;
33 }
34

```

```

35 void qmi(mat &ans,mat a, long long b){
36     while(b){
37         if(b & 1)ans = ans * a;
38         b >>= 1;
39         a = a * a;
40     }
41 }
42
43 int main(){
44     cin >> n >> k;
45     mat base = {};
46     for(int i = 1;i <= n;i++){
47         for(int j = 1;j <= n;j++){
48             cin >> base.a[i][j];
49         }
50     }
51
52     mat ans = qmi(base,k);
53
54     for(int i = 1;i <= n;i++){
55         for(int j = 1;j <= n;j++){
56             cout << ans.a[i][j] << ' ';
57         }cout << '\n';
58     }
59 }

```

[P10502 Matrix Power Series - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P10502)

给定一个 $n \times n$ 的矩阵 A 和一个正整数 k ，求 $S = A + A^1 + A^2 + \dots + A^k$ 。

$$S(k) = \begin{cases} S(k-1) + A^k & k \text{ 为奇数} \\ S(k/2) + S(k/2) * A^{\frac{k}{2}} & k \text{ 为偶数} \end{cases}$$

```

1  #include <iostream>
2  #include <vector>
3
4  int mod;
5
6  struct Mat{
7      int n;
8      std::vector<std::vector<int>> >a;
9
10     Mat(){}
11     Mat(int _n) {
12         n = _n;
13         a = std::vector<std::vector<int>> >(n,std::vector<int> (n));
14     }
15
16     Mat operator * (const Mat &m2){

```

```

17     Mat ans(n);
18     for(int i = 0; i < n; i++){
19         for(int j = 0; j < n; j++){
20             for(int k = 0; k < n; k++){
21                 ans.a[i][j] = (ans.a[i][j] + a[i][k] * m2.a[k][j]) % mod;
22             }
23         }
24     }
25     return ans;
26 }
27
28 Mat operator + (const Mat &m2){
29     Mat ans = *this;
30     for(int i = 0; i < n; i++){
31         for(int j = 0; j < n; j++){
32             ans.a[i][j] = (ans.a[i][j] + m2.a[i][j]) % mod;
33         }
34     }
35     return ans;
36 }
37
38 Mat qmi(long long b){
39     Mat base = *this;
40     Mat ans(n);
41     for(int i = 0; i < n; i++) ans.a[i][i] = 1;
42     while(b){
43         if(b & 1) ans = ans * base;
44         b >>= 1;
45         base = base * base;
46     }
47     return ans;
48 }
49 };
50
51 Mat A;
52
53 Mat f(int k){
54     if(k == 1) return A;
55     if(k & 1) {
56         return f(k-1) + A.qmi(k);
57     }
58     else{
59         Mat B = f(k/2);
60         return B + B * A.qmi(k/2);
61     }
62 }
63
64 int main(){
65     int n, k; std::cin >> n >> k >> mod;
66     A = Mat(n);
67     for(int i = 0; i < n; i++){
68         for(int j = 0; j < n; j++){

```

```
69         std::cin >> A.a[i][j];
70         A.a[i][j] %= mod;
71     }
72 }
73
74 A = f(k);
75
76 for(int i = 0; i < n; i++){
77     for(int j = 0; j < n; j++){
78         std::cout << A.a[i][j] << ' ';
79     }
80     std::cout << '\n';
81 }
82 }
```

矩阵封装

默认0_idx、N*N的矩阵大小，根据实际需要修改代码。

MatA(n);		
A +* B	矩阵加/乘法	
auto B = A.qmi(k)	矩阵快速幂	
A.norm()	化为单位矩阵	
auto B = inv(A)	求逆矩阵	若不存在则 B[0][0] == -1
det(A)	行列式求值	
add(x,y,w)	矩阵树连边	注意0_idx

```
1  const int mod = 1e9+7;
2  namespace MAT{
3
4      template<typename T>
5      struct Mat{//0_idx
6          int n;
7          std::vector<std::vector<T>>>a;
8
9          Mat(int _n,T val = 0) {
10              n = _n;
11              a = std::vector<std::vector<T>>>(n,std::vector<T>(n,val));
```

```

12     }
13
14     Mat operator * (const Mat &m2){
15         Mat ans(n,0);
16         for(int i = 0;i < n;i++){
17             for(int j = 0;j < n;j++){
18                 for(int k = 0;k < n;k++){
19                     ans.a[i][j] = (ans.a[i][j] + a[i][k] * m2.a[k][j]) %
mod;
20                 }
21             }
22         }
23         return ans;
24     }
25
26     Mat operator + (const Mat &m2){
27         auto ans = *this;
28         for(int i = 0;i < n;i++){
29             for(int j = 0;j < n;j++){
30                 ans.a[i][j] = (ans.a[i][j] + m2.a[i][j]) % mod;
31             }
32         }
33         return ans;
34     }
35     void operator *= (const Mat &m2) {*this = *this * m2;}
36     void operator += (const Mat &m2) {*this = *this + m2;}
37
38     void norm(){
39         for(int i = 0;i < n;i++) a[i][i] = 1;
40     }
41
42     void add(int x,int y,int w){//Mat_tree:add_edge(x,y,w);
43         if(x == y) return;
44         a[y][y] = (a[y][y] + w) % mod;
45         a[x][y] = (a[x][y] - w) % mod;
46     }
47
48     Mat qmi(long long b){
49         auto base = *this;
50         Mat ans(n);
51         ans.norm();
52         while(b) {
53             if(b&1) ans = ans * base;
54             b >>= 1;
55             base = base * base;
56         }
57         return ans;
58     }
59 };
60
61 long long qmi(long long a,long long b,long long p){
62     long long ans = 1;

```

```

63     while(b){
64         if(b&1) ans = ans*a%p;
65         b>>=1;
66         a = a*a%p;
67     }
68     return ans;
69 }
70
71 template<typename T>
72 Mat<T> inv(Mat<T> mt){ //矩阵求逆
73     int n = mt.n;
74     std::vector<std::vector<long long>> aug(n, std::vector<long long>(2 *
n, 0));
75     for (int i = 0; i < n; i++) {
76         for (int j = 0; j < n; j++) { aug[i][j] = mt.a[i][j]; }
77         aug[i][i + n] = 1;
78     }
79
80     for (int i = 0; i < n; i++) {
81         int pivot = -1;
82         for (int r = i; r < n; r++) {
83             if (aug[r][i] != 0) {
84                 pivot = r;
85                 break;
86             }
87         }
88         if (pivot == -1) { mt.a[0][0] = -1;return mt; }//No_inv
89         if (i != pivot) { std::swap(aug[i], aug[pivot]); }
90         long long inv_val = qmi(aug[i][i], mod - 2, mod);
91         for (int j = 0; j < 2 * n; j++) {
92             aug[i][j] = aug[i][j] * inv_val % mod;
93         }
94         for (int j = 0; j < n; j++) {
95             if (j == i) continue;
96             long long mul = aug[j][i];
97             for (int k = 0; k < 2 * n; k++) {
98                 aug[j][k] = (aug[j][k] - mul * aug[i][k] % mod + mod) %
mod;
99             }
100         }
101     }
102
103     for (int i = 0; i < n; i++) {
104         for (int j = 0; j < n; j++) {
105             mt.a[i][j] = aug[i][j + n];
106         }
107     }
108     return mt;
109 }
110
111 template<typename T>
112 T det(Mat<T> mt){ //行列行列式求值

```

```

113     int n = mt.n;
114     long long ans = 1;
115     for(int i = 0; i < n; i++){//if(mat_tree) please i begin with 1
116         for(int j = i+1; j < n; j++){
117             while(mt.a[j][i]){
118                 long long t = mt.a[i][i]/mt.a[j][i];
119                 for(int k = i; k < n; k++) {
120                     mt.a[i][k] = (mt.a[i][k] - mt.a[j][k] * t % mod + mod)
% mod;
121                     //mt.a[i][k] -= t * mt.a[j][k];
122                 }
123                 std::swap(mt.a[i], mt.a[j]);
124                 ans = -ans;
125             }
126         }
127         if(!mt.a[i][i]) return 0;
128         ans = (ans * mt.a[i][i] % mod + mod) % mod;
129         //ans *= mt.a[i][i];
130     }
131     return std::abs(ans);
132 }
133 }
134 using MAT::Mat;

```

行列式

定义

$$A = \begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix}$$

对于一个矩阵 $A[1 \dots n][1 \dots n]$, 其行列式为 $\det(A) = \sum_P (-1)^{\mu(P)} \prod_{i=1}^n A[i][p_i]$ 。
(枚举排列 $P[1 \dots n]$, 其中 $\mu(P)$ 为排列 P 的逆序对数, $\det(A)$ 又称作 $|A|$)

一些性质

- 单位矩阵的行列式为1。
- 交换两行（列），行列式的值变号。
- 若某一行（列）乘以 t , 行列式的值也就乘以 t 。
- 若行列式某一行（列）的元素是两数之和，则行列式可拆成两个行列式的和。

$$\begin{vmatrix} a+a' & b+b' \\ c & d \end{vmatrix} = \begin{vmatrix} a & b \\ c & d \end{vmatrix} + \begin{vmatrix} a' & b' \\ c & d \end{vmatrix}$$

- 行列式某一行（列）元素加上另一行（列）对应元素的k倍，行列式的值不变。
- 若有两行（列）一样，则行列式的值为0

行列式求值

高斯消元实现，时间复杂度 $O(N^3)$

```
1 //模版 https://www.luogu.com.cn/problem/P7112
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 int mod;
6
7 struct Mat{//1_idx
8     std::vector<std::vector<long long>>>a;
9
10    Mat(int n){
11        a = std::vector<std::vector<long long>>>(n+1,std::vector<long long>
12        (n+1));
13    }
14
15    long long det(int n){
16        long long ans = 1;
17        for(int i = 1;i <= n;i++){
18            for(int j = i+1;j <= n;j++){
19                while(a[j][i]){
20                    long long t = a[i][i]/a[j][i];
21                    for(int k = i;k <= n;k++) {
22                        a[i][k] = (a[i][k] - a[j][k] * t % mod + mod) % mod;
23                    }
24                    std::swap(a[i],a[j]);
25                    ans = -ans;
26                }
27            }
28            if(!a[i][i]) return 0;
29            ans = (ans * a[i][i] % mod + mod) % mod;
30        }
31        return std::abs(ans);
32    }
33};
34
35 int main(){
36     int n; cin >> n >> mod;
37     Mat mat(n);
38     for(int i = 1;i <= n;i++){
39         for(int j = 1;j <= n;j++){
40             cin >> mat.a[i][j];
41         }
42     }
43     cout << mat.det(n);
44 }
```

线性基

常用来解决子集异或一类题目

应用：

- 查询一个数是否可以被一堆数异或出来 $O(\log)$
- 查询一堆数可以异或出来的 最大值 / 最小值 $O(\log)$
- 查询一堆数可以异或出来的不同的数中 第k小值 / 第k大值 $O(\log^2)$ 预处理, $O(\log)$ 查询
- 查询能异或出的不同数的个数 (若 0 能被异或出来: 2^{cnt} , 否则: $2^{cnt} - 1$)
- 每个能被异或出来的数, 其方案数均为 2^{n-cnt} 种。

```
1 struct LB { // Linear Basis
2     const int BASE = 63;
3     vector<long long> d, p;
4     int cnt, flag; // flag 标记能否表示 0
5
6     LB() {
7         d.resize(BASE + 1);
8         p.resize(BASE + 1);
9         cnt = flag = 0;
10    }
11
12    bool insert(long long val) { // 插入一个数
13        for (int i = BASE - 1; i >= 0; i--) {
14            if (val >> i & 1) {
15                if (!d[i]) {
16                    d[i] = val;
17                    return true; // 插入成功
18                }
19                val ^= d[i];
20            }
21        }
22        flag = 1; // 特判能否异或出 0
23        return false; // 插入失败
24    }
25
26    bool check(long long val) { // 判断 val 是否能被异或得到
27        for (int i = BASE - 1; i >= 0; i--) {
28            if (val >> i & 1) {
29                if (!d[i]) return false;
30                val ^= d[i];
31            }
32        }
33        return true;
```

```

34     }
35
36     long long ask_max() { // 查询能异或出的最大值
37         long long res = 0;
38         for (int i = BASE - 1; i >= 0; i--) {
39             if ((res ^ d[i]) > res) res ^= d[i];
40         }
41         return res;
42     }
43
44     long long ask_min() { // 查询能异或出的最小值
45         if (flag) return 0; // 特判 0
46         for (int i = 0; i <= BASE - 1; i++) {
47             if (d[i]) return d[i];
48         }
49     }
50
51     void rebuild() { // 查询第k小、第k大、能异或出的数的个数前独立预处理 (第1小为最小)
52         // cnt = 0; // 如过要多次rebuild, 每次都要给cnt清空
53         for (int i = BASE - 1; i >= 0; i--) {
54             for (int j = i - 1; j >= 0; j--) {
55                 if (d[i] & (1ll << j)) d[i] ^= d[j];
56             }
57         }
58         for (int i = 0; i <= BASE - 1; i++) {
59             if (d[i]) p[cnt++] = d[i];
60         }
61     }
62
63     long long kth_min(long long k) { // 查询能被异或得到的第 k 小值, 如不存在则返回 -1
64         if (flag) k--; // 特判 0, 如果不需要 0, 直接删去
65         if (!k) return 0;
66         long long res = 0;
67         if (k >= (1ll << cnt)) return -1;
68         for (int i = BASE - 1; i >= 0; i--) {
69             if (k & (1ll << i)) res ^= p[i];
70         }
71         return res;
72     }
73
74     long long kth_max(long long k) { // 第k大 = 第(total - k + 1)小
75         long long total = (1ll << cnt) - 1;
76         if (flag) total++; // 如果能表示0, 总数+1
77         if (k > total) return -1; // 检查k是否在有效范围内
78         return kth_min(total - k + 1);
79     }
80
81     void Merge(const LB &b) { // 合并两个线性基
82         for (int i = BASE - 1; i >= 0; i--) {
83             if (b.d[i]) {
84                 insert(b.d[i]);
85             }

```

```
86 |         }
87 |     }
88 | };
```

概率论

一般情况下，解决概率问题需要顺序循环，而解决期望问题使用逆序循环

期望

离散型随机变量：设离散型随机变量 X 的概率分布为 $p_i = P\{X = x_i\}$ ，若和式

$$\sum x_i p_i$$

绝对收敛，则称其值为 X 的期望，记作 EX 。

连续型随机变量：设连续型随机变量 X 的密度函数为 $f(x)$ 。若积分

$$\int_{\mathbb{R}} x f(x) dx$$

绝对收敛，则称其值为 X 的期望，记作 EX 。

期望的性质

线性性：若随机变量 X, Y 的期望存在，则

- 对任意实数 a, b ，有 $E(aX + b) = a \cdot EX + b$
- $E(X + Y) = EX + EY$

随机变量乘积的期望：若随机变量 X, Y 的期望存在且[互相独立](#)，则有

$$E(XY) = EX \times EY$$

通常把终止状态作为初值，把起始状态作为目标，倒着进行计算。因为在很多情况下，起始状态是唯一的，而终止状态很多。根据数学期望的定义，若我们正着计算，则还需求出从起始状态到每个终止状态的概率，与F值相乘求和才能得到答案，增加了难度，也很容易出错

[P4316 绿豆蛙的归宿 - 洛谷](#)

给定 n 个点 m 条边的有向无环图，起点为 1，终点为 n ，每条边都有一个长度 w_i ，并且保证从起点出发能到达所有的点，所有的点也能到达终点。

从起点出发，每次等概率地选择一条边离开。求起点到终点走过的路径期望长度。保留两位小数。

$F[n] = 0$ ，我们的目标是求出 $F[1]$ ，从终点出发，在反图执行拓扑排序，在拓扑排序的过程中顺便计算 $F[x]$ 即可

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  const int N = 200005;
4  int n,m;
5  int h[N],e[N],ne[N],w[N],idx;
6  int in[N],out[N];
7  double ans[N];
8
9  void add(int a,int b,int c){
10     w[idx] = c,e[idx] = b,ne[idx] = h[a],h[a] = idx++;
11 }
12
13 void uuz(int beg){
14     ans[beg] = 0;
15     queue<int>q;
16     q.push(beg);
17     while(q.size()){
18         auto t = q.front();q.pop();
19         for(int i = h[t];~i;i = ne[i]){
20             int k = e[i];
21             ans[k] += (ans[t] + w[i])/in[k];
22             out[k]--;
23             if(out[k] == 0) q.push(k);
24         }
25     }
26 }
27
28 void sol(){
29     memset(h,-1,sizeof h);
30     cin >> n >> m;
31     for(int i = 1;i <= m;i++){
32         int a,b,c;cin >> a >> b >> c;
33         add(b,a,c);//反向建边
34         in[a]++;out[a]++;
35     }
36     uuz(n);
37     cout << fixed << setprecision(2) << ans[1];
```

```

38 }
39
40 int main() {
41     while(T--){ sol(); }
42 }

```

4855. 骰子 (三) - AcWing题库

给定一个 n 面骰，投出每个面的概率相等。计算每个面都出现所需要投掷的期望次数。

递推: $dp_i = dp_{i-1} + \frac{n}{n-i+1}$

预处理调和级数: $dp_n = \sum_{i=1}^n \frac{n}{n-i+1} = n \sum_{k=1}^n \frac{1}{k} = nH_n$

```

1  #include <iostream>
2
3  const int N = 100005;
4  double h[N];
5
6  void init(){
7      for(int i = 1; i < N; i++){
8          h[i] = h[i-1] + 1.0/i;
9      }
10 }
11
12 void sol(){
13     int n; std::cin >> n;
14     printf("%.9lf\n", h[n]*n);
15 }
16
17 int main(){
18     init();
19     int t; std::cin >> t;
20     for(int i = 1; i <= t; i++){
21         printf("Case %d: ", i);
22         sol();
23     }
24 }

```

4849. 危险的迷宫 - AcWing题库

给定 n 扇门，第 i 扇门权值为 x_i ，如果 x_i 为正，进入后可在 x_i 分钟后逃离迷宫，如果 x_i 为负，会浪费 $|x_i|$ 的时间并回到起点。当位于起点时，每次都会随机选择一扇门进入，每扇门被选中的概率相同，计算逃离迷宫的期望时间。

设 sum_1, n_1 表示正整数之和，正整数个数。

设 sum_2, n_2 表示负数的绝对值之和，负数个数。

则期望 $E = \frac{sum_1}{n} + (\frac{sum_2}{n} + \frac{n_2}{n} E)$

化简得 $E = \frac{sum_1 + sum_2}{n - n_2}$

```

1  #include <iostream>
2  #include <algorithm>
3
4  const int N = 105;
5  int n;
6
7  void sol(){
8      std::cin >> n;
9      int sum = 0;
10     int n1 = 0;
11     for(int i = 1; i <= n; i++){
12         int x; std::cin >> x;
13         if(x >= 0) n1++;
14         sum += abs(x);
15     }
16     if(n1 == 0) {
17         std::cout << "inf\n";
18         return;
19     }
20     int gd = std::__gcd(sum, n1);
21     std::cout << sum/gd << '/' << n1/gd << '\n';
22 }
23
24 int main(){
25     int t; std::cin >> t;
26     for(int i = 1; i <= t; i++){
27         printf("Case %d: ", i);
28         sol();
29     }
30 }

```

方差

设随机变量 X 的期望 EX 存在，且期望 $E(X - EX)^2$ 也存在，则称上式的值为随机变量 X 的 **方差**，记作 DX 或 $Var(x)$ ，各个数据与均值的差的平方和。方差的算术平方根称为 **标准差**，记作 $\sigma(X) = \sqrt{DX}$ 。

方差的性质

若随机变量 X 的方差存在，则

- 对任意常数 a, b 都有 $D(aX + b) = a^2 \cdot DX$
- $DX = E(X^2) - (EX)^2$

多项式

快速傅里叶变换

快速傅里叶变换(Fast Fourier Transform, FFT) 支持在 $O(N\log N)$ 的时间复杂度内计算两个 N 次多项式的乘法。常用于加速卷积计算。

[快速傅里叶变换\(FFT\) bilibili](#)

注意观察能否将问题模型转换为卷积形式: $f(x) = \sum_{i+j=x} a_i b_j$

[P3803 【模板】多项式乘法 \(FFT\) - 洛谷](#)

给定一个 n 次多项式和 $F(x)$ 和一个 m 次多项式 $G(x)$ 。求 $F(x)$ 和 $G(x)$ 的乘积。

例如 $F(x) = 5x^2 + 1x + 4$, $G(x) = 2x^2 + 2x + 3$

则乘积 $H(x) = 10x^4 + 12x^3 + 25x^2 + 11x + 12$

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const long double PI = std::acos(-1.0);
5  void FFT(std::vector<std::complex<long double>>& a, bool invert){//倍增实现
6      int n = a.size();
7
8      for(int i = 1, j = 0; i < n; i++) { //位逆序置换
9          int bit = n >> 1;
10         for (; j & bit; bit >>= 1) j ^= bit;
11         j ^= bit;
12         if (i < j) std::swap(a[i], a[j]);
13     }
14
15     for(int len = 2; len <= n; len <= 1) {
16         long double ang = 2 * PI / len * (invert ? -1 : 1); //夹角 = 2π/len
17         std::complex<long double> wlen(cos1(ang), sin1(ang)); //欧拉公式 计算旋转因子
18
19         for (int i = 0; i < n; i += len) {
20             std::complex<long double> w(1.0);
21             for (int j = 0; j < len/2; j++) {
22                 std::complex<long double> u = a[i+j];
23                 std::complex<long double> v = a[i+j + len/2] * w;
24                 a[i+j] = u+v;
25                 a[i+j + len/2] = u-v;
26                 w *= wlen; //w=e^(2πi/len)
27             }
28         }
29     }
```

```

27     }
28 }
29 if(invert){//逆矩阵即原矩阵每一项取倒数，再除以变换的长度n
30     for(std::complex<long double> &x:a) x /= n;
31 }
32 }
33
34 std::vector<long long> operator*(std::vector<long long>&A,std::vector<long
long>&B){
35     std::vector<long long>C(n+m);
36     int n = A.size(), m = B.size();
37
38     int MAX_N = 1;
39     while (MAX_N < n + m) MAX_N <= 1;//将长度补成2的幂
40
41     std::vector<std::complex<long double>> a(MAX_N,0.0), b(MAX_N,0.0);
42     for(int i = 0; i < n; i++) a[i] = std::complex<long double>(A[i],0);
43     for(int i = 0; i < m; i++) b[i] = std::complex<long double>(B[i],0);
44
45     FFT(a, false);//FFT:系数表示法->点值表示法
46     FFT(b, false);
47     for(int i = 0; i < MAX_N; i++) a[i] *= b[i];//值相乘
48     FFT(a, true);//IFFT:点值表示法->系数表示法
49
50     for(int i = 0; i < n + m; i++) {
51         c[i] = (long long)(a[i].real() + 0.5);
52     }
53
54     while(c.size() >= 2 && c.back() == 0) c.pop_back();
55     return c;
56 }
57
58 void soviet(){
59     int n,m; std::cin >> n >> m;
60     std::vector<long long>a(n+1),b(m+1);//本题从高位 $x^n$ 到低位 $x^0$ 依次读入
61     for(int i = n;i >= 0;i--) { std::cin >> a[i]; }
62     for(int i = m;i >= 0;i--) { std::cin >> b[i]; }
63
64     auto c = a*b;
65     for(int i = c.size()-1;i >= 0;i--){
66         std::cout << c[i] << ' ';
67     }
68 }
69
70 int main() {
71     int M_T = 1; std::ios::sync_with_stdio(false); std::cin.tie(0);
72     // std::cin >> M_T;
73     while(M_T--){ soviet(); }
74     return 0;
75 }

```

给定长为 m 的字符串 s_1 和长为 n 的字符串 s_2 ，求 s_2 中匹配了多少次 s_1 ？其中符号 $*$ 可以匹配任意字符。

$$1 \leq m \leq n \leq 3 \times 10^5$$

平方防止正负抵消，分别乘上 $a[i]$ 、 $b[i]$ 适配通配符

$$\text{设 } dis(A, B) = \sum_{i=0}^{n-1} (A[i] - B[i])^2 A[i] B[i]$$

翻转 A ，使其下标之和为定值满足卷积形式，再推柿子

$$P(x) = \sum_{i+j=x} S(i)^3 B(j) + \sum_{i+j=x} S(i) B(j)^3 - 2 \sum_{i+j=x} S(i)^2 B(j)^2$$

若 $P(x)$ 为 0，则 $s_2[x - m + 1, x]$ 与 s_1 匹配

```

1  #include <bits/stdc++.h>
2
3  const long double PI = std::acos(-1.0);
4  void FFT(std::vector<std::complex<long double>>& a, bool invert) {
5      int n = a.size();
6
7      for(int i = 1, j = 0; i < n; i++) {
8          int bit = n >> 1;
9          for (; j & bit; bit >>= 1) j ^= bit;
10         j ^= bit;
11         if (i < j) std::swap(a[i], a[j]);
12     }
13
14     for(int len = 2; len <= n; len <<= 1) {
15         long double ang = 2 * PI / len * (invert ? -1 : 1);
16         std::complex<long double> wlen(cos1(ang), sin1(ang));
17         for (int i = 0; i < n; i += len) {
18             std::complex<long double> w(1.0);
19             for (int j = 0; j < len/2; j++) {
20                 std::complex<long double> u = a[i+j];
21                 std::complex<long double> v = a[i+j + len/2] * w;
22                 a[i+j] = u+v;
23                 a[i+j + len/2] = u-v;
24                 w *= wlen;
25             }
26         }
27     }
28     if(invert){
29         for(std::complex<long double> &x:a) x /= n;
30     }
31 }
32
33 std::vector<long long> operator * (const std::vector<long long>&A, const
std::vector<long long>&B){
34     int n = A.size(), m = B.size();

```

```

35
36     int MAX_N = 1;
37     while (MAX_N < n + m) MAX_N <= 1;
38
39     std::vector<std::complex<long double>> a(MAX_N,0.0), b(MAX_N,0.0);
40     for(int i = 0; i < n; i++) a[i] = std::complex<long double>(A[i],0);
41     for(int i = 0; i < m; i++) b[i] = std::complex<long double>(B[i],0);
42
43     FFT(a, false);
44     FFT(b, false);
45     for(int i = 0; i < MAX_N; i++) a[i] *= b[i];
46     FFT(a, true);
47
48     std::vector<long long> C(n+m);
49     for(int i = 0; i < n + m; i++) {
50         C[i] = (long long)(a[i].real() + 0.5);
51     }
52
53     while(C.size() >= 2 && C.back() == 0) C.pop_back();
54     return C;
55 }
56
57 int main(){
58     int m,n; std::cin >> m >> n;
59     std::string s1,s2; std::cin >> s1 >> s2;
60     std::reverse(s1.begin(),s1.end());
61
62     std::vector<long long> a(m),b(n);
63     for(int i = 0;i < m;i++) a[i] = (s1[i] == '*' ? 0 : s1[i]-'a'+1);
64     for(int i = 0;i < n;i++) b[i] = (s2[i] == '*' ? 0 : s2[i]-'a'+1);
65
66     auto A = a,B = b;
67     for(int i = 0;i < m;i++) A[i] = a[i]*a[i]*a[i];
68     auto p1 = A*B;
69
70     for(int i = 0;i < m;i++) A[i] = a[i];
71     for(int i = 0;i < n;i++) B[i] = b[i]*b[i]*b[i];
72     auto p2 = A*B;
73
74     for(int i = 0;i < m;i++) A[i] = a[i]*a[i];
75     for(int i = 0;i < n;i++) B[i] = b[i]*b[i];
76     auto p3 = A*B;
77
78     std::vector<int> ans;
79     for(int i = m-1;i < n;i++){
80         auto now = p1[i] + p2[i] - p3[i]*2;
81         if(now == 0) { ans.emplace_back(i-m+2); }
82     }
83
84     std::cout << ans.size() << '\n';
85     for(auto x:ans){
86         std::cout << x << ' ';

```

```
87     }
88 }
```

快速数论变换

快速数论变换 (fast number-theoretic transform, FNTT) 是FFT在数论基础上的实现。解决多项式乘法带模数的情况，而FFT用到浮点数运算容易出现精度问题。目前最常见的模数有998244353、1004535809、469762049等（三模NTT的常用模数，其原根均为3）。

将复数根替换为原根： $w_n^k = e^{-\frac{2k\pi}{n}i} \Rightarrow g_n^k = (G^{\frac{n-1}{n}})^k \bmod p$

[P3803 【模板】多项式乘法 \(FFT\) - 洛谷](#)

998244353的一个原根G=3，其逆元GI=332748118

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int mod = 998244353, G = 3, GI = 332748118;
5
6  long long qmi(long long a, long long b, long long p){
7      long long ans = 1;
8      while(b){
9          if(b&1) ans = ans*a%p;
10         b>>=1;
11         a = a*a%p;
12     }
13     return ans%p;
14 }
15
16 void NTT(std::vector<long long>& a, bool invert) {
17     int n = a.size();
18
19     for(int i = 1, j = 0; i < n; i++) {
20         int bit = n >> 1;
21         for (; j&bit; bit >>= 1) j ^= bit;
22         j ^= bit;
23         if (i < j) std::swap(a[i], a[j]);
24     }
25
26     for(int len = 2; len <= n; len <<= 1) {
27         long long wlen = qmi(invert ? G : GI, (mod-1)/len, mod);
28         for (int i = 0; i < n; i += len) {
29             long long w = 1;
30             for (int j = 0; j < len/2; j++) {
31                 long long u = a[i+j];
32                 long long v = a[i+j + len/2] * w % mod;
```

```

33         a[i+j] = (u+v) % mod;
34         a[i+j + len/2] = (u-v+mod) % mod;
35         w = w * wlen % mod;
36     }
37 }
38 }
39 if(invert){
40     long long inv_n = qmi(n,mod-2,mod);
41     for(auto &x:a) x = x * inv_n % mod;
42 }
43 }
44
45 std::vector<long long> operator * (std::vector<long long>a,std::vector<long
long>b){
46     int n = a.size(), m = b.size();
47
48     int MAX_N = 1;
49     while (MAX_N < n + m) MAX_N <= 1;
50     a.resize(MAX_N);
51     b.resize(MAX_N);
52
53     NTT(a, false);
54     NTT(b, false);
55     for(int i = 0; i < MAX_N; i++) a[i] = a[i] * b[i] % mod;
56     NTT(a, true);
57
58     while(a.size() >= 2 && a.back() == 0) a.pop_back();
59     return a;
60 }
61
62 void soviet(){
63     int n,m; std::cin >> n >> m;
64     vector<long long> a(n+1),b(m+1);
65     for(int i = n;i >= 0;i--) std::cin >> a[i];
66     for(int i = m;i >= 0;i--) std::cin >> b[i];
67     auto c = a * b;
68     for(int i = c.size()-1;i >= 0;i--) {
69         std::cout << c[i] << ' ';
70     }
71 }
72
73 int main() {
74     int M_T = 1; std::ios::sync_with_stdio(false); std::cin.tie(0);
75     // std::cin >> M_T;
76     while(M_T--){ soviet(); }
77 }

```

快速沃尔什变换

类似于FFT处理多项式乘法，快速沃尔什变换（Fast Walsh-Hadamard Transform, FWT）常用来处理按位运算的卷积计算。

$F(x) = \sum_{i \oplus j = x} A_i B_j$ ，其中 $\oplus$ 是位运算中的一种。

P4717 【模板】快速莫比乌斯/沃尔什变换 (FMT/FWT) - 洛谷

给定两个长度为 2^n 的序列 A, B ，设 $C_i = \sum_{j \oplus k = i} A_j \times B_k$
当 $\oplus$ 分别是 $|$ $\&$ $\wedge$ 时，求出 C ，对998244353取模。

```
1  #include <bits/stdc++.h>
2
3  const int mod = 998244353;
4
5  void FWT(std::vector<long long>&a, bool invert, char op){
6      int n = a.size();
7      for(int len = 2, k = 1; len <= n; len <= 1, k <= 1){
8          for(int i = 0; i < n; i += len){
9              for(int j = 0; j < k; j++){
10                 if(op == '|'){
11                     (a[i+j+k] += a[i+j]*(invert ? mod-1 : 1)) %= mod;
12                 }
13                 if(op == '&'){
14                     (a[i+j] += a[i+j+k]*(invert ? mod-1 : 1)) %= mod;
15                 }
16                 if(op == '^'){
17                     (a[i+j] += a[i+j+k]) %= mod;
18                     (a[i+j+k] = (a[i+j] - a[i+j+k]*2)%mod + mod) %= mod;
19                     (a[i+j] *= invert ? (mod+1)/2 : 1) %= mod; // 1/2的逆元
20                     (a[i+j+k] *= invert ? (mod+1)/2 : 1) %= mod;
21                 }
22             }
23         }
24     }
25 }
26
27 std::vector<long long> fwt (std::vector<long long> a, std::vector<long long>
b, char op){
28     int n = a.size();
29     FWT(a, false, op);
30     FWT(b, false, op);
31     for(int i = 0; i < n; i++) a[i] = (a[i] * b[i]) % mod;
32     FWT(a, true, op);
33     return a;
34 }
35
36 int main(){
37     int n; std::cin >> n; n = 1 << n;
```

```

38     std::vector<long long> a(n),b(n);
39     for(int i = 0;i < n;i++) std::cin >> a[i];
40     for(int i = 0;i < n;i++) std::cin >> b[i];
41
42     auto c1 = fwt(a,b,'|'),c2 = fwt(a,b,'&'),c3 = fwt(a,b,'^');
43
44     for(int i = 0;i < c1.size();i++) std::cout << c1[i] << ' ';
45     std::cout << '\n';
46     for(int i = 0;i < c2.size();i++) std::cout << c2[i] << ' ';
47     std::cout << '\n';
48     for(int i = 0;i < c3.size();i++) std::cout << c3[i] << ' ';
49     std::cout << '\n';
50 }

```

数值算法

数值积分

自适应辛普森算法

求定积分 $\int_l^r f(x)dx$, 即为 $f(x)$ 在区间 $[l,r]$ 上与 x 轴围成的面积 (其中 x 轴上方为正值, 下方为负值)

```

1  double f(double x){//f(x)
2
3  }
4
5  double simpson(double l, double r) {
6      double mid = (l + r) / 2;
7      return (r - l) * (f(l) + 4 * f(mid) + f(r)) / 6; // 辛普森公式
8  }
9
10 double asr(double l, double r, double eps, double ans, int step) {
11     double mid = (l + r) / 2;
12     double fl = simpson(l, mid), fr = simpson(mid, r);
13     if (abs(fl + fr - ans) <= 15 * eps && step < 0)
14         return fl + fr + (fl + fr - ans) / 15; // 足够相似的话就直接返回
15     return asr(l, mid, eps / 2, fl, step - 1) + asr(mid, r, eps / 2, fr, step -
16 1); // 否则分割成两段递归求解
17 }
18
19 double calc(double l, double r, double eps) {// calc(l,r,eps)
20     return asr(l, r, eps, simpson(l, r), 12);
21 }

```

试计算积分 $\int_l^r \frac{cx+d}{ax+b} dx$ 结果保留至小数点后6位。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  double a,b,c,d,l,r;
4
5  double f(double x){
6      return (c*x+d)/(a*x+b);
7  }
8
9  double simpson(double l, double r) {
10     double mid = (l + r) / 2;
11     return (r - l) * (f(l) + 4 * f(mid) + f(r)) / 6;
12 }
13
14 double asr(double l, double r, double eps, double ans, int step) {
15     double mid = (l + r) / 2;
16     double fl = simpson(l, mid), fr = simpson(mid, r);
17     if (abs(fl + fr - ans) <= 15 * eps && step < 0)
18         return fl + fr + (fl + fr - ans) / 15;
19     return asr(l, mid, eps / 2, fl, step - 1) + asr(mid, r, eps / 2, fr, step -
20 1);
21 }
22
23 double calc(double l, double r, double eps) {
24     return asr(l, r, eps, simpson(l, r), 12);
25 }
26
27 int main() {
28     cin >> a >> b >> c >> d >> l >> r;
29     cout << fixed << setprecision(6) << calc(l,r,1e-9);
30 }

```

高斯消元

高斯消元解线性方程组

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,n}x_n &= b_1 \\ a_{2,1}x_1 + a_{2,2}x_2 + \cdots + a_{2,n}x_n &= b_2 \\ \cdots &\cdots \\ a_{m,1}x_1 + a_{m,2}x_2 + \cdots + a_{m,n}x_n &= b_m \end{cases}$$

运用初等行变换，把增广矩阵，变为阶梯型矩阵。最后再把阶梯型矩阵从下到上回代到第一层即可得到方程的解。

时间复杂度 $O(N^3)$

枚举每一列c

找到当前列绝对值最大的一行，放到上面

将该行该列第一个数变成1

再将下面其他所有行该列变成0

```
1 //https://www.acwing.com/problem/content/885/
2 #include <bits/stdc++.h>
3 using namespace std;
4
5 const double eps = 1e-9;
6 int gauss(vector<vector<double>>&a) { //a[m+1][n+2]; 1_idx
7     int m = a.size()-1, n = a[0].size()-2;
8     int c, r; //col列 row行
9
10    for (c = 1, r = 1; c <= n; c++) {
11        int tr = r; //找到当前这一列，绝对值最大的所在行
12        for (int i = r+1; i <= m; i++) {
13            if (fabs(a[i][c]) > fabs(a[tr][c])) { tr = i; }
14        }
15        if (fabs(a[tr][c]) < eps) continue; //全都是0直接跳过;
16
17        swap(a[r], a[tr]); //把当前这一行换到第最上面(第r行)
18        for (int j = n+1; j >= c; j--) { a[r][j] /= a[r][c]; } //把当前该行该列第一个数变
成1(从后往前系数倒着算)
19
20        for (int i = r+1; i <= m; i++) { //把当前列下面所有系数变为0
21            if (fabs(a[i][c]) > eps) { //已经是0则可以跳过
22                for (int j = n+1; j >= c; j--) { //每一行从后往前，系数-=行首系数*第c行同一
列系数
23                    a[i][j] -= a[i][c]*a[r][j];
24                }
25            }
26        }
27        r++;
28    }
29
30    if (r < n+1) { //r<n+1说明已经解出来的x不足以解出原方程
31        for (int i = r; i <= m; i++) {
32            if (fabs(a[i][n+1]) > eps) { //左边=0，右边!=0，无解
33                return 0;
34            }
35        }
36        return 2; //0=0说明有无穷解
37    }
38
39    for (int i = n; i >= 1; i--) { //从下往上回代，解出原方程
40        for (int j = i+1; j <= n; j++) {
41            a[i][n+1] -= a[j][n+1]*a[i][j];
42        }
43    }
```

```

43     }
44     return 1;
45 }
46
47 int main() {
48     int n, m; cin >> n; m = n;
49
50     vector<vector<double>> a(m+1, vector<double>(n+2));
51     for (int i = 1; i <= m; i++) {
52         for (int j = 1; j <= n+1; j++) {
53             cin >> a[i][j];
54         }
55     }
56     int t = gauss(a);
57     if (t == 0) cout << "No solution";
58     if (t == 2) cout << "Infinite group solutions";
59     if (t == 1) {
60         for (int i = 1; i <= n; i++) {
61             printf("%.21f\n", a[i][n+1]);
62         }
63     }
64 }

```

高斯消元解异或方程组

$$\begin{cases} a_{1,1}x_1 \oplus a_{1,2}x_2 \oplus \cdots \oplus a_{1,n}x_n & = b_1 \\ a_{2,1}x_1 \oplus a_{2,2}x_2 \oplus \cdots \oplus a_{2,n}x_n & = b_2 \\ \cdots & \cdots \\ a_{m,1}x_1 \oplus a_{m,2}x_2 \oplus \cdots \oplus a_{m,n}x_n & = b_m \end{cases}$$

在消元的时候使用「异或消元」而非「加减消元」，且不需要进行乘除改变系数（因为系数均为0和1）

异或可以看做不进位的加法

```

1 //https://www.acwing.com/problem/content/description/886/
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5
6 int gauss_xor(vector<vector<int>>&a){//a[m+1][n+2]; 1_idx
7     int m = a.size()-1, n = a[0].size()-2;
8     int c, r;
9     for(c = 1, r = 1; c <= n; c++){
10         int tr = r;
11         for(int i = r; i <= m; i++){
12             if(a[i][c]) {tr = i; break;}
13         }
14         if(a[tr][c] == 0) continue;
15
16         swap(a[r], a[tr]);
17

```

```

18         for(int i = r+1; i <= m; i++){
19             if(a[i][c]){
20                 for(int j = n+1; j >= c; j--){
21                     a[i][j] ^= a[r][j];
22                 }
23             }
24         }
25         r++;
26     }
27
28     if(r < n+1){
29         for(int i = r; i <= m; i++){
30             if(a[i][n+1]) return 0;
31         }
32         return 2;
33     }
34
35     for(int i = n; i >= 1; i--){
36         for(int j = i+1; j <= n; j++){
37             if(a[i][j]) a[i][n+1] ^= a[j][n+1];
38         }
39     }
40     return 1;
41 }
42
43 int main(){
44     int n,m; cin >> n; m = n;
45     vector<vector<int>>>a(m+1,vector<int>(n+2));
46     for(int i = 1; i <= m; i++){
47         for(int j = 1; j <= n+1; j++){
48             cin >> a[i][j];
49         }
50     }
51
52     int t = gauss_xor(a);
53     if(t == 1){
54         for(int i = 1; i <= n; i++){
55             cout << a[i][n+1] << '\n';
56         }
57     }
58     if(t == 2) cout << "Multiple sets of solutions";
59     if(t == 0) cout << "No solution";
60 }

```

插值

插值是一种通过已知的、离散的数据点推算一定范围内的新数据点的方法。插值法常用于函数拟合中。

拉格朗日插值

$$f(x) = \sum_{i=1}^n (y_i \cdot \prod_{j \neq i} \frac{x - x_j}{x_i - x_j})$$

P4781 【模板】拉格朗日插值 - 洛谷 (luogu.com.cn)

对于 n 个点 (x_i, y_i) , 如果满足 $\forall i \neq j, x_i \neq x_j$, 那么经过这 n 个点可以唯一地确定一个 $n - 1$ 次多项式 $y = f(x)$ 。

现在, 给定这样 n 个点, 请你确定这个 $n - 1$ 次多项式, 并求出 $f(k) \bmod 998244353$ 的值。

```
1 //O(N^2)实现
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5 const int mod = 998244353;
6
7 long long qmi(long long a, long long b, long long p){
8     long long ans = 1;
9     while(b){
10         if(b&1) ans = ans*a%p;
11         b>>=1;
12         a = a*a%p;
13     }
14     return ans%p;
15 }
16
17 long long lagrange_interpolation(vector<pair<long long, long long>> &v, int k)
18 { //1_idx
19     long long ans = 0;
20     for(int i = 1; i < v.size(); i++){
21         auto &[x1, y1] = v[i];
22         long long s1 = y1, s2 = 1;
23         for(int j = 1; j < v.size(); j++){
24             if(i == j) continue;
25             auto &[x2, y2] = v[j];
26             s1 = s1 * (k - x2) % mod;
27             s2 = s2 * (x1 - x2) % mod;
28         }
29         ans += s1 * qmi(s2, mod - 2, mod) % mod;
30         ans = (ans + mod) % mod;
31     }
32     return ans;
33 }
34
35 int main(){
```

```

35     int n,k; cin >> n >> k;
36     vector<pair<long long,long long>>v(n+1);
37     for(int i = 1;i <= n;i++){
38         cin >> v[i].first >> v[i].second;
39     }
40     cout << lagrange_interpolation(v,k) << '\n';
41 }

```

博弈论

[博弈论简介 - OI Wiki \(oi-wiki.org\)](https://oi-wiki.org/)

Nim游戏

n 堆物品，每堆有 a_i 个，两个玩家轮流取走任意一堆的任意一个物品(不能不取)，取走最后一个物品的人获胜

博弈图和状态

如果将每个状态视为一个节点，再从每个状态向它的后继状态连边，我们就可以得到一个博弈状态图。

例如，如果节点 (i, j, k) 表示局面为 i, j, k 时的状态，则我们可以画出下面的博弈图

定义 必胜状态 为 先手必胜的状态，必败状态 为 先手必败的状态。

通过推理，我们可以得出下面三条定理：

- 定理 1：没有后继状态的状态是必败状态。
- 定理 2：一个状态是必胜状态当且仅当存在至少一个必败状态为它的后继状态。
- 定理 3：一个状态是必败状态当且仅当它的所有后继状态均为必胜状态。

对于定理 1，如果游戏进行不下去了，那么这个玩家就输掉了游戏。

对于定理 2，如果该状态至少有一个后继状态为必败状态，那么玩家可以通过操作到该必败状态；此时对手的状态为必败状态——对手必定是失败的，而相反地，自己就获得了胜利。

对于定理 3，如果不存在一个后继状态为必败状态，那么无论如何，玩家只能操作到必胜状态；此时对手的状态为必胜状态——对手必定是胜利的，自己就输掉了游戏。

如果博弈图是一个有向无环图，则通过这三个定理，我们可以在绘出博弈图的情况下用 $O(N+M)$ 的时间（其中 N 为状态种数， M 为边数）得出每个状态是必胜状态还是必败状态。

Nim和

通过绘画博弈图，我们可以在 $O(\prod_{i=1}^n a_i)$ 的时间里求出该局面是否先手必赢。

但是，这样的时间复杂度实在太高。有没有什么巧妙而快速的方法呢？

定义 Nim 和 $= a_1 \oplus a_2 \oplus \dots \oplus a_n$ 。

当且仅当 Nim和为 0 时，该状态为必败状态；否则该状态为必胜状态。

```

1 //https://www.acwing.com/problem/content/893/
2 #include <iostream>
3 using namespace std;
4
5 int main(){
6     int ans = 0;
7     int n;cin >> n;
8     for(int i = 1;i <= n;i++){
9         int x;cin >> x;
10        ans^=x;
11    }
12    if(ans) cout << "Yes";
13    else cout << "No";
14 }

```

SG函数

有向图游戏是一个经典的博弈游戏——实际上，大部分的公平组合游戏都可以转换为有向图游戏。在一个有向无环图中，只有一个起点，上面有一个棋子，两个玩家轮流沿着有向边推动棋子，不能走的玩家判负。

定义 mex 函数的值为不属于集合 S 中的最小非负整数，即：

$$\text{mex}(S) = \min\{x\} \quad (x \notin S, x \in \mathbb{N})$$

例如 $\text{mex}(\{0, 2, 4\}) = 1$, $\text{mex}(\{1, 2\}) = 0$ 。

对于状态 x 和它的所有 k 个后继状态 $y_1, y_2, \dots, y_k$ ，定义 SG 函数：

$$\text{SG}(x) = \text{mex}\{\text{SG}(y_1), \text{SG}(y_2), \dots, \text{SG}(y_k)\}$$

而对于由 n 个有向图游戏组成的组合游戏，设它们的起点分别为 $s_1, s_2, \dots, s_n$ ，则有定理：**当且仅当 $\text{SG}(s_1) \oplus \text{SG}(s_2) \oplus \dots \oplus \text{SG}(s_n) \neq 0$ 时，这个游戏是先手必胜的。同时，这是这一个组合游戏的游戏状态 x 的 SG 值。**

这一定理被称作 **Sprague-Grundy 定理**(Sprague-Grundy Theorem), 简称 SG 定理，适用于任何公平的两人游戏，它常被用于决定游戏的输赢结果。

计算给定状态的 Grundy 值的步骤一般包括：

- 获取从此状态所有可能的转换；
- 每个转换都可以导致 **一系列独立的博弈**（退化情况下只有一个）。计算每个独立博弈的 Grundy 值并对它们进行 **异或求和**。
- 在为每个转换计算了 Grundy 值之后，状态的值是这些数字的 mex。
- 如果该值为零，则当前状态为输，否则为赢。

将 Nim 游戏转换为有向图游戏

我们可以将一个有 x 个物品的堆视为节点 x ，则当且仅当 $y < x$ 时，节点 x 可以到达 y 。那么，由 n 个堆组成的 Nim 游戏，就可以视为 n 个有向图游戏了。根据上面的推论，可以得出 $SG(x) = x$ 。再根据 SG 定理，就可以得出 Nim 和的结论了。

893. 集合-Nim游戏 - AcWing题库

给定 n 堆石子和一个由 m 个不同正整数构成的数字集合 S 。两位玩家轮流操作，每次可以从任意一堆石子中拿取石子，每次拿取的石子数量必须包含于集合 S ，判断先手是否必胜？

$1 \leq n, m \leq 100, 1 \leq s_i, h_i \leq 10000$

```
1  #include <iostream>
2  #include <cstring>
3  #include <algorithm>
4  #include <set>
5  using namespace std;
6  const int N = 105, M = 10004;
7  int n, m;
8  int f[M], s[N]; // f 储存所有情况的 sg 值, s 存可供选择的集合
9
10 int sg(int x){
11     if(f[x] != -1) return f[x]; // 如果当前数的 sg 已经确定, 直接返回即可
12     set<int> st;
13     for(int i = 0; i < m; i++){
14         if(x >= s[i]) st.insert(sg(x-s[i])); // set 存当前能到达状态的所有 sg 值
15     }
16     for(int i = 0; i < m; i++){
17         if(st.find(i) == st.end()) {
18             return f[x] = i; // f[x] = 不属于集合 set 的最小非负整数
19         }
20     }
21 }
22
23 int main(){
24     memset(f, -1, sizeof f);
25     cin >> m;
26     for(int i = 0; i < m; i++) cin >> s[i];
27
28     cin >> n;
29     int ans = 0;
30     for(int i = 0; i < n; i++){
31         int x; cin >> x;
32         ans ^= sg(x);
33     }
34     if(ans) cout << "Yes";
35     else cout << "No";
36 }
```

给定n堆石子，每堆石子由w[i]颗白石和b[i]颗蓝石。两人轮流进行以下操作之一：

- 选择一堆白石 $w \geq 1$ 的石子，向选择的石子中加入w颗蓝石，然后移除1颗白石。
- 选择一堆蓝石 $b \geq 2$ 的石子，移除k颗蓝石子，其中 $1 \leq k \leq \lfloor \frac{b}{2} \rfloor$ 。

最后无法操作的输掉比赛。

```
1  #include <iostream>
2  #include <set>
3  #include <cstring>
4  using namespace std;
5  const int N = 100005;
6  int w[N],b[N];
7  int f[55][1505];
8
9  int sg(int x,int y){
10     if(f[x][y] != -1) return f[x][y];
11     set<int>se;
12     if(x >= 1) se.insert(sg(x-1,y+x));
13     if(y >= 2) {
14         for(int i = 1;i*2 <= y;i++){
15             se.insert(sg(x,y-i));
16         }
17     }
18     for(int i = 0;;i++){
19         if(se.find(i) == se.end()){
20             return f[x][y] = i;
21         }
22     }
23 }
24
25 int main(){
26     memset(f,-1,sizeof f);
27     int n;cin >> n;
28     for(int i = 1;i <= n;i++) cin >> w[i];
29     for(int i = 1;i <= n;i++) cin >> b[i];
30
31     int ans = 0;
32     for(int i = 1;i <= n;i++){
33         ans ^= sg(w[i],b[i]);
34     }
35     if(ans) cout << "First";
36     else cout << "Second";
37 }
```

计算几何

距离

距离	二维计算	多维计算
欧氏距离	$ AB = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$	三维: $ AB = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$
曼哈顿距离	$d(A, B) = x_1 - x_2 + y_1 - y_2 $	n维: $d(A, B) = \sum_{i=1}^n x_i - y_i $
切比雪夫距离	$d(A, B) = \max(x_1 - x_2 , y_1 - y_2)$	n维: $d(A, B) = \max\{ x_i - y_i \} (i \in [1, n])$

曼哈顿距离与切比雪夫距离转换

曼哈顿->切比雪夫: $(x, y) \Rightarrow (x + y, x - y)$, 如果要避免负数, 可以在 $x - y$ 基础上加一个偏移量。

切比雪夫->曼哈顿: $(x, y) \Rightarrow (\frac{x+y}{2}, \frac{x-y}{2})$, 一般为了避免小数, 我们可以横纵坐标同时乘2(即转换时不除以2), 最后答案除以2。

曼哈顿坐标系是通过切比雪夫坐标系旋转 45° 后, 再缩小到原来的一半得到的。碰到求切比雪夫距离或曼哈顿距离的题目时, 我们往往可以相互转化来求解。两种距离在不同的题目中有不同的优缺点, 应该灵活运用。

[P5098 [USACO04OPEN](#)] [Cave Cows 3 - 洛谷 \(luogu.com.cn\)](#)

求平面内任意一点到其它点的最大距离

直接曼哈顿转切比雪夫坐标, 答案就是max(当前的点的 x 和 x的极值做差的绝对值, 当前的点的 y 和 y 的极值做差的绝对值的)

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  #include <tuple>
5  using namespace std;
6  const int N = 100005;
7  int n;
8  vector<long long>dx, dy;
9
10 struct node{
11     int x, y;
12 }p[N];
13
14 int main(){
15     cin >> n;
16     for(int i = 1; i <= n; i++){
17         auto &[x, y] = p[i]; cin >> x >> y;
18         tie(x, y) = pair{x+y, x-y};
19         dx.emplace_back(x); //也可以用4个变量分别维护x和y的极大值和极小值, O(N)
20         dy.emplace_back(y);
21     }
```

```

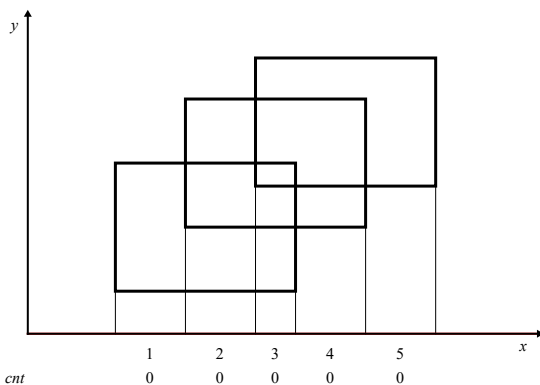
22     sort(dx.begin(),dx.end());
23     sort(dy.begin(),dy.end());
24     long long ans = 0;
25     for(int i = 1;i <= n;i++){
26         auto [x,y] = p[i];
27         long long nx = max(abs(dx.back()-x),abs(dx[0]-x));
28         long long ny = max(abs(dy.back()-y),abs(dy[0]-y));
29         long long now = max(nx,ny);
30         ans = max(ans,now);
31     }
32     cout << ans;
33 }

```

扫描线

扫描线的思路就是数据结构维护一维，暴力模拟另一维。

面积并



[P5490 【模板】扫描线 & 矩形面积并 - 洛谷 \(luogu.com.cn\)](https://www.luogu.com.cn/problem/P5490)

给每一个矩形的上下边进行标记，下面的边标记为 1，上面的边标记为 -1。每遇到一个水平边时，让这条边（在横轴投影区间）的权值加上这条边的标记。当前的宽度就是整个数轴上权值大于 0 的区间总长度，总面积即为 $\sum (\text{线段长度} * \text{扫过的高度})$

```

1  线段树维护的是区间段(左闭右开)而不是离散点
2  点:    1      3      5      7
3  区间:  [1,3) [3,5) [5,7)
4  索引:   1      2      3
5  modify当处理矩形边[1,5]时,覆盖的区间段索引 = [hs[1],hs[5]-1] = [1,2]
6  pushup计算节点p对应区间段时,节点p的左右端点[1,2],对应区间[v[1],v[2+1)) = [1,5)

```

```

1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  const int N = 200005;
6  int n;
7  std::vector<int>hs(1,-2e9);
8
9  struct line{//从下往上扫描
10     int l,r,h;//记录每条线其左右区间[l,r]和所在高度h
11     int w;//w=1为下边,w=-1为上边
12     bool operator < (const line &e2)const{
13         if(h == e2.h) return w > e2.w;
14         return h < e2.h;
15     }
16 };
17
18 struct ST{
19     int l,r;
20     int cnt,len;//cnt:被覆盖次数,len:覆盖长度
21 }t[N<<2];
22
23 void build(int p,int l,int r){
24     t[p] = {l,r};
25     if(l == r) {
26         return;
27     }
28     int mid = l + r >> 1;
29     build(p<<1,l,mid);build(p<<1|1,mid+1,r);
30 }
31
32 void pushup(int p){
33     if(t[p].cnt){//完全覆盖,取整个区间长度
34         t[p].len = hs[t[p].r+1] - hs[t[p].l];
35     }
36     else{
37         if(t[p].l == t[p].r) t[p].len = 0; //叶子节点且未覆盖
38         else t[p].len = t[p<<1].len + t[p<<1|1].len; //合并子节点
39     }
40 }
41
42 void modify(int p,int l,int r,int x){
43     if(l <= t[p].l && r >= t[p].r){
44         t[p].cnt += x;
45         pushup(p);//更新当前节点,标记永久化(利用覆盖计数的特性)
46         return;
47     }//不需要pushdown下传标记
48     int mid = t[p].l + t[p].r >> 1;
49     if(l <= mid) modify(p<<1,l,r,x);
50     if(r > mid) modify(p<<1|1,l,r,x);
51     pushup(p);//更新当前节点
52 }

```

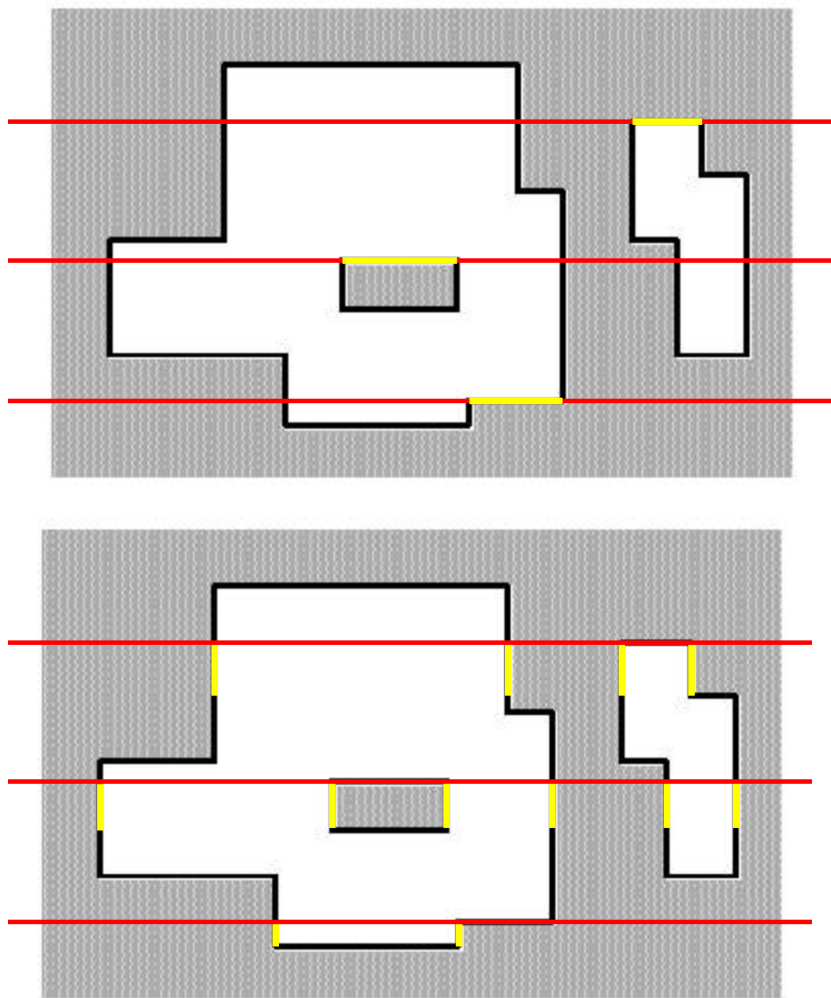
```

53
54 int main(){
55     std::cin >> n;
56     std::vector<line>e(1);
57     for(int i = 1;i <= n;i++){//读入坐标,并转化为线
58         int x1,y1,x2,y2;std::cin >> x1 >> y1 >> x2 >> y2;
59         e.push_back({x1,x2,y1,1});
60         e.push_back({x1,x2,y2,-1});
61         hs.push_back(x1);
62         hs.push_back(x2);
63     }
64     //离散化
65     std::sort(e.begin()+1,e.end());
66     std::sort(hs.begin()+1,hs.end());
67     hs.erase(std::unique(hs.begin()+1,hs.end()),hs.end());
68     int m = hs.size()-1;
69     for(int i = 1;i <= n << 1;i++){
70         e[i].l = std::lower_bound(hs.begin()+1,hs.end(),e[i].l) - hs.begin();
71         e[i].r = std::lower_bound(hs.begin()+1,hs.end(),e[i].r) - hs.begin();
72     }
73
74     build(1,1,m-1);//建树范围为区间段个数,m个点对应m-1个区间(开m个也没关系)
75
76     long long ans = 0;
77     for(int i = 1;i <= n << 1;i++){//最后一条边不用累加上答案,但仍需要modify处理,否则多
测会导致线段树数据残留
78         auto &[l,r,h,w] = e[i];
79         modify(1,l,r-1,w);//左闭右开区间
80         if(i < n << 1) ans += (long long)t[1].len * (e[i+1].h - e[i].h);//面积+=
当前全局覆盖长度*上下高度差
81     }
82     std::cout << ans;
83 }

```

周长并

[P1856 \[IOI 1998\]\[USACO5.5\] 矩形周长 Picture - 洛谷 \(luogu.com.cn\)](#)



横边的总长度 = $\sum | \text{当前截得的总长度} - \text{上次截得的总长度} |$

竖边的总长度 = $\sum (2 \times \text{当前线段条数} \times \text{扫过的高度})$

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  const int N = 10004;
6  int n;
7  std::vector<int> hs(1, -2e9);
8
9  struct line{
10     int l, r, h, w;
11     bool operator < (const line &e2) const{
12         if(h == e2.h) return w > e2.w; //高度相同先扫描底边, 否则会多算这条边
13         return h < e2.h;
14     }
15 };
16
17 struct ST{
18     int l, r;
19     int len, cnt;
20     int c; //c: 区间线段条数

```

```

21     bool lc,rc;//lc、rc:左右端点是否被覆盖,辅助计算c
22 }t[N<<2];
23
24 void build(int p,int l,int r){
25     t[p] = {l,r};
26     if(l == r){
27         return;
28     }
29     int mid = l + r >> 1;
30     build(p<<1,l,mid);build(p<<1|1,mid+1,r);
31 }
32
33 void pushup(int p){
34     if(t[p].cnt){//当前区间完全被覆盖
35         t[p].len = hs[t[p].r+1] - hs[t[p].l];
36         t[p].lc = t[p].rc = t[p].c = 1;
37     }
38     else{
39         if(t[p].l == t[p].r) {//当前区间未被覆盖且为子区间
40             t[p].len = 0;
41             t[p].lc = t[p].rc = t[p].c = 0;
42         }
43         else{//合并两个子区间
44             t[p].len = t[p<<1].len + t[p<<1|1].len;
45             t[p].lc = t[p<<1].lc;
46             t[p].rc = t[p<<1|1].rc;
47             t[p].c = t[p<<1].c + t[p<<1|1].c;
48             if(t[p<<1].rc && t[p<<1|1].lc) t[p].c--;
49         }
50     }
51 }
52
53 void modify(int p,int l,int r,int x){
54     if(l <= t[p].l && r >= t[p].r){
55         t[p].cnt += x;
56         pushup(p);
57         return;
58     }
59     int mid = t[p].l + t[p].r >> 1;
60     if(l <= mid) modify(p<<1,l,r,x);
61     if(r > mid) modify(p<<1|1,l,r,x);
62     pushup(p);
63 }
64
65 int main(){
66     std::cin >> n;
67     std::vector<line>e(1);
68     for(int i = 1;i <= n;i++){
69         int x1,y1,x2,y2;std::cin >> x1 >> y1 >> x2 >> y2;
70         e.push_back({x1,x2,y1,1});
71         e.push_back({x1,x2,y2,-1});
72         hs.push_back(x1);

```

```

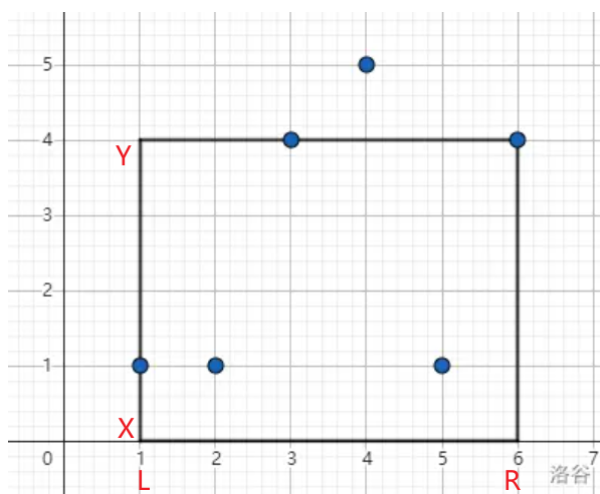
73     hs.push_back(x2);
74 }
75
76 std::sort(e.begin()+1,e.end());
77 std::sort(hs.begin()+1,hs.end());
78 hs.erase(std::unique(hs.begin()+1,hs.end()),hs.end());
79 for(int i = 1;i <= n << 1;i++){
80     e[i].l = std::lower_bound(hs.begin()+1,hs.end(),e[i].l) - hs.begin();
81     e[i].r = std::lower_bound(hs.begin()+1,hs.end(),e[i].r) - hs.begin();
82 }
83 int m = hs.size() - 1;
84
85 build(1,1,m-1);
86
87 long long ans = 0;
88 long long last = 0;
89 for(int i = 1;i <= n << 1;i++){
90     auto &[l,r,h,w] = e[i];
91     modify(1,l,r-1,w);
92     ans += std::abs(t[1].len - last); //计算横边(i <= 2*n)
93     last = t[1].len;
94     if(i < n << 1) ans += 2 * t[1].c * (e[i+1].h - e[i].h); //计算竖边(i <=
2*n-1)
95 }
96 std::cout << ans;
97 }

```

二维偏序

[U245713 【模板】二维数点 - 洛谷 \(luogu.com.cn\)](#)

给定一个长度为 n 的序列 $a[]$ ，然后进行 m 次询问：求区间 $[l,r]$ 内大小在 $[x,y]$ 范围内的数的个数。



把 $a[i]$ 看作平面上的一个点 $(i, a[i])$ ，询问即为求矩形内包含多少个点。

我们将询问离线，用一条线从左往右扫描，每遇到一个点就把它加入一个集合，表示当前所有扫过的点，用权值树状数组维护集合。每个询问的答案即为 $1 \sim r$ 内位于 $[x, y]$ 的数的个数减去 $1 \sim l-1$ 内位于 $[x, y]$ 的数的个数。

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  const int N = 1000006;
6
7  struct ask{
8      int l,r,x,y;
9  };
10
11 struct node{
12     int x,y,w,id;//w = 1表示当前为r,w = -1表示当前为l-1
13 };
14 std::vector<node>e[N];
15
16 int t[N*3],siz;//对离散化后的a[],x[],y[]建立权值树状数组,至少开三倍大小
17 void add(int i,int x){
18     while(i <= siz){
19         t[i] += x;
20         i += i&-i;
21     }
22 }
23 int query(int i){
24     int res = 0;
25     while(i){
26         res += t[i];
27         i -= i&-i;
28     }
29     return res;
30 }
31
32 std::vector<int> sol(std::vector<int>&a,std::vector<ask>&q){
33     std::vector<int>ans(q.size());
34     std::vector<int>hs(1,-2e9);
35
36     for(int i = 1;i < a.size();i++){
37         hs.push_back(a[i]);
38     }
39
40     for(int i = 1;i < q.size();i++){
41         auto &[l,r,x,y] = q[i];
42         hs.push_back(x);
43         hs.push_back(y);
44     }
45
46     std::sort(hs.begin()+1,hs.end());
```

```

47     hs.erase(std::unique(hs.begin()+1,hs.end()));
48
49     siz = hs.size();
50
51     for(int i = 1;i < a.size();i++){
52         a[i] = std::lower_bound(hs.begin()+1,hs.end(),a[i]) - hs.begin();
53     }
54     for(int i = 1;i < q.size();i++){
55         auto &[l,r,x,y] = q[i];
56         x = std::lower_bound(hs.begin()+1,hs.end(),x) - hs.begin();
57         y = std::lower_bound(hs.begin()+1,hs.end(),y) - hs.begin();
58         e[l-1].push_back({x,y,-1,i});
59         e[r].push_back({x,y,1,i});
60     }
61
62     for(int i = 1;i < a.size();i++){
63         add(a[i],1);
64         for(auto &[x,y,w,id]:e[i]){
65             ans[id] += w*(query(y) - query(x-1)); //query(y) - query(x-1)即为当前
            位于[x,y]的点的个数
66         }
67     }
68     return ans;
69 }
70
71 int main(){
72     std::ios::sync_with_stdio(false);std::cin.tie(0);
73     int n,m; std::cin >> n >> m;
74     std::vector<int> a(n+1);
75     std::vector<ask> q(m+1);
76     for(int i = 1;i <= n;i++){
77         std::cin >> a[i];
78     }
79
80     for(int i = 1;i <= m;i++){
81         auto &[l,r,x,y] = q[i];
82         std::cin >> l >> r >> x >> y;
83     }
84
85     auto ans = sol(a,q);
86
87     for(int i = 1;i <= m;i++){
88         std::cout << ans[i] << '\n';
89     }
90 }

```

类似的，若给定二维平面上的点集 $[x,y]$ ，然后给出若干个询问求矩形 $[x_1,y_1,x_2,y_2]$ 内有多少个点，则需要分别将所有横纵坐标都离散化。例题[P2163 [SHOI2007 园丁的烦恼](https://www.luogu.com.cn/problem/P2163) - 洛谷(luogu.com.cn)]

```

2  #include <vector>
3  #include <algorithm>
4
5  const int N = 500005;
6
7  struct Point{
8      int x,y;
9  };
10
11 struct Ask{
12     int x1,y1,x2,y2;
13 };
14
15 struct Node{
16     int x,y,w,id;
17 };
18 std::vector<Node>e[N*3]; //x,x1,x2
19
20 int t[N*3],siz; //y,y1,y1
21 void add(int i,int x){
22     while(i <= siz){
23         t[i] += x;
24         i += i&-i;
25     }
26 }
27 int query(int i){
28     int res = 0;
29     while(i){
30         res += t[i];
31         i -= i&-i;
32     }
33     return res;
34 }
35
36 std::vector<int> sol(std::vector<Point>&p, std::vector<Ask>&ask){
37     std::vector<int> ans(ask.size());
38     std::vector<int> dx(1,-2e9), dy(1,-2e9);
39     for(int i = 1; i < p.size(); i++){
40         auto &[x,y] = p[i];
41         dx.push_back(x);
42         dy.push_back(y);
43     }
44
45     for(int i = 1; i < ask.size(); i++){
46         auto &[x1,y1,x2,y2] = ask[i];
47         dx.push_back(x1); dx.push_back(x2);
48         dy.push_back(y1); dy.push_back(y2);
49     }
50
51     std::sort(dx.begin()+1, dx.end());
52     dx.erase(std::unique(dx.begin()+1, dx.end()), dx.end());

```

```

52     std::sort(dy.begin()+1,dy.end());
    dy.erase(std::unique(dy.begin()+1,dy.end()),dy.end());
53     siz = dy.size()-1;
54
55     for(int i = 1;i < p.size();i++){
56         auto &[x,y] = p[i];
57         x = std::lower_bound(dx.begin()+1,dx.end(),x) - dx.begin();
58         y = std::lower_bound(dy.begin()+1,dy.end(),y) - dy.begin();
59         e[x].push_back({y,y,0,0});
60     }
61
62     for(int i = 1;i < ask.size();i++){
63         auto &[x1,y1,x2,y2] = ask[i];
64         x1 = std::lower_bound(dx.begin()+1,dx.end(),x1) - dx.begin();
65         x2 = std::lower_bound(dx.begin()+1,dx.end(),x2) - dx.begin();
66         y1 = std::lower_bound(dy.begin()+1,dy.end(),y1) - dy.begin();
67         y2 = std::lower_bound(dy.begin()+1,dy.end(),y2) - dy.begin();
68         e[x1-1].push_back({y1,y2,-1,i});
69         e[x2].push_back({y1,y2,1,i});
70     }
71
72     //这里没有分开将点和询问分开存储,而是用e[i].w为0或者1/-1,表示当前是点还是询问。因为是先
    push点,再push询问,保证了处理询问前会先处理与询问相关的点
73     for(int i = 1;i < dx.size();i++){
74         for(auto &[y1,y2,w,id]:e[i]){
75             if(w == 0){ add(y1,1); }//扫描到点,则将点加入集合
76             else{ ans[id] += w*(query(y2) - query(y1-1)); }//否则处理询问
77         }
78     }
79     return ans;
80 }
81
82 int main(){
83     int n,m; std::cin >> n >> m;
84     std::vector<Point>p(n+1);
85     std::vector<Ask>ask(m+1);
86     for(int i = 1;i <= n;i++){
87         auto &[x,y] = p[i];
88         std::cin >> x >> y;
89     }
90
91     for(int i = 1;i <= m;i++){
92         auto &[x1,y1,x2,y2] = ask[i];
93         std::cin >> x1 >> y1 >> x2 >> y2;
94     }
95
96     auto ans = sol(p,ask);
97
98     for(int i = 1;i <= m;i++){
99         std::cout << ans[i] << '\n';
100     }
101 }

```

平面几何

点线封装

点、向量 Pointp1(x1,y1);		
p1 +- p2	向量加减	$\vec{p_1} + \vec{p_2} = (x_1 + x_2, y_1 + y_2)$
p * / x	向量乘除标量	$v \vec{p_1} = (x_1 \times v, y_1 \times v)$
dot(p1,p2)	点积	$\vec{p_1} \cdot \vec{p_2} = x_1x_2 + y_1y_2 = \vec{p_1} \vec{p_2} \cos\theta$
cross(p1,p2)	叉积	$\vec{p_1} \times \vec{p_2} = x_1y_2 - x_2y_1 = \vec{p_1} \vec{p_2} \sin\theta\vec{n}$
length(p)	向量的模	$ \vec{p_1} = \sqrt{x_1^2 + y_1^2}$
normalize(p)	单位向量	$\hat{p_1} = \frac{\vec{p_1}}{ \vec{p_1} }$
distance(p1,p2)	两点之间距离（欧几里得距离）	
rotate(p)	向量逆时针旋转90度	
sgn(p)	判断向量所在象限 (一/四:1, 二/三:-1)	
pointInPolygon(p,vec<Point>)	判断点是否在多边形内部或多边形边上(射线法,0_idx)	多边形的点需要按顺/逆时针顺序排列，仅支持简单多边形(不自交)
area(vec<Point>)	返回简单多边形(可凸可凹)面积的两倍，顶点需要按顺时针或逆时针排序	$\sum \text{cross}(v[i], v[(i+1)\%n])$
hp(vec<Point> &v)	返回给定点集的最小凸包顶点（逆时针顺序，不含共线中间点）	Andrew算法求凸包 $O(n\log n)$

线 Line1({x1,y1},{x2,y2});		
length(l)	线段长度	
distancePS(p,l)	点到线段Segment距离	
pointOnSegment(p,l)	判断点是否在线段上	
distanceSS(l1,l2)	两线段最短距离	

线 <code>Line1({x1,y1},{x2,y2});</code>		
<code>segmentIntersection(l1,l2)</code>	线段相交判定 返回值: <code>tuple(类型,交点1,交点2)</code>	0:不相交 1:严格相交(交点在两线段内部) 2:重叠(返回重叠部分的两个端点) 3:端点相交(交点为线段端点)
<code>segmentInPolygon(l,vec<Point>)</code>	线段是否在多边形内部	(需满足:端点在内且不与任何边严格相交)。多变形的点需要按顺/逆时针顺序排列
<code>distancePL(p,l)</code>	点到直线距离	
<code>parallel(l1,l2)</code>	判断两直线是否平行或共线(叉积为0)	1:共线 2:平行 0:相交
<code>lineIntersection(l1,l2)</code>	求两直线交点(需要保证不平行/共线)	
<code>pointOnLineLeft(p,l)</code>	点是否在直线 ($\vec{ab}$) 左侧(叉积为正)	以直线的方向向量 $\vec{ab}$ 方向为基准
<code>hp(vec<Line>)</code>	返回凸多边形顶点集(半平面交S&I算法,并按逆时针排列)	传入的参数不需要预先排列

```

1  template<typename T>
2  struct Point {
3      T x,y;
4      Point (const T &_x = 0,const T &_y = 0) : x(_x),y(_y){}
5
6      template<typename U> //自动类型转换
7      operator Point<U>() {
8          return Point<U>(U(x), U(y));
9      }
10
11     Point &operator+=(const Point &p) & {
12         x += p.x; y += p.y;
13         return *this;
14     }
15     Point &operator-=(const Point &p) & {
16         x -= p.x; y -= p.y;
17         return *this;
18     }
19     Point &operator*=(const T &v) & {
20         x *= v; y *= v;
21         return *this;
22     }
23     Point &operator/=(const T &v) & {

```

```

24     x /= v; y /= v;
25     return *this;
26 }
27 Point operator-() const { return Point(-x, -y); }
28 friend Point operator+(Point a, const Point &b) { return a += b; }
29 friend Point operator-(Point a, const Point &b) { return a -= b; }
30 friend Point operator*(Point a, const T &b) { return a *= b; }
31 friend Point operator/(Point a, const T &b) { return a /= b; }
32 friend Point operator*(const T &a, Point b) { return b *= a; }
33 friend bool operator==(const Point &a, const Point &b) { return a.x == b.x
&& a.y == b.y; }
34 friend bool operator!=(const Point &a, const Point &b) { return !(a == b); }
35
36 friend std::istream &operator>>(std::istream &is, Point &p) {
37     return is >> p.x >> p.y;
38 }
39 friend std::ostream &operator<<(std::ostream &os, const Point &p) {
40     return os << "(" << p.x << ", " << p.y << ")";
41 }
42 };
43
44 template<typename T> //点乘
45 T dot(const Point<T> &p1, const Point<T> &p2) {
46     return p1.x*p2.x + p1.y*p2.y;
47 }
48
49 template<typename T> //叉乘
50 T cross(const Point<T> &p1, const Point<T> &p2) {
51     return p1.x*p2.y - p1.y*p2.x;
52 }
53
54 template<typename T> //向量的模
55 double length(const Point<T> &p) {
56     return std::sqrt(dot(p, p));
57 }
58
59 template<typename T> //单位向量
60 Point<T> normalize(const Point<T> &p) {
61     return p/length(p);
62 }
63
64 template<typename T> //两点之间距离(欧几里得距离)
65 double distance(const Point<T> &p1, const Point<T> &p2) {
66     return length(p1-p2);
67 }
68
69 template<typename T> //向量逆时针旋转90度
70 Point<T> rotate(const Point<T> &p) {
71     return Point(-p.y, p.x);
72 }
73
74 template<typename T> // 判断向量所在象限(第一/四象限为1, 第二/三象限为-1)

```

```

75 int sgn(const Point<T> &a) {
76     return a.y > 0 || (a.y == 0 && a.x > 0) ? 1 : -1;
77 }
78
79 template<typename T> //返回简单多边形面积的两倍
80 T half_area(const std::vector<Point<T>> &v) {
81     T ans = 0;
82     for (int i = 0; i < v.size(); i++) {
83         ans += cross(v[i], v[(i + 1) % v.size()]);
84     }
85     return std::abs(ans);
86 }
87
88
89
90 template<typename T> // 返回点集的最小凸包顶点（逆时针顺序，不含共线中间点）
91 std::vector<Point<T>> hp(const std::vector<Point<T>> &v) {
92     auto p = v;
93     std::sort(p.begin(), p.end(), [&](const Point<T> &a, const Point<T> &b) {
94         return a.x < b.x || (a.x == b.x && a.y < b.y);
95     });
96     p.erase(std::unique(p.begin(), p.end()), p.end());
97     int n = p.size();
98     if (n <= 2) return p;
99     std::vector<Point<T>> hull;
100     for (int i = 0; i < n; ++i) { // 下凸壳
101         while (hull.size() >= 2 && cross(hull.back() - hull[hull.size()-2],
102 p[i] - hull.back()) <= 0) {
103             hull.pop_back();
104         }
105         hull.push_back(p[i]);
106     }
107
108     int lowerSize = hull.size();
109     for (int i = n - 2; i >= 0; --i) { // 上凸壳
110         while (hull.size() > lowerSize && cross(hull.back() -
111 hull[hull.size()-2], p[i] - hull.back()) <= 0) {
112             hull.pop_back();
113         }
114         hull.push_back(p[i]);
115     }
116     if (hull.size() > 1) hull.pop_back(); // 去掉最后一个重复起点
117     return hull;
118 }
119
120 template<typename T>
121 struct Line{
122     Point<T>a,b;
123     Line (const Point<T> &a = Point<T>(),const Point<T> &b = Point<T>()) :
124     a(_a),b(_b){}

```

```

124     template<typename U>
125     operator Line<U>(){
126         return Line<U>(Point<U>(a),Point<U>(b));
127     }
128 };
129
130 template<typename T> //线段长度
131 double length(const Line<T> &l){
132     return length(l.a-l.b);
133 }
134
135 template<typename T> //判断两直线是否平行(叉积为0)
136 int parallel(const Line<T> &l1,const Line<T> &l2){
137     if(cross(l1.b-l1.a,l2.b-l2.a) == 0){
138         if(distancePL(l2.b,l1) == 0) return 1; //共线
139         else return 2; //平行
140     }
141     return 0;
142 }
143
144 template<typename T> //点到直线距离
145 double distancePL(const Point<T> &p,const Line<T> &l){
146     return std::abs(cross(l.a-l.b,l.a-p)) / length(l);
147 }
148
149 template<typename T> //点到线段距离
150 double distancePS(const Point<T> &p, const Line<T> &l) {
151     if (dot(p - l.a, l.b - l.a) < 0) { return distance(p, l.a); }
152     if (dot(p - l.b, l.a - l.b) < 0) { return distance(p, l.b); }
153     return distancePL(p, l);
154 }
155
156 template<typename T> //点是否在直线左侧
157 bool pointOnLineLeft(const Point<T> &p, const Line<T> &l) {
158     return cross(l.b - l.a, p - l.a) > 0;
159 }
160
161 template<typename T> //求两直线交点(需确保不平行)
162 Point<T> lineIntersection(const Line<T> &l1, const Line<T> &l2) {
163     return l1.a + (l1.b - l1.a) * (cross(l2.b - l2.a, l1.a - l2.a) /
164     cross(l2.b - l2.a, l1.a - l1.b));
165 }
166
167 template<typename T> //判断点是否在线段上
168 bool pointOnSegment(const Point<T> &p, const Line<T> &l) {
169     return cross(p - l.a, l.b - l.a) == 0 && std::min(l.a.x, l.b.x) <= p.x &&
170     p.x <= std::max(l.a.x, l.b.x)
171     && std::min(l.a.y, l.b.y) <= p.y && p.y <= std::max(l.a.y, l.b.y);
172 }
173
174 template<typename T> //判断点是否在多边形内部(或多边形上) 0_idx,多边形点集需要按顺序排列
175 bool pointInPolygon(const Point<T> &a, const std::vector<Point<T>> &p) {

```

```

174     int n = p.size();
175     for (int i = 0; i < n; i++) {
176         if (pointOnSegment(a, Line(p[i], p[(i + 1) % n]))) {
177             return true;
178         }
179     }
180
181     int t = 0;
182     for (int i = 0; i < n; i++) {
183         auto u = p[i];
184         auto v = p[(i + 1) % n];
185         if (u.x < a.x && v.x >= a.x && pointOnLineLeft(a, Line(v, u))) {
186             t ^= 1;
187         }
188         if (u.x >= a.x && v.x < a.x && pointOnLineLeft(a, Line(u, v))) {
189             t ^= 1;
190         }
191     }
192     return t == 1;
193 }
194
195 /*
196 线段相交判定
197 返回值:tuple(类型,交点1,交点2)
198 类型:
199     0:不相交
200     1:严格相交(交点在两线段内部)
201     2:重叠(返回重叠部分的两个端点)
202     3:端点相交(交点为线段端点)
203 */
204 template<typename T>
205 std::tuple<int, Point<T>, Point<T>> segmentIntersection(const Line<T> &l1,
206 const Line<T> &l2) {
207     if (std::max(l1.a.x, l1.b.x) < std::min(l2.a.x, l2.b.x)) {
208         return {0, Point<T>(), Point<T>()};
209     }
210     if (std::min(l1.a.x, l1.b.x) > std::max(l2.a.x, l2.b.x)) {
211         return {0, Point<T>(), Point<T>()};
212     }
213     if (std::max(l1.a.y, l1.b.y) < std::min(l2.a.y, l2.b.y)) {
214         return {0, Point<T>(), Point<T>()};
215     }
216     if (std::min(l1.a.y, l1.b.y) > std::max(l2.a.y, l2.b.y)) {
217         return {0, Point<T>(), Point<T>()};
218     }
219     if (cross(l1.b - l1.a, l2.b - l2.a) == 0) {
220         if (cross(l1.b - l1.a, l2.a - l1.a) != 0) {
221             return {0, Point<T>(), Point<T>()};
222         }
223         else {
224             auto maxx1 = std::max(l1.a.x, l1.b.x);
225             auto minx1 = std::min(l1.a.x, l1.b.x);

```

```

225         auto maxy1 = std::max(l1.a.y, l1.b.y);
226         auto miny1 = std::min(l1.a.y, l1.b.y);
227         auto maxx2 = std::max(l2.a.x, l2.b.x);
228         auto minx2 = std::min(l2.a.x, l2.b.x);
229         auto maxy2 = std::max(l2.a.y, l2.b.y);
230         auto miny2 = std::min(l2.a.y, l2.b.y);
231         Point<T> p1(std::max(minx1, minx2), std::max(miny1, miny2));
232         Point<T> p2(std::min(maxx1, maxx2), std::min(maxy1, maxy2));
233         if (!pointOnSegment(p1, l1)) { std::swap(p1.y, p2.y); }
234         if (p1 == p2) return {3, p1, p2};
235         else return {2, p1, p2};
236     }
237 }
238 auto cp1 = cross(l2.a - l1.a, l2.b - l1.a);
239 auto cp2 = cross(l2.a - l1.b, l2.b - l1.b);
240 auto cp3 = cross(l1.a - l2.a, l1.b - l2.a);
241 auto cp4 = cross(l1.a - l2.b, l1.b - l2.b);
242
243 if ((cp1 > 0 && cp2 > 0) || (cp1 < 0 && cp2 < 0) || (cp3 > 0 && cp4 > 0)
|| (cp3 < 0 && cp4 < 0)) {
244     return {0, Point<T>(), Point<T>()};
245 }
246
247 Point p = lineIntersection(l1, l2);
248 if (cp1 != 0 && cp2 != 0 && cp3 != 0 && cp4 != 0) return {1, p, p};
249 else return {3, p, p};
250 }
251
252 template<typename T> //返回两线段最短距离
253 double distanceSS(const Line<T> &l1, const Line<T> &l2) {
254     if (std::get<0>(segmentIntersection(l1, l2)) != 0) {
255         return 0.0;
256     }
257     return std::min({distancePS(l1.a, l2), distancePS(l1.b, l2),
distancePS(l2.a, l1), distancePS(l2.b, l1)});
258 }
259
260 template<typename T> //线段是否在多边形内部(需满足:端点在内且不与任何边严格相交),多边形点
集需要按顺序排列
261 bool segmentInPolygon(const Line<T> &l, const std::vector<Point<T>> &p) {
262     int n = p.size();
263     if (!pointInPolygon(l.a, p)) { return false; }
264     if (!pointInPolygon(l.b, p)) { return false; }
265     for (int i = 0; i < n; i++) {
266         auto u = p[i];
267         auto v = p[(i + 1) % n];
268         auto w = p[(i + 2) % n];
269         auto [t, p1, p2] = segmentIntersection(l, Line(u, v));
270
271         if (t == 1) { return false; }
272         if (t == 0) { continue; }
273         if (t == 2) {

```

```

274         if (pointOnSegment(v, l) && v != l.a && v != l.b) {
275             if (cross(v - u, w - v) > 0) {
276                 return false;
277             }
278         }
279     }
280     else {
281         if (p1 != u && p1 != v) {
282             if (pointOnLineLeft(l.a, Line(v,u)) || pointOnLineLeft(l.b,
Line(v,u))) {
283                 return false;
284             }
285         }
286         else if (p1 == v) {
287             if (l.a == v) {
288                 if (pointOnLineLeft(u, l)) {
289                     if (pointOnLineLeft(w, l) && pointOnLineLeft(w,
Line(u, v))) {
290                         return false;
291                     }
292                 }
293                 else {
294                     if (pointOnLineLeft(w, l) || pointOnLineLeft(w,
Line(u, v))) {
295                         return false;
296                     }
297                 }
298             }
299             else if (l.b == v) {
300                 if (pointOnLineLeft(u, Line(l.b, l.a))) {
301                     if (pointOnLineLeft(w, Line(l.b, l.a)) &&
pointOnLineLeft(w, Line(u,v))) {
302                         return false;
303                     }
304                 }
305                 else {
306                     if (pointOnLineLeft(w, Line(l.b,l.a)) ||
pointOnLineLeft(w, Line(u,v))) {
307                         return false;
308                     }
309                 }
310             }
311             else {
312                 if (pointOnLineLeft(u, l)) {
313                     if (pointOnLineLeft(w, Line(l.b,l.a)) ||
pointOnLineLeft(w, Line(u,v))) {
314                         return false;
315                     }
316                 }
317                 else {
318                     if (pointOnLineLeft(w, l) || pointOnLineLeft(w,
Line(u, v))) {

```

```

319         return false;
320     }
321 }
322 }
323 }
324 }
325 }
326 return true;
327 }
328
329 template<typename T> //返回凸多边形顶点集 0_idx
330 std::vector<Point<T>> hp(std::vector<Line<T>> lines) {
331     std::sort(lines.begin(), lines.end(), [&](auto l1, auto l2) {
332         auto d1 = l1.b - l1.a;
333         auto d2 = l2.b - l2.a;
334
335         if (sgn(d1) != sgn(d2)) {
336             return sgn(d1) == 1;
337         }
338
339         return cross(d1, d2) > 0;
340     });
341
342     std::deque<Line<T>> ls;
343     std::deque<Point<T>> ps;
344     for (auto l : lines) {
345         if (ls.empty()) {
346             ls.push_back(l);
347             continue;
348         }
349
350         while (!ps.empty() && !pointOnLineLeft(ps.back(), l)) {
351             ps.pop_back();
352             ls.pop_back();
353         }
354
355         while (!ps.empty() && !pointOnLineLeft(ps[0], l)) {
356             ps.pop_front();
357             ls.pop_front();
358         }
359
360         if (cross(l.b - l.a, ls.back().b - ls.back().a) == 0) {
361             if (dot(l.b - l.a, ls.back().b - ls.back().a) > 0) {
362                 if (!pointOnLineLeft(ls.back().a, l)) {
363                     assert(ls.size() == 1);
364                     ls[0] = l;
365                 }
366                 continue;
367             }
368             return {};
369         }
370     }

```

```

371     ps.push_back(lineIntersection(ls.back(), l));
372     ls.push_back(l);
373 }
374
375 while (!ps.empty() && !pointOnLineLeft(ps.back(), ls[0])) {
376     ps.pop_back();
377     ls.pop_back();
378 }
379 if (ls.size() <= 2) { return {}; }
380
381 ps.push_back(lineIntersection(ls[0], ls.back()));
382
383 return std::vector(ps.begin(), ps.end());
384 }

```

其它

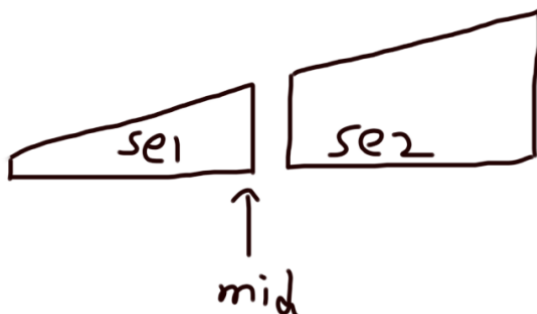
中位数

一般要求最大化中位数之类的题目，可以二分一个数 x ，若 $a[i] \geq x$ 则赋权值 $w[i]$ 为1，否则为-1，判断某一区间中位数是否大于等于 x ，即求该区间和是否大于0。

对顶堆

动态维护中位数，插入/删除 $O(\log N)$ ，查找 $O(1)$

se1的末尾元素即为中位数



```

1 //该实现常数较大,建议参考其它写法
2 multiset<int> se1, se2;
3
4 void balance() { //每次插入/删除后平衡两个集合
5     while (se1.size() > se2.size()) {
6         se2.insert(*se1.rbegin());

```

```

7         se1.erase(prev(se1.end()));
8     }
9     while(se1.size() < se2.size()){
10         se1.insert(*se2.begin());
11         se2.erase(se2.begin());
12     }
13 }
14
15 void add(int x){//插入元素
16     se1.insert(x);
17     balance();
18 }
19
20 void del(int x){//删除元素
21     if(x <= *se1.rbegin()) se1.erase(se1.find(x));
22     else se2.erase(se2.find(x));
23     balance();
24 }
25
26 int query(){//查找中位数
27     return *se1.rbegin();
28 }

```

例题: [Problem - K - Rainbow Subarray - Codeforces](#)

题意: 定义一个数组的连续子数组($a[l] \sim a[r]$)每一项满足 $a_{i+1} = a_i + 1$ 为一个彩虹子数组, 可以至多进行 k 次操作使任意 $a_i + 1$ 或 $a_i - 1$, 求能构造出的最长彩虹子数组

思路: 考虑 $a_{i+1} = a_i + 1$ 可以转化为 $a_{i+1} - (i + 1) = a_i - i$, 所以先预处理把所有的 a_i 减去自己下标, 问题就变成了: 把一个区间的所有数字全部变成一个相同的数字, 很容易知道, 最小代价就是把所有数变成中位数即可, 所以接下来就用一个双指针滑动窗口来维护, 求最大的区间长度, 这里动态维护滑动窗口的中位数用两个 multiset

```

1  #include <iostream>
2  #include <vector>
3  #include <set>
4  using namespace std;
5  using ll = long long;
6  int n;
7
8  multiset<int>se1, se2;
9  ll sum1, sum2;//sum1统计se1的和, sum2统计se2的和
10
11 void balance(){
12     while(se1.size() > se2.size()){
13         sum2 += *se1.rbegin();
14         sum1 -= *se1.rbegin();
15         se2.insert(*se1.rbegin());
16         se1.erase(prev(se1.end()));
17     }

```

```

18     while(se1.size() < se2.size()){
19         sum1 += *se2.begin();
20         sum2 -= *se2.begin();
21         se1.insert(*se2.begin());
22         se2.erase(se2.begin());
23     }
24 }
25
26 void add(int x){
27     se1.insert(x);
28     sum1 += x;
29     balance();
30 }
31
32 void del(int x){
33     if(x <= *se1.rbegin()) {se1.erase(se1.find(x)),sum1 -= x;}
34     else {se2.erase(se2.find(x)),sum2 -= x;}
35     balance();
36 }
37
38 ll query(){
39     return *se1.rbegin();
40 }
41
42 ll all(){//返回 使所有数变为中位数所需要的操作次数
43     return (se1.size()*query()-sum1) + (sum2-se2.size()*query());
44 }
45
46 void sol(){
47     se1.clear();se2.clear();
48     sum1 = sum2 = 0;
49     ll n,k;cin >> n >> k;
50     vector<ll>a(n+1);
51     for(int i = 1;i <= n;i++){
52         cin >> a[i];
53         a[i] -= i;
54     }
55     int ans = 1;
56     ll sum = 0;
57     for(int l = 0,r = 0;r <= n;){
58         while(sum <= k){
59             ans = max(ans,r-l+1);
60             r++;
61             if(r > n) break;
62             add(a[r]);
63             sum = all();
64         }
65         while(sum > k){
66             l++;
67             del(a[l]);
68             sum = all();
69         }

```

```

70     }
71     cout << ans-1 << endl;
72 }
73
74 int main(){
75     int tt;cin >> tt;
76     while(tt--){ sol(); }
77     return 0;
78 }

```

nth第k小数求中位数

时间复杂度 $O(N)$

[快排](#) 或 STL 函数 `nth_element()`

主席树

详见可持久化权值线段树

$O(\log N)$ 求区间第k小，不支持修改

均值不等式

$$\frac{a_1 + a_2 + \dots + a_n}{n} \geq \sqrt[n]{a_1 * a_2 * \dots * a_n}$$

$$\frac{2}{\frac{1}{a} + \frac{1}{b}} \leq \sqrt{ab} \leq \frac{a+b}{2} \leq \sqrt{\frac{a^2 + b^2}{2}}$$

约瑟夫环

n 个人标号 $0, 1, \dots, n-1$ 。逆时针站一圈，从 0 号开始，每一次从当前的人逆时针数 k 个，然后让这个人出局。问最后剩下的人是谁。

```

1 //O(N)
2 int josephus(int n, int k) {
3     int res = 0;
4     for (int i = 1; i <= n; ++i) res = (res + k) % i;
5     return res;
6 }

```

```

1 //O(KlogN)
2 int josephus(int n, int k) {
3     if (n == 1) return 0;
4     if (k == 1) return n - 1;
5     if (k > n) return (josephus(n - 1, k) + k) % n; // 线性算法
6     int res = josephus(n - n / k, k);
7     res -= n % k;
8     if (res < 0) res += n; // mod n
9     else res += res / (k - 1); // 还原位置
10    return res;
11 }

```

吉姆拉尔森日期公式

给定具体日期，推算当天是星期几。也可以求两个日期的天数之差等等。

```

1 const int d[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
2
3 bool isLeap(int y) { //判断当前年是否为闰年
4     return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
5 }
6
7 int daysInMonth(int y, int m) {
8     return d[m - 1] + (isLeap(y) && m == 2);
9 }
10
11 int getDay(int y, int m, int d) {
12     int ans = 0;
13     for (int i = 1970; i < y; i++) {
14         ans += 365 + isLeap(i);
15     }
16     for (int i = 1; i < m; i++) {
17         ans += daysInMonth(y, i);
18     }
19     ans += d;
20     //return ans; //返回今天是从1970.1.1起的第几天? (1970.1.1为第1天, 星期4)
21     return (ans + 2) % 7 + 1; //返回今天是星期几? [1,2,3,4,5,6,7]
22 }

```

二维和一维坐标转化

下标从0开始

```
n行m列

idx = i*m + j

(i,j) = (idx/m,idx%m)
```

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19

下标从1开始

```
n行m列

idx = (i-1)*m + j

(i,j) = ((idx+m-1)/m,(idx-1)%m + 1)
```

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20

多维动态数组

对于 $1 \leq n \times m \times k \leq 10^6$ ，需要创建数组 `a[n][m][k]`，但由于无法知道每一维具体大小，无法直接定义 `int a[N][N][N]`，且使用多层 `vector` 嵌套时开销太大。我们可以将多维小标映射到一维下标。

比静态数组慢，优于多层vector嵌套

```
1 // 以三维为例 0_idx
2 #include <bits/stdc++.h>
3
4 template<class T = int>
5 struct Darray {
6     std::vector<T> data;
```

```

7   std::vector<int> d;
8   Darray (const std::vector<int> dims) {
9       int tot = 1;
10      for (int i = 0; i < dims.size(); i++) {
11          tot *= dims[i];
12      }
13      data.resize(tot);
14      d.resize(dims.size());
15      d.back() = 1;
16      for (int i = (int)dims.size() - 2; i >= 0; i--){ //预处理维度数组
17          d[i] = d[i + 1] * dims[i + 1];
18      }
19  }
20
21  T &operator[] (const std::initializer_list<int> &v) {
22      int idx = 0;
23      int i = 0;
24      for (int x : v) {
25          idx += x * d[i++];
26      }
27      return data[idx];
28  }
29  };
30
31  int main() {
32      int n, m, k; std::cin >> n >> m >> k;
33      Darray<int> a({n, m, k});
34      for (int i0 = 0; i0 < n; i0++) {
35          for (int i1 = 0; i1 < m; i1++) {
36              for (int i2 = 0; i2 < k; i2++) {
37                  std::cin >> a[{i0, i1, i2}];
38                  std::cout << a[{i0, i1, i2}] << ' ';
39              }
40              std::cout << '\n';
41          }
42          std::cout << '\n';
43      }
44  }

```

如果维度固定，建议直接用静态数组，方便好写且快。

```

1   #include <bits/stdc++.h>
2
3   const int N = 1000006;
4   int a[N];
5   int n, m, k;
6
7   int idx(int i0, int i1, int i2) {

```

```
8     return i0 * m * k + i1 * k + i2;
9 }
10
11 int main() {
12     std::cin >> n >> m >> k;
13
14     for (int i0 = 0; i0 < n; i0++) {
15         for (int i1 = 0; i1 < m; i1++) {
16             for (int i2 = 0; i2 < k; i2++) {
17                 std::cin >> a[idx(i0, i1, i2)];
18                 std::cout << a[idx(i0, i1, i2)] << ' ';
19             }
20             std::cout << '\n';
21         }
22         std::cout << '\n';
23     }
24 }
```
